

Rubik's Cube Solver & Trainer

Liam Short 3005

Completeness of Solution (15 marks)	Pg No	Yes	No
Have all objectives, as set out in the Analysis section, been completed?			

NOTE – Even if you have completed all your objectives, the final mark for this section is based on the completion AND the suitability of the objectives.

Techniques Used (27 marks)	Pg no	Yes	No
Program commented to highlight programming techniques used.			
Appropriate naming conventions used.			

GROUP A : Have you successfully included the following programming techniques in your project? If Yes, input name of unit	Which Form? Pg No	Yes	No
Creating a complex data model in database (e.g. several interlinked tables)			x
Cross table parameterised SQL. <i>(Querying your database using more than one table).</i>	108	x	
Aggregate SQL Functions <i>(Use of MIN, MAX, SUM, AVG, etc. e.g. SELECT AVG(Salary) FROM Employee;)</i>			x
User generated DDL script. <i>(Creating your database using Data Definition Language).</i>	86	x	

Rubik's Cube Solver & Trainer

Hash tables, lists, stacks, queues, graphs, trees or structures of equivalent standard.			
Graph/Tree Traversal	120	x	
List Operations	75 List data structure	x	
Stack/Queue Operations	77 stack data structure	x	
Linked List Maintenance			x
File(s) organised for Direct Access			x
Hashing	118	x	
Complex Scientific/Mathematical/Robotics/Control/Business Model			
Advanced Matrix Operations			x
Recursive Algorithms	120	x	
Complex user-defined algorithms (e.g. optimisation, minimisation, scheduling, pattern matching) or equivalent difficulty.	119	x	
Merge Sort or similar efficient sort.			x
Complex user-defined use of Object-Oriented Programming (OOP) model.			
Dynamic generation of objects based on complex user-defined use of OOP model including:			
Classes		x	
Inheritance			x
Composition		x	

Rubik's Cube Solver & Trainer

Polymorphism			x
Interfaces			x
Complex Client-Server Model			x
Server-side scripting using request and response objects and server-side extensions for complex client-server model.			x
Calling parameterised Web service APIs and parsing JSON/XML to service a complex client-server model.			x

GROUP B : Have you successfully included the following programming techniques in your project? – If Yes, input name of unit	Which Form? Page No	Yes	No
Creating a simple data model in databases (e.g. 2 or 3 interlinked tables)	86	x	
Single table or non-parameterised SQL. <i>(Querying your database with no parameters using one table).</i>	107	x	
Multi-dimensional arrays, dictionaries, records	141	x	
Bubble Sort			x
Binary Search			x
Reading and writing to/from a Text File	73	x	
Files organised for sequential access			x
Simple scientific/mathematical/robotics/control/business model			
Simple user-defined algorithms (e.g. a range of mathematical/statistical calculation).	119	x	
Simple OOP Model			
Generation of objects based on simple OOP model including:			
Classes		x	
Inheritance			x

Rubik's Cube Solver & Trainer

Simple Client-Server Model			
Server-side scripting using request and response objects and server-side extensions for simple client-server model.			x
Calling Web service APIs and parsing JSON/XML to service a simple client-server model.			x

GROUP C : Have you successfully included the following programming techniques in your project? – If Yes, input name of unit	Which Form? Page No?	Yes	No
Single-dimensional array	65	x	
Linear Search.			x
Appropriate choice of simple data types		x	
Simple mathematical calculations (e.g. average)		x	
Single table database		x	
Non-SQL table access			x

Analysis

Background to Project

Leo started solving Rubik's cubes at the age of 13 in 2020 and immediately became obsessed with the international phenomenon of the 1980s. He quickly got to grasp the Beginner Method of solving the Rubik's cube and immediately wanted a greater challenge and to improve his speed.

People across the globe began competing for the fastest time just 2 years after the cube's global debut. The first ever Rubik's Cube World Championship was held in 1982 and hosted 20 speedcubers. Winning the competition and being the introductory world champion was Minh Thai setting a record of 22.95 seconds. This record would not be officially broken for over 20 years. In 1997 the CFOP Method was published by Jessica Fridrich. CFOP stands for Cross, First 2 layers, Orientation of last layer, Permutation of last layer (F2L, OLL, PLL).

CFOP is the method Leo wishes to learn to its fullest, learning all 119 different algorithms for the primary method. This is broken into 41 algorithms for F2L, 57 for OLL, and 21 for PLL. Leo knows most of the algorithms for "beginner CFOP" which is a simplified version of the full method that allows for fewer algorithms to be learnt at the cost of more moves. The main issue Leo faces is being able to practise the different permutations during a solve. The selling point of CFOP is that it provides an algorithm to apply to any permutation of the cube. For example, the 21 PLL algorithms provide a single algorithm that solves the cube from all possible states after OLL.

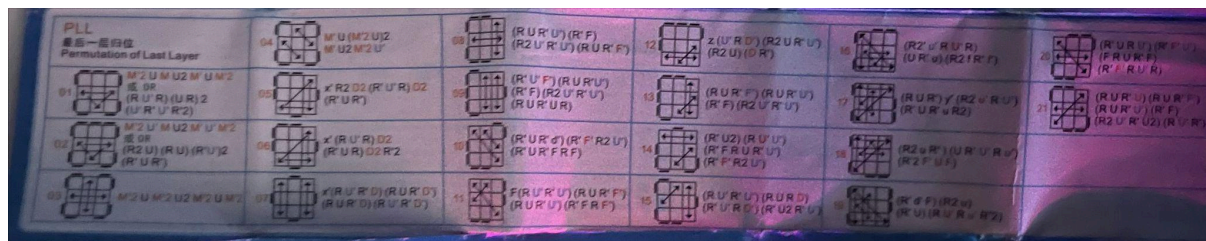
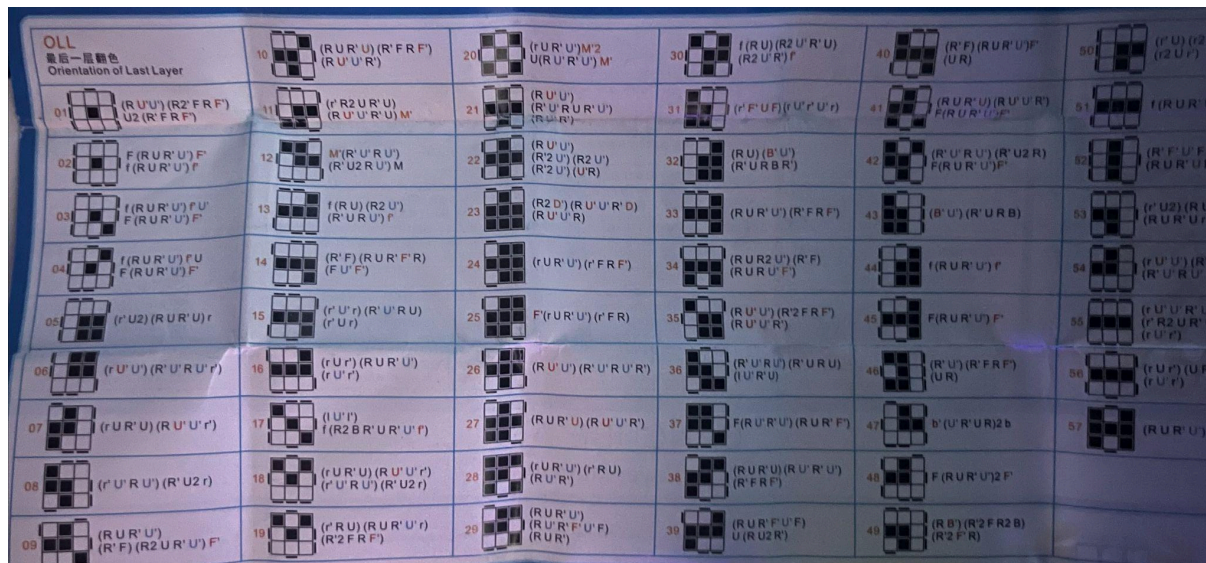
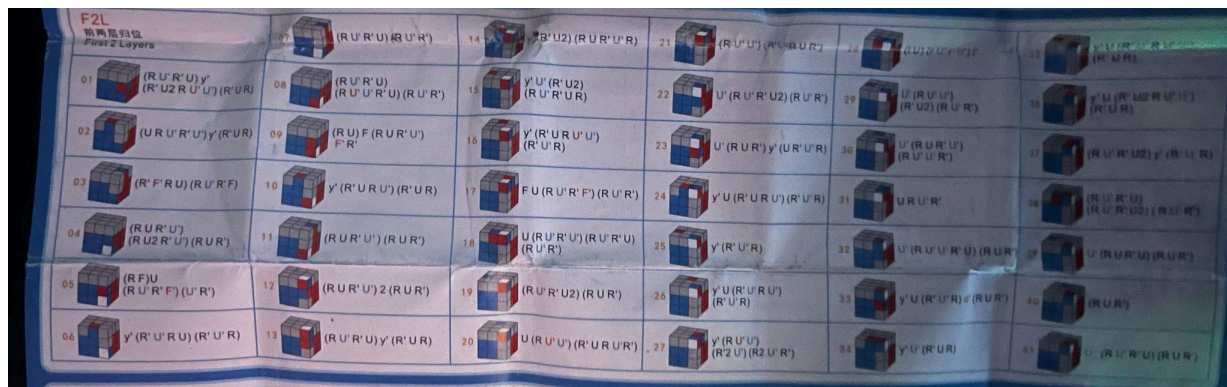
I also share a love for speedcubing like Leo, and I am in a similar position to his learning block. Creating a training application will help both me and Leo develop our speedcubing ability at an accelerated rate. My passion for cubing is what drew me towards this project and I believe that it will help me make this application the best it can be for any cuber.

Description of Current System

Leo currently uses a paper-based system for his learning. He locates all the algorithms required for CFOP and organises them into a cheat sheet. This sheet is then written out so he can read them on the go. He will then repeat the algorithm many times onto his Rubik's cube and then cover up the cheat sheet and attempt to complete the algorithm without his notes. After he has successfully completed this many times, he will mark the algorithm as complete. Once he has learnt one part of the algorithm sets (f2l, oll, pll), he will attempt that part of a solve to learn the different cases in which he needs to use the algorithm.

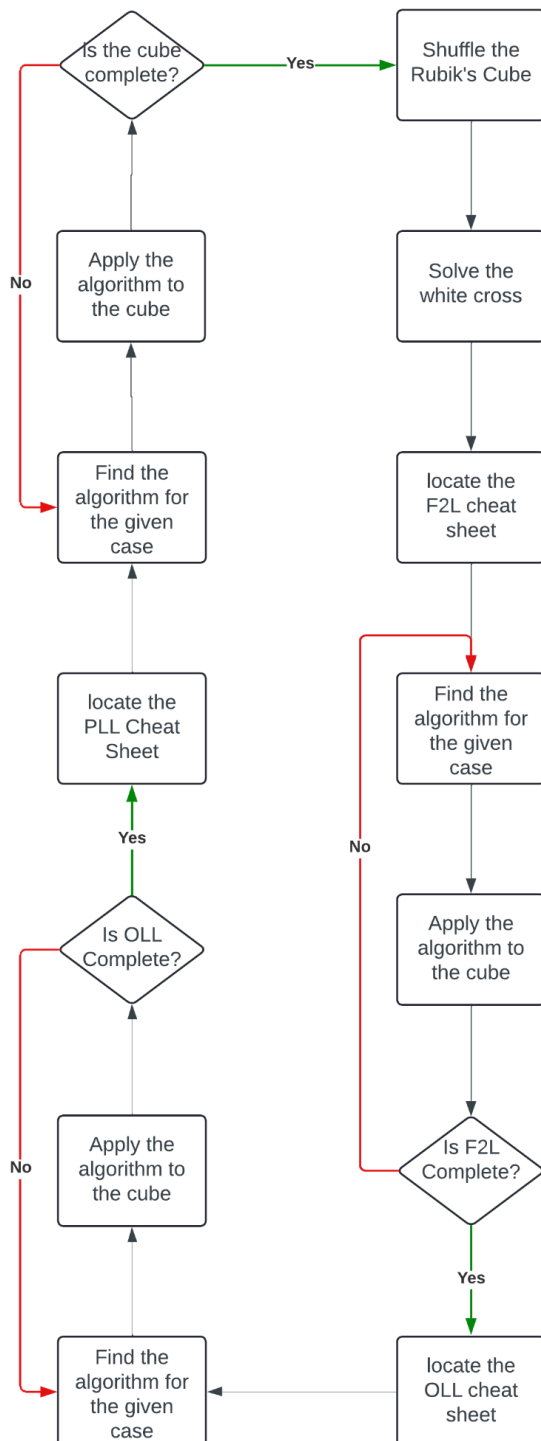
Evidence of Current System

Rubik's Cube Solver & Trainer



Rubik's Cube Solver & Trainer

Flowchart of Current System



Issues with Current System

There are many issues and inefficiencies with the current system Leo has implemented. Firstly, using a paper system means that data loss is common due to misplacing sheets and sheets being ruined over time. In addition, there is no way to determine whether he has been improving his algorithms since the sheets just tell him the algorithm. This also has issues

Rubik's Cube Solver & Trainer

with teaching him the different cases for each algorithm; the sheets tell him what the cube should look like and do not allow him to practice recognising the different permutations.

Reasons for a change of system

Leo has been trying intermittently for the past few years to improve his speed cubing ability with moderate success. He believes that the major problem inhibiting his success is his learning system. Leo would like a system that he can come back to at any time and have everything there as he left it rather than some of his notes that don't exhibit the different algorithms. He would also like a system that learns his weaknesses to tailor his learning experience to him.

Identification of the end user/supervisor

The primary user for this software will be Leo Thompson, Peter Symmonds College L6 student.

Prospective users

The secondary users of the software would be people attempting to understand how to solve a Rubik's cube; people learning new methods of solving a Rubik's cube; and people reinforcing their knowledge of algorithms for the Rubik's cube. Each download will be completely local so if a user adds an algorithm to the program, no other user across the world will be able to see that unless they are on the same system.

Evidence of Analysis

Interview

How would you like to enter the data of your Rubik's cube?

I would like to be able to paint on the colours on the face of the cube by pressing a colour then pressing a face or by clicking multiple times on the piece and it switches through each colour.

1) Are there any other data inputs you would need?

I want to be able to input an algorithm and it to help me practice

2) What algorithms would you like to see represented in the trainer?

I would like to have the beginner method as I want to teach my friend how to solve the cube as well as all of the 2-look CFOP. I would like the ability to add / change algorithms in the future as well

3) What information would you like to be stored?

I want to store my name and each algorithm

4) How would you like to store each algorithm?

A database that stores: the name of the algorithm, my name, store the algorithm in standard Rubik's notation, time taken to do each algorithm on average.

5) Would you like to be able to interact with the cube within the software?

Yes but I want to be able to press buttons to move the cube and not drag it with my mouse

Rubik's Cube Solver & Trainer

6) Should the solver solve in the fewest moves or be able to solve with different algorithms?

I don't want the fewest moves as that wouldn't teach me anything, so I want it to solve the cube with either the CFOP or the Beginner method and allow me to choose which one during runtime

7) Should the solver be able to give hints?

Yes it should give the first few moves of an algorithm if the user gets stuck

8) What happens if an invalid cube formation is entered?

I want it to tell me that an invalid cube is entered and to say whether there are more than 9 of each colour, the faces are on the wrong sides or if a corner is twisted and parity has occurred

9) How should the system train you? Should you enter the notation / do the algorithm in the software / tell the user how to get the cube to that formation and then time the user?

I want to enter the notation and then the program to check if I entered the correct one, I don't want it to time me

10) Do you need the software to connect to the internet – Would you like to be able to cube against other users?

No, I want the program to be fully functional offline

11) Would you like to save your current cube formation to the software?

Yes, only the person who has saved the cube should be able to access this.

12) How would you like the system to store the moves?

In the official Rubik's cube set notation

13) Should the system teach users how to read the moves?

Yes, only a small diagram of each notation and the move would be necessary

Observation Notes

My user is clearly very adamant about the use of Rubik's Cube Notation and clearly wants other users to have an understanding of this before using the software. He is highly focused on the training aspect of the software since he does not want there to be a fastest solve option. This makes sense as seeing a computer solve the cube as fast as possible is not useful for improving his cubing ability and does not teach anything. Leo is clear that he wants the ability to easily add more algorithms to the software so that when he becomes a more experienced cuber, he can add the more advanced CFOP methods.

List of User Requirements

- There should be a cube displayed on the screen in the form of a template.
- This cube should have each edge be able to be painted in one of the six colours of the Rubik's cube.
- The program should output any moves in the standard Rubik's notation.

Rubik's Cube Solver & Trainer

- The software should have a training mode to test the user's retention of an algorithm by getting them to enter it in the standard Rubik's notation
- There should be a save function so that the cube is able to be loaded from any state in different instances of the program but only by the user who saved it.
- A tutorial for the Rubik's notation should be accessible within the software.
- The solver should be able to solve the CFOP method
- There should be buttons to rotate the faces of the cube so that the user can solve it within software
- A hint button will be in place to guide the user in the next steps forward.
- An invalid cube should not be allowed to be attempted to solve by the user or the program.
- New algorithms should be addable to the trainer.

Further Research

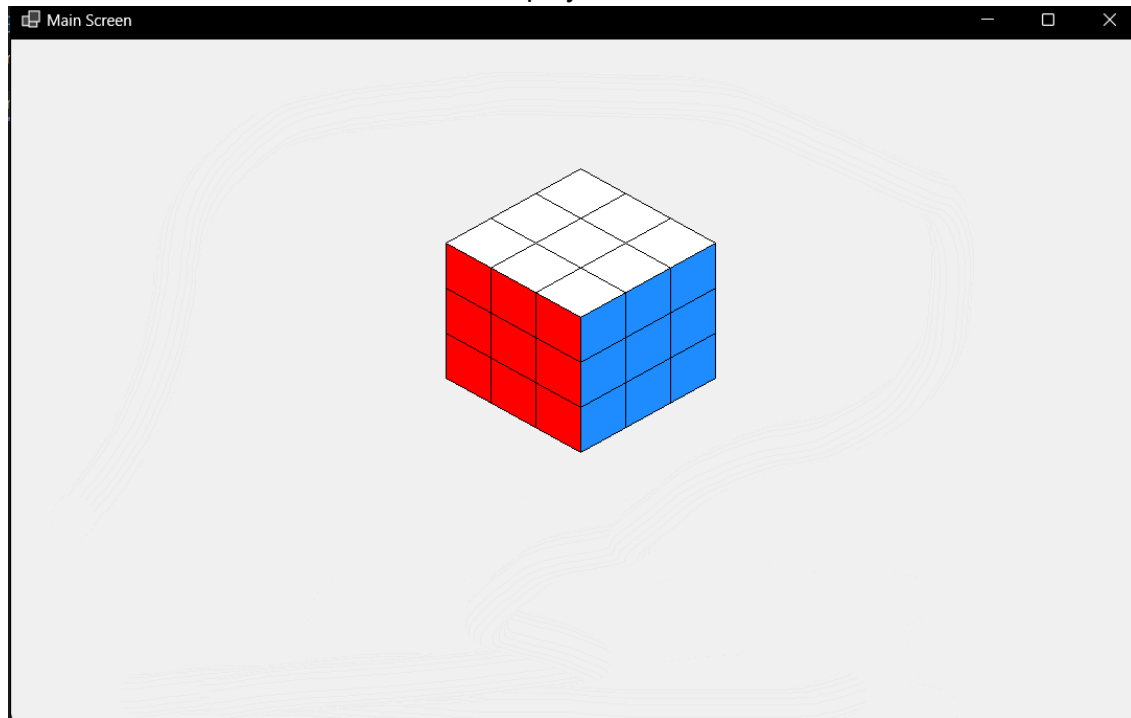
How to draw coloured shapes onto a WinForms Application

<https://learn.microsoft.com/en-us/dotnet/api/system.drawing.graphics.fillpolygon?view=net-8.0>

For my project, I need to be able to draw coloured stickers onto a projection of a cube within a WinForms environment. I have used the Microsoft dotnet documentation to research the system.drawing class.

Rubik's Cube Solver & Trainer

With some prototyping, I managed to use the documentation to create a simple display of a rubik's cube with each of the stickers displayed.



How to encrypt and securely store confidential user information

[Salt \(cryptography\) - Wikipedia](#)

[Hash Algorithm Comparison: MD5, SHA-1, SHA-2 & SHA-3 \(codesigningstore.com\)](#)

In order to store a password securely in a database, the entered password needs to pass through some key steps.

First, a salt needs to be randomly generated. A salt is defined as random data that is passed through a hashing algorithm along with the passphrase. Salting is used in order to prevent attacks that use a precompiled dictionary of the hashing algorithm. There is no one way to generate a salt but it should be generated using a large amount of noise to ensure salts are rarely ever reused for different passwords.

Next, the passphrase and the salt will be joined together to form one long string of data. This is the string that will be passed into the hash function.

A multitude of hashing algorithms exist, but the most prominent ones include MD5, SHA-1, SHA-2, SHA-3, and SHA-256. For security, SHA-256 is the best-fitted algorithm since no known collisions have occurred. A collision is when two different data strings hash to the same output; two different passwords are the same after hashing.

Only once the user's password has been concatenated with a randomly generated salt and this string passed through the SHA-256 hashing function can a password be securely stored inside the database, alongside its salt.

To allow a user to log in, the username will be used as a primary key to access the hashed password and the salt. The password will be added to the salt and passed through the hashing algorithm in the same way a password is stored. If this output matches with the output stored in the database, the user has entered the correct password and will be allowed to log in successfully.

IDA* For solving the white cross

Currently, whilst it has been proven that any white cross can be solved in 8 moves, there are no known algorithms for determining the fastest solution. This stage is always solved via inspection by a human. Subsequently, this is challenging to emulate using a computed

Rubik's Cube Solver & Trainer

solution and so heuristic methods are required for fast queries. The IDA* algorithm is a combination of using iterative deepening, and the A* searching algorithm. I have chosen to use IDA* since it tends to reduce the memory usage of the program. It is used to estimate the required moves to reach the solved white cross, given any state, and find a possible shortest path in that amount of moves. Since this is a heuristic algorithm, the best solution is not necessarily found, but a reasonable solution is. A high level overview of the steps is as follows:

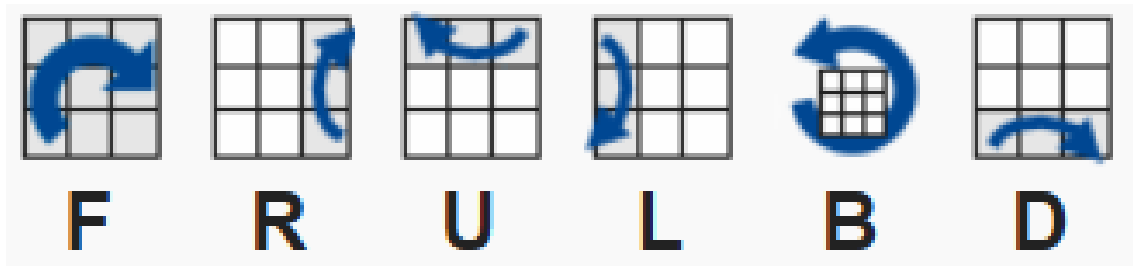
1. **Initialise** the puzzle (the cube with the white cross not solved).
2. **Set an initial threshold** based on the heuristic of the starting state (initial heuristic might be based on how misplaced the white cross edges are).
3. **Perform Depth-Limited Search:**
 - Explore the state space, starting with the initial configuration.
 - Track the current path and move cost.
 - Prune paths that exceed the current threshold.
4. **Iteratively increase the threshold** if no solution is found and repeat the search.
5. **Terminate** when a solution (sequence of moves) is found that achieves the solved white cross.

Modelling of the Problem

Rubik's Cube Notation:

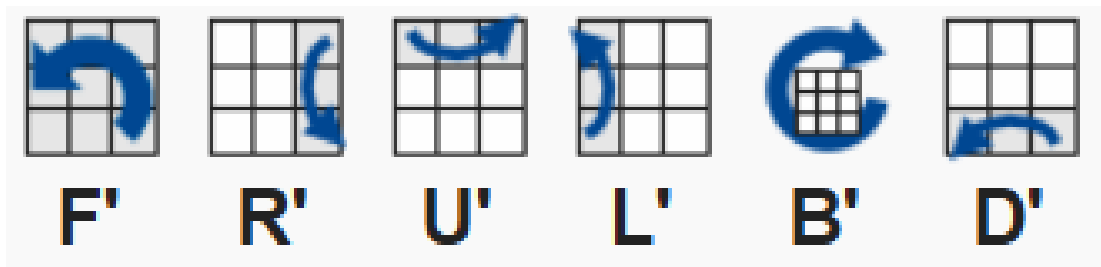
Rubik's cube notation is used to clearly communicate the different possible rotations onto a rubik's cube. For the simplest moves, a single letter is used to indicate a clockwise rotation about a face. An example of this would be **R**; representing a rotation 90 degrees clockwise about the right face of the cube.

Rubik's Cube Solver & Trainer



^ Diagram of move rotations on a cube; source: <https://ruwix.com/the-rubiks-cube/notation/>

To represent a move in the opposite direction, an anticlockwise rotation, can be described as a “prime” move. In notation, this is scribed represented by an apostrophe '. Concatinated with a move we arrive at **R'**, meaning an rotation 90 degrees anti clockwise about the right face of the cube

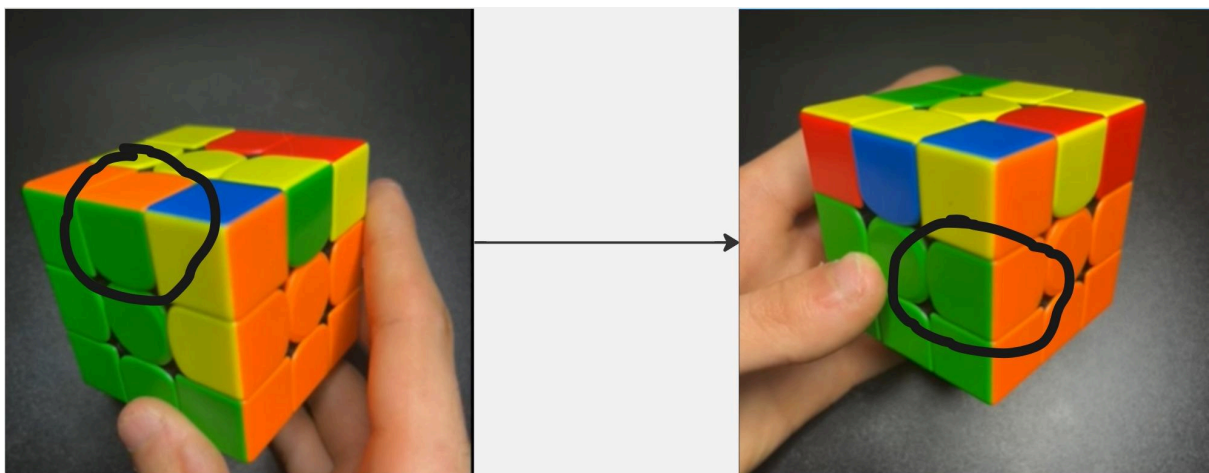


^ Diagram of move rotations on a cube; source: <https://ruwix.com/the-rubiks-cube/notation/>

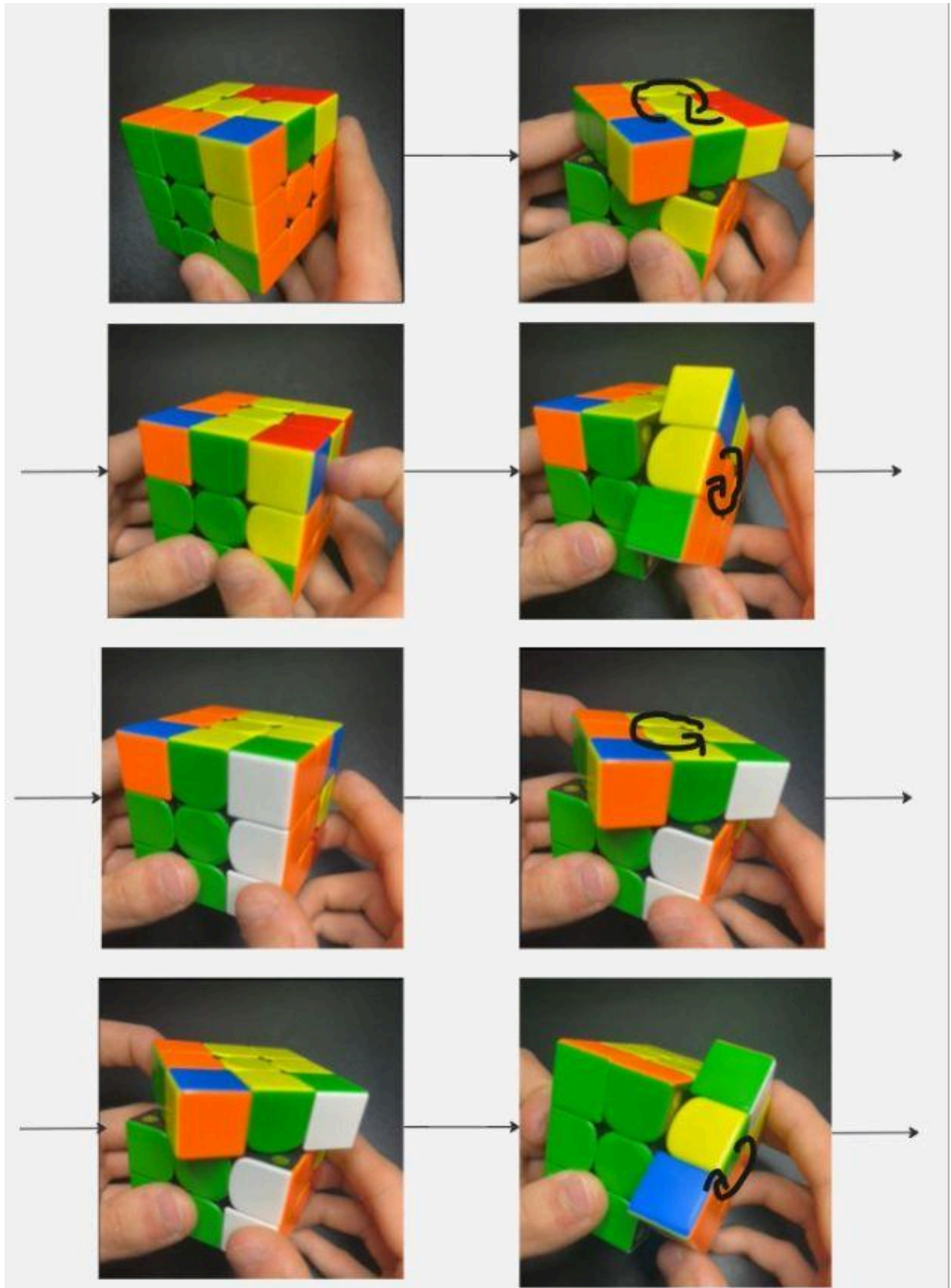
Putting this knowledge into practice, a full algorithm will be written as follows:

U R U' R' U' F' U F

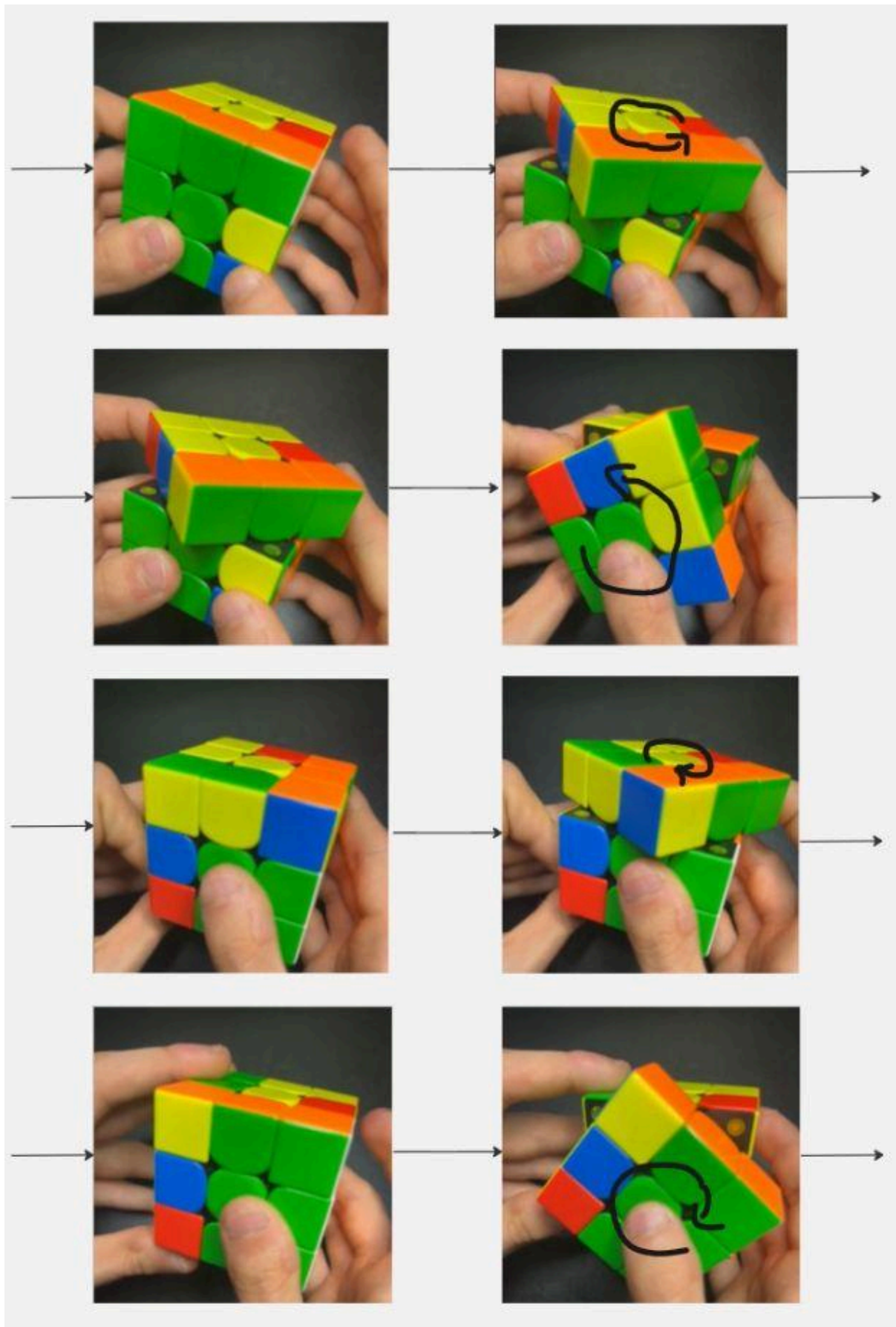
This is one of the two beginner algorithms for inserting a second layer piece into its slot.



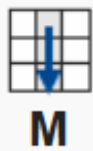
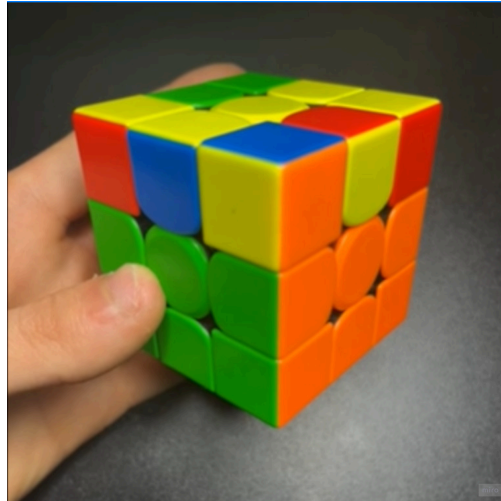
Rubik's Cube Solver & Trainer



Rubik's Cube Solver & Trainer



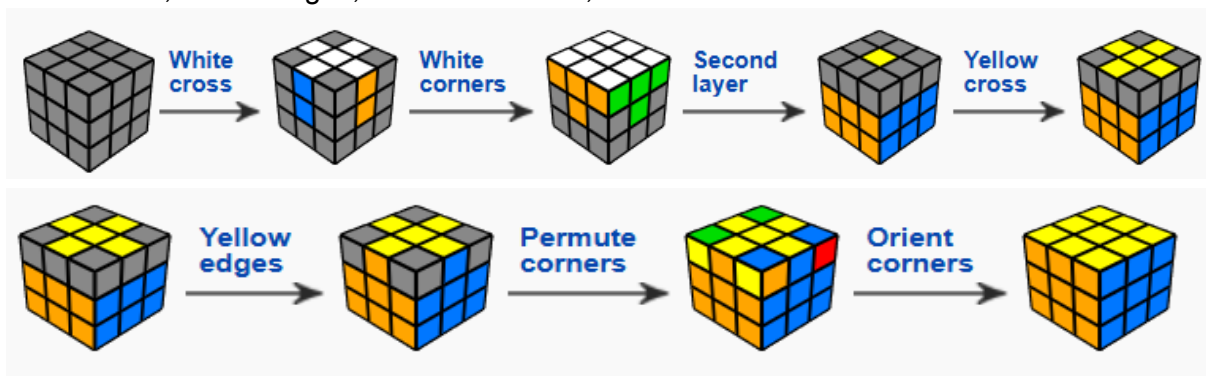
Rubik's Cube Solver & Trainer



For My implementation of the rubik's cube, I am restricting the system to only the moves mentioned on page 8 and **M** as described in the diagram below. This is because other more complex moves can be done using a combination of the standard moves. Having too many buttons will lead to a clustered user interface and could lead to users clicking the wrong buttons.

Beginner Algorithm:

The beginner algorithm is made up of 7 stages: White Cross, White Corners, Second layer, Yellow cross, Yellow edges, Permute corners, Orient corners

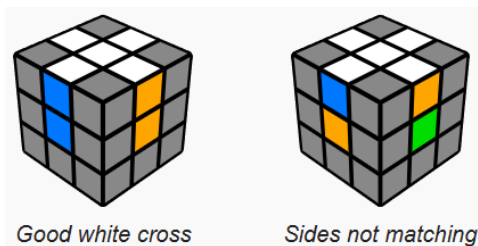


^ Flow diagram of the Beginner Algorithm;

Source: <https://ruwix.com/the-rubiks-cube/how-to-solve-the-rubiks-cube-beginners-method/>

White Cross:

The first stage is to form a white cross. This is achieved by moving the four white side pieces to the white face. The tricky part with the white cross is ensuring that the side pieces also match the adjacent face.



A bad white cross will look similar to the one in the diagram. The side tiles of the white pieces do not match with the middle tile of the adjacent faces. This is not the desired result from the white cross face and you should aim to have a copy of what the good white cross example on the diagram

Rubik's Cube Solver & Trainer

^ Diagram of good and bad white crosses;

Source: <https://ruwix.com/the-rubiks-cube/how-to-solve-the-rubiks-cube-beginners-method/>

White Corners:

In this stage, the user is aiming to get the correct 4 white corner pieces inserted into the corners of the white face.



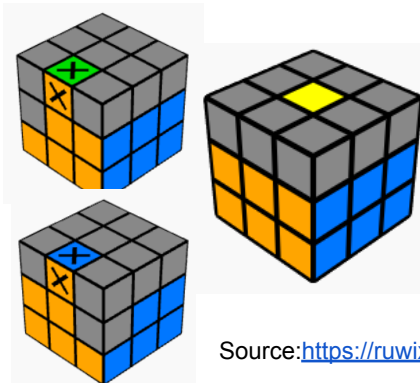
The user must ensure that the 3 colours all line up correctly after being inserted into the corner. There are 4 algorithms a user can use to get a piece into the correct position.

^ Diagram of a solved white face

Source: <https://ruwix.com/the-rubiks-cube/how-to-solve-the-rubiks-cube-beginners-method/>

Second layer:

In this stage the user is aiming to solve the first two layers (also known as F2L in more complex methods).



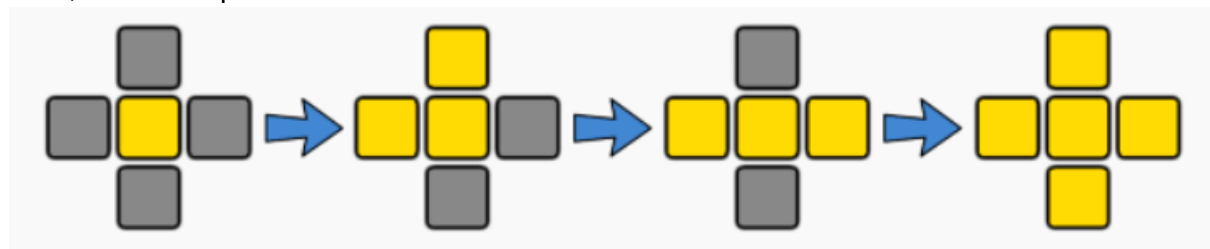
There are only two algorithms for this stage. Arguably, these are the hardest two algorithms for the beginner method but there is a trick to only learn one. These algorithms are known as symmetric algorithms because they are essentially the same algorithm but being performed on the opposite side of the face.

^ Diagram of the two algorithms and the solved F2L;

Source: <https://ruwix.com/the-rubiks-cube/how-to-solve-the-rubiks-cube-beginners-method/>

Yellow cross:

The yellow cross is the easiest stage of the beginner method. There is only one algorithm used, and it is repeated until the desired cross is achieved



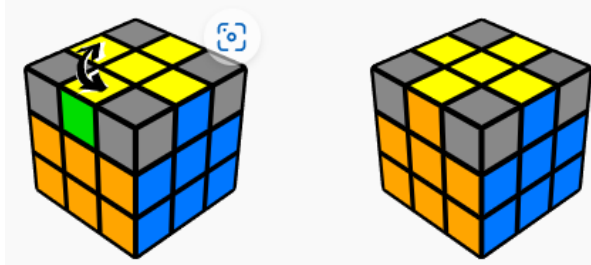
^ Diagram of the yellow cross stage

Source: <https://ruwix.com/the-rubiks-cube/how-to-solve-the-rubiks-cube-beginners-method/>

Rubik's Cube Solver & Trainer

Yellow edges:

This stage is not always necessary if you are lucky but it will appear more often than not. This stage aims to rotate the yellow edges around until the correct edge is aligned with the face.



^ Diagram of the flipping of a yellow edge

Source: <https://ruwix.com/the-rubiks-cube/how-to-solve-the-rubiks-cube-beginners-method/>

Permute corners:

This stage permutes the yellow corners to the correct position on the cube.



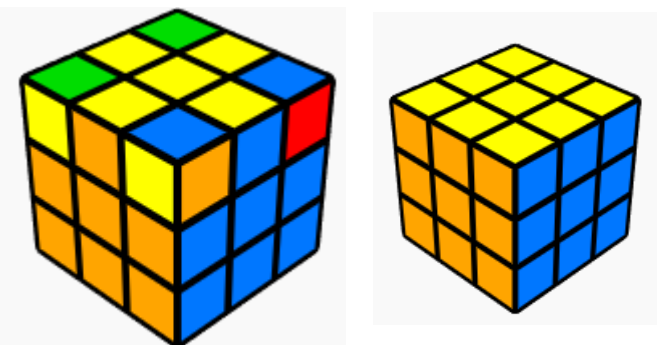
There is again only one algorithm for this stage but it may need to be repeated. If there are no corners in the correct permutation (i.e. not in the OK position on the diagram where the 3 colours match the surrounding faces), you will need to do the algorithm on any side of the cube to get a correct result. Once you have done this, find the correctly permuted corner and do the algorithm on that side of the cube. This will result in all the corners being correctly permuted.

^ Diagram of the permutations of a yellow edge

Source: <https://ruwix.com/the-rubiks-cube/how-to-solve-the-rubiks-cube-beginners-method/>

Orient corners:

The final stage of the beginner method is an algorithm repeated up to 24 times. This stage clearly shows why this is known as the beginner method as it peaks into how inefficient the algorithm as a whole is.



^ Diagram of the cube before the final stage and the solved cube

Source: <https://ruwix.com/the-rubiks-cube/how-to-solve-the-rubiks-cube-beginners-method/>

Rubik's Cube Solver & Trainer

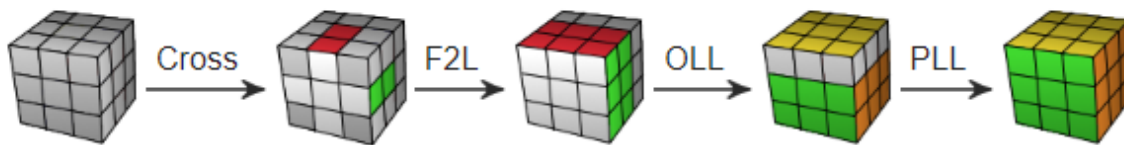
CFOP Method

^ All diagrams in this section are sourced from ruwix

Source: <https://ruwix.com/the-rubiks-cube/advanced-cfop-fridrich/>

The CFOP method, otherwise known as the Advanced Fridrich Method, has now evolved to a far complexer method for cubers attempting to set new records. However, this method is a major stepping stone for enthusiasts and is known worldwide.

It consists of only 4 steps: white Cross, First 2 layers, Orientation of last layer, Permutation of last layer.



White Cross:

The first stage is to form a white cross. This is the same first step as the beginner method. There is not a predetermined set of algorithms for this stage and it just needs to be learnt how to recognise pieces.



Good white cross



Sides not matching

A bad white cross will look similar to the one in the diagram. The side tiles of the white pieces do not match with the middle tile of the adjacent faces. This is not the desired result from the white cross face and you should aim to have a copy of what the good white cross example on the diagram

First 2 layers:

This is the first major improvement over the beginner method. This combines the insertion of the white corners and the insertion of the side pieces into one step. In order to achieve this, there are multiple complex algorithms to learn to do effectively, but for beginning to learn CFOP, only a couple of algorithms are required but will need to be used multiple times per side. This solver will focus on the more complex algorithms. There are 42 cases for f2l (including the solved case). They are separated into sections for the different positions of the corner and edge pair positions as seen in the capture below. A cube solved to the F2L stage will have its white face completed like found in previous stages along with the first 2 layers uniform.



(Source: <https://ruwix.com/the-rubiks-cube/advanced-cfop-fridrich/first-two-layers-f2l/>)

Rubik's Cube Solver & Trainer

(Source: https://cubingcheatsheet.com/algs3x_f2l.html)

Basics



Case 1
U R U' R'



Case 2
U' (F' U F)



Case 3
F' U' F



Case 4
R U R'

Corner & Edge Top



Case 5
(U' R U R') (U' R U2 R')



Case 6
(U F' U' F) (U F' U2 F)



Case 7
(U' R U2 R') (U' R U2 R')



Case 8
(U F' U2 F) (U F' U2 F)



Case 9
(U F' U') (F U' F') U' F



Case 10
(U' R U) (R' U R) U R'



Case 11
U' (R U2 R') d (R' U' R)



Case 12
d (R' U2 R) d' (R U R')



Case 13
(U F' U) (F U' F') U' F



Case 14
(U' R U') (R' U R) U R'



Case 15
(F U F U') d' (F U F')



Case 16
(R U' R' U) d (R' U' R)

Corner Point Up



Case 17
(R U2 R' U') (R U R')



Case 18
(F' U2 F U) (F' U' F)



Case 19
(U R U2 R') (U R U' R')



Case 20
(U' F' U2 F) (U' F' U F)



Case 21
U2 (R U R' U) (R U' R')



Case 22
U2 (F' U' F U') (F' U F)



Case 23
(R U R' U') U' (R U R' U')
(R U R')



Case 24
y' (R' U' R U) U
(R' U' R U) (R' U' R)

Liam Short

3005

20

Rubik's Cube Solver & Trainer

Corner Bottom & Edge Top

 Case 25 (U' F' U F) (U R U' R')	 Case 26 (U R U' R') (U' F' U F)	 Case 27 (R U' R') (U R U' R')
 Case 28 (F' U' F) (U' F' U F)	 Case 29 (F' U' F) (U' F' U F)	 Case 30 (R U' R') (U' R U' R')

Corner Top & Edge Middle

 Case 31 (R U' R') (d R' U R)	 Case 32 (R U' R' U') (R U' R' U') (R U' R')	 Case 33 (U' R U' R') (U' R U' R')
 Case 34 (U' F' U F) (U' F' U' F)	 Case 35 (U' R U' R') (d R' U' R)	 Case 36 (U' F' U' F) (d' F U' F')

C.Bottom & Edge Middle

 Case 37 Solved	 Case 38 (R U' R' d R' U' R) (U' R' U' R)	 Case 39 (R U' R' U') (R U' R' U') (R U' R')
 Case 40 (R U' R' U') R U' R' (U R U' R')	 Case 41 (R U' R') d (R' U' R U') (R' U' R)	 Case 42 (R U' R' U') (R U' R') U d (R' U' R)

Orientation of Last Layer:

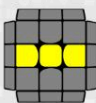
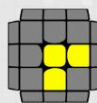


Orientation of Last Layer (OLL) aims to make the yellow face consist entirely of yellow stickers. At this point, unlike the solved white face, the adjacent sides' colour does not matter. Full CFOP does this with a multitude of algorithms similar to that of F2L, but this often only grants a minor advantage over the alternative 2-Look OLL. This narrows it down to only 10 algorithms to learn compared to 58.

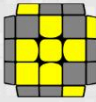
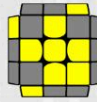
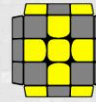
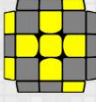
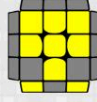
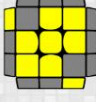
Rubik's Cube Solver & Trainer

(Source: https://cubingcheatsheet.com/algs3x_2loll.html)

EOLL 1st Look

 LINE $F (R U R' U') F'$	 SMALL L $f (R U R' U') f'$	 DOT $R U2 (R2' F R F') U2 (R' F R F')$
---	--	---

OCLL 2nd Look

 27 SUNE $(R U R' U') (R U2 R')$	 26 ANTI SUNE $(L' U' L U') (L' U2 L)$	 21 H $F (R U R' U') (R U R' U') (R U R' U') F'$
 22 Pi $R U2 (R2' U' R2 U') (R2' U2 R)$	 23 HEADLIGHTS $R2 D (R' U2 R) D' (R' U2 R')$	 24 T $(r U R' U') (r' F R F')$
 25 BOWTIE $F' (r U R' U') (r' F R)$		

Permutation of Last Layer:

Permutation of Last Layer (PLL) is the final stage of CFOP. Like OLL, there is an advanced variation with 21 algorithms, or a 2-look with 6. The trainer will be again focused on the 2-look method.

(Source: https://cubingcheatsheet.com/algs3x_2lppl.html)

CPLL 1st Look

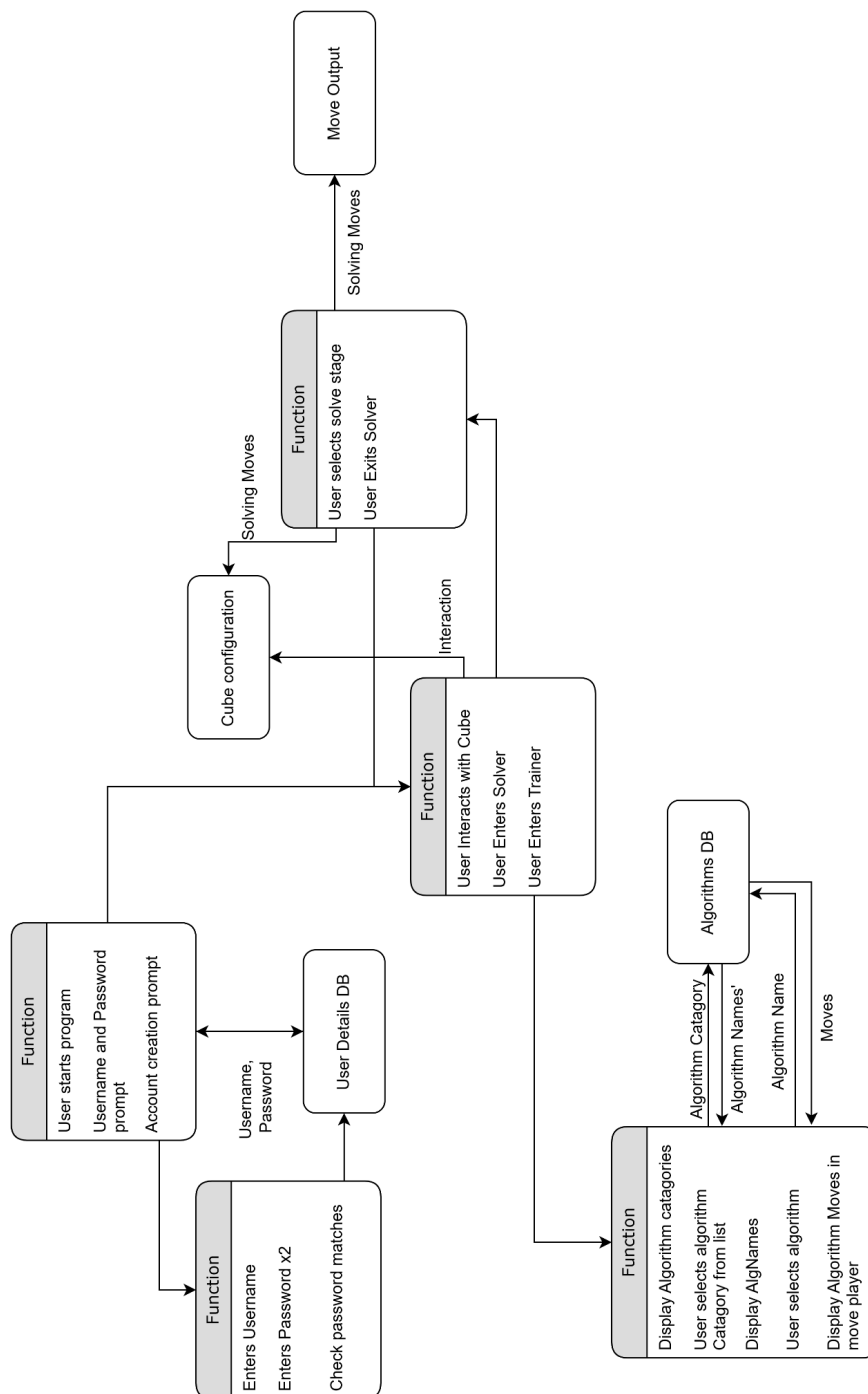
 HEADLIGHTS BACK (Aa Perm) $x' (R' D R') U2' (R D' R') U2' R2 F' x$	 NO HEADLIGHTS $F R U' R' U' R U R' F' (R U R' U') (R' F R F')$
---	--

EPLL 2nd Look

 Ua Perm $M2' U M U2 M' U M2'$	 Ub Perm $M2' U' M U2' M' U' M2'$	 Z Perm $(M2' U' M2' U') M' (U2 M2' U2) M'$
 H Perm $(M2' U M2') U2 (M2' U M2')$		

Rubik's Cube Solver & Trainer

Context Diagram (Level 0):



The Context Diagram offers a high-level overview of the system. It shows how Leo (the user) interacts with the software without diving into the internal processes.

User: User uses the system for Rubik's Cube training and solving.

System: Receives input from Leo (such as login credentials, cube configuration, and algorithm selection). It then provides output, like training performance data and solutions.

Data Flows: The system exchanges information with Leo. Leo provides inputs (login details, cube configuration, etc.), and the system returns feedback (e.g., successful login, solutions to cube configurations, and training progress).

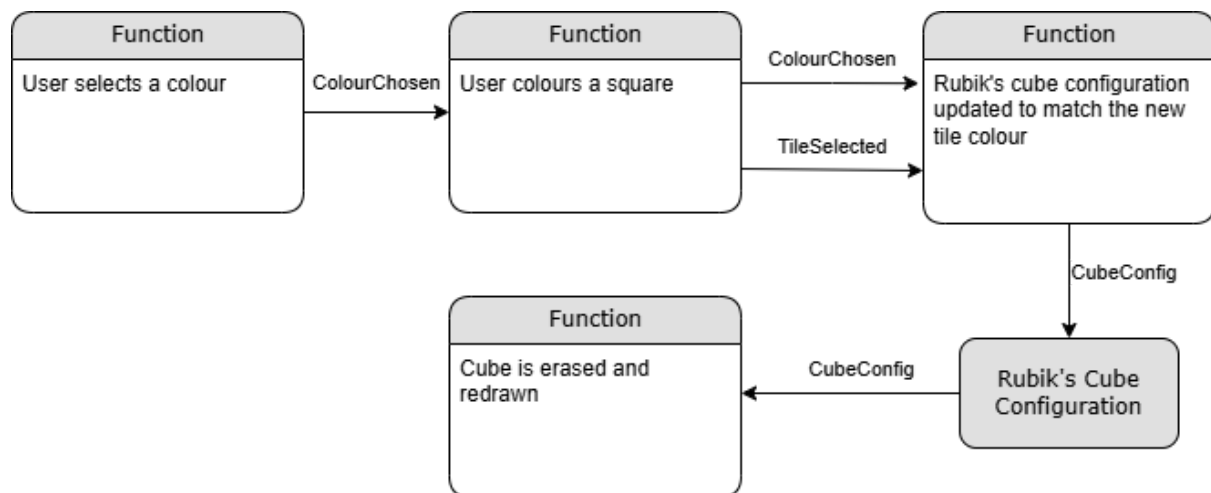
Rubik's Cube Solver & Trainer

Level 1 DFD - User Authentication: Login/Logout:

This diagram focuses on how the system handles the login and logout processes.

- **Process: User Authentication:** When Leo attempts to log in, the system verifies credentials.
- **Inputs:** Login details (username, password).
- **Outputs:** If successful, the system grants access and logs Leo into the system. If unsuccessful, it provides an error message.
- **Data Store:** User credentials are stored securely in the database.
- **Actors:** User interacts with the authentication process.

Level 1 DFD - Input Cube Configuration:

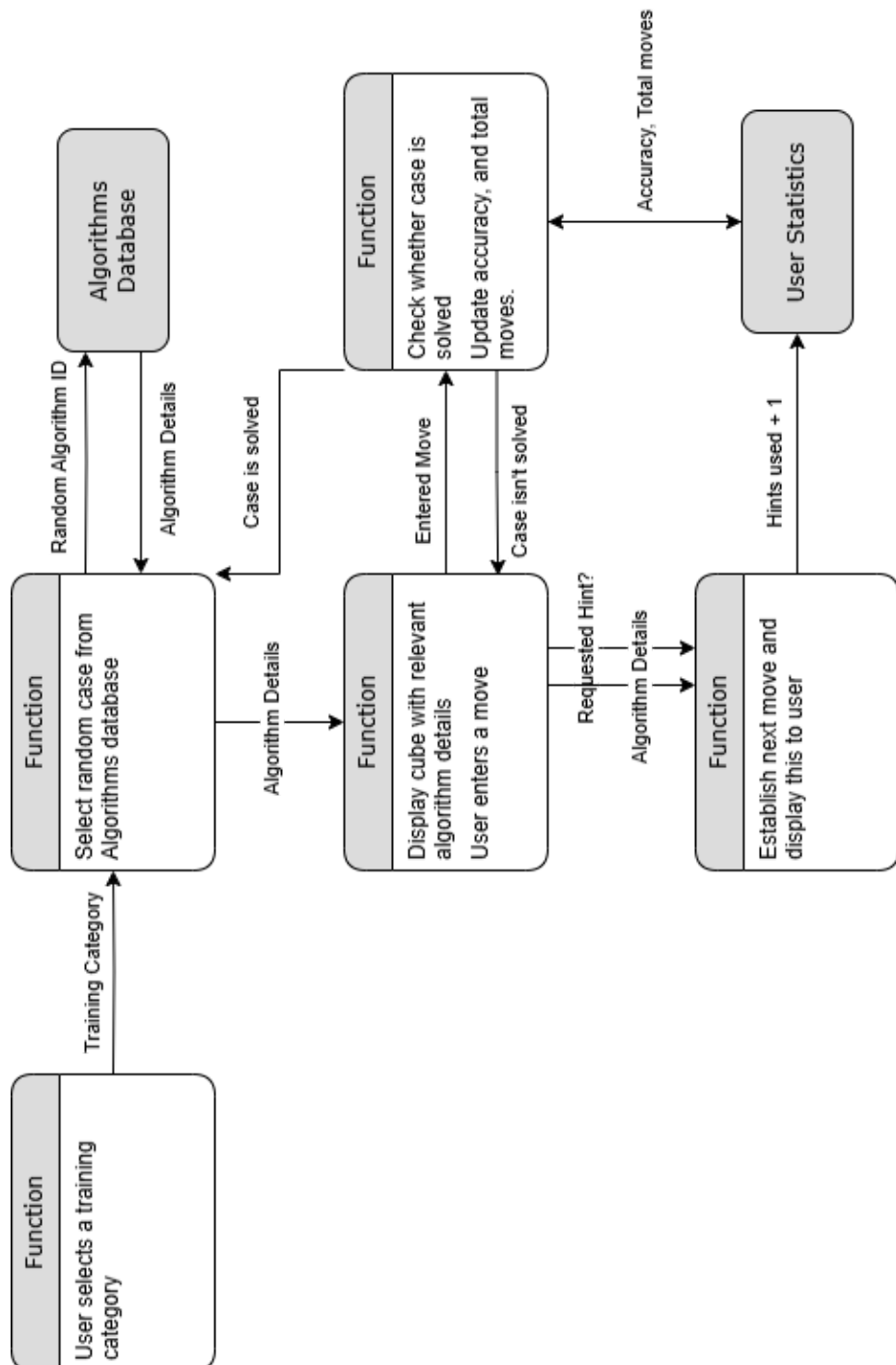


- **Process: Input Cube Configuration:** Leo can input the cube's current state by selecting a colour from the 6 available tile colours and selecting a tile on the cube
- **Inputs:** Cube colors. Cube tile
- **Outputs:** The system stores the new configuration and redraws the cube to depict what the user changed.
- **Data Store:** Cube configuration is saved in a CSV

Rubik's Cube Solver & Trainer

Level 1 DFD - Training Module:

This diagram details the flow of data during the user's training sessions. The user selects a training category and a relevant case is displayed to them. The user enters moves until the case is solved. The user can request a hint if they are stuck which will show the user the next move. Relevant statistics are saved in order to track progress.



Process: The system presents training cases (e.g., specific F2L, OLL, or PLL cases) for the user to solve. It records the user's performance and progress.

Inputs: The system presents a set of scrambled cases based on the selected algorithm category.

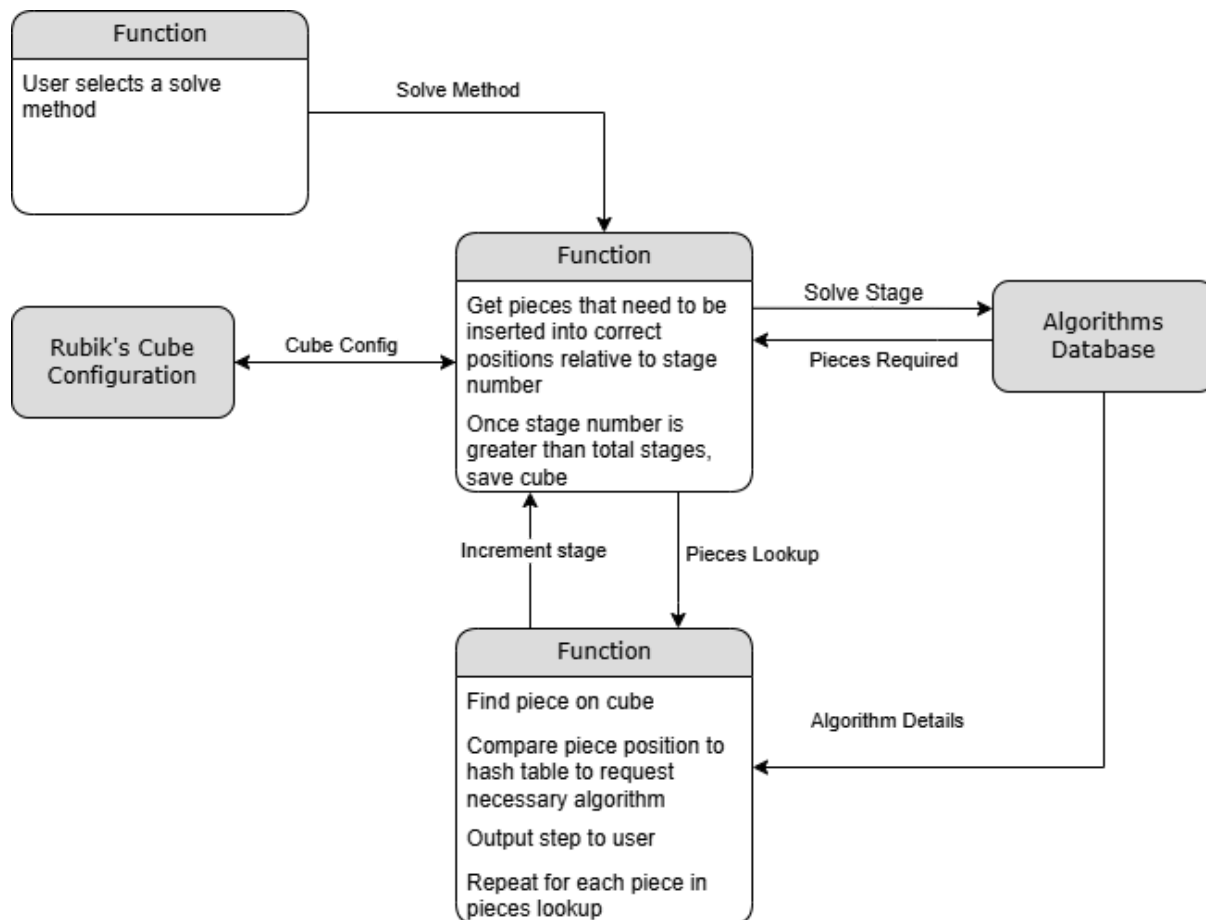
Outputs: Cube configuration is displayed. Hints are displayed when requested. Performance feedback, including time taken, moves made, minimum moves required and accuracy.

Data Store: Performance data is saved to track progress and offer insights during future training.

Rubik's Cube Solver & Trainer

Level 1 DFD - Solver Module:

This diagram shows the process of providing solutions to the cube configuration entered by the user based on the users selected solve method. The module requests for each of the pieces required to be in the right place in order for the next stage to start. Each piece is found on the cube and its position relates to a hash stored in the algorithm's database that relates to the required algorithm to get the piece in the correct location. The algorithm is outputted to the user. This is repeated for each stage in the solving process until the cube is solved.



- **Process: Solver Module:** After receiving the cube configuration, the system requests information about the next steps and how to insert pieces.
- **Inputs:** Cube configuration. Solve method. Next step button.
- **Outputs:** A step-by-step solution for Leo to solve the cube. Current cube configuration displayed
- **Data Store:** The solution is saved to the cube configuration.

Rubik's Cube Solver & Trainer

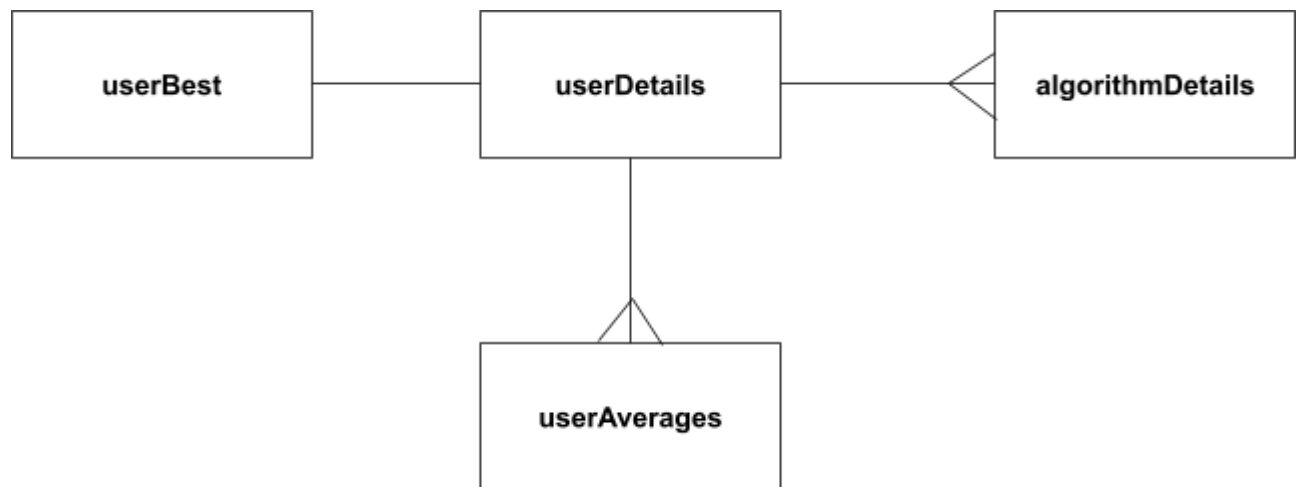
Data sources

The system will manage various types of data, requiring a robust database solution to handle the large volumes processed by the application. To facilitate user interaction, a graphical user interface will be developed using WinForms, allowing users to access specific data seamlessly without direct database interaction. The underlying database will be managed by **EDIT**, ensuring efficient data storage and retrieval. This approach simplifies the user experience, as all necessary information can be accessed through the interface, eliminating the need for users to engage with the database directly.

Current System		
Data	Source	Destination
Rubik's Cube configuration	Edited by the user	Storage box
Algorithm sheet	Created by the user	Storage box
Solve times	Created by the user	Storage box
Random Scramble	Generated online	Rubik's Cube configuration

Proposed System		
Data	Source	Destination
Rubik's Cube configuration	Cube state on the previous session	System files
Algorithm sheet	Integrated in the software / added by the user	algorithmDetails table in the database
Solve times	Users 3 / 5 solve average And their best	userTimes table in the database
Random Scramble	Generated by the software	Rubik's Cube configuration

ER Diagram of Proposed Database



Data Dictionary

During the development process, sample data will be employed to ensure the system's security is robust and that sensitive information remains protected. Once the data is assigned to the appropriate variables, it will be transferred to the correct database to prevent loss when the system is closed or exited.

In contrast, the old system, being non-digital, lacks variables entirely. In this context, the storage box in Leo's room can be likened to a database, with the algorithm sheets serving as the tables. However, a paper-based system inherently lacks the capability for automatic computer processing. In the new system, various variables will be introduced to efficiently store and manage user data.

The program will use many different of variables for storage of data. Each different piece of data will have a different variable name as this is the only way to store data that is needed inside the program. Field names must be representative of the data stored inside it. For example, a user's fastest solve would be stored under "personalBest" and not "username". Using correctly named fields makes it easier to understand the program during development. Example data will be used through the development of the system to test that the security of the information is not compromised.

Rubik's Cube Solver & Trainer

Data Dictionary					
userDetails					
Field Name	Field Purpose	Field Type	Field Size	Example Data	Validation
username	To store the name of the user for login	varchar	20	leo	Field must contain alphabetical characters only. Field must not be blank. Field must be unique
password	To store the hashed password of the user for login	varchar	64	74e7b02f1fd5d94845b5d74fcc11ed48552bd6b91d3871ef7860d746b9711bbf	Field must not be blank
salt	To store the salt used in order to hash the password	varchar	16	He1nBjTiAll5bT9a	Field must not be blank
userAverages					
username	To uniquely identify the timings to the user who achieved them.	varchar	20	leo	Field must not be blank
average	To store the average the user got on this solve set.	int	4	49	Field must be an integer. Field must not be blank
setType	To determine whether it was a 3 solve or 5 solve average.	char	1	5	Field must be a character. Field must not be blank
userBest					
personalBest	To store the best time the user has.	int	4	67	Field must be an integer.
username	To link the times to a user	string	20	leo	Field must not be blank
algorithmDetails					

Rubik's Cube Solver & Trainer

AlgorithmID	Unique number to identify each algorithm	int	3	25	Field must be an integer. Field must not be blank. Field must be unique.
algName	Name of the algorithm	varchar	20	sune	Field must not be blank
category	What category does the algorithm belong to?	varchar	15	1-look OLL	

Objectives

The main objective of this project is to create a system where users can learn and practice solving a Rubik's cube using human methods. My system will focus on the CFOP method of solving the cube.

1. Database

- a. When launching the program, it will check if the database has been created
 - i. If the database does not exist, the database will be created
 - ii. If the database already exists, an output will tell the user as such
- b. The database will be created using DDL commands
- c. The database will have multiple interlinked tables

2. Log in

- a. A database table will store a username and encrypted password
 - i. The encrypted password will be hashed using SHA-256
 - ii. A salt will be generated for each user and stored in the table
- b. If a record with the inputted username does not exist, a prompt will ask the user if they would like to create an account
 - i. If they create an account, the username and password will be inserted into the table
- c. If a record with the inputted username exists, but the password is not correct, an error message will be displayed telling the user
- d. If the user successfully logs in to an account, the log in screen will disappear and the program will start in the playground.
- e. Usernames should be at least 4 characters
- f. Passwords should be at least 6 characters

3. Menu

- a. The main menu will have 4 buttons
 - i. Scramble
 - ii. Trainer
 - iii. Solver
 - iv. New Cube
- b. If the user selects a choice, they will be taken to that section of the program
- c. A back button will appear when inside a menu option.
 - i. The back button will bring the user back to the playground | main menu

4. 2d cube display

- a. The cube will be displayed to the user as a 2D projection
- b. The projection will be an isometric view of the cube meaning 3 faces of the cube will be displayed at all times.

Rubik's Cube Solver & Trainer

- c. The gaps between tiles and faces will be illustrated by a black grid-like structure

5. Display rotation

- a. There will be 4 buttons to control the cubes orientation
 - i. Rotate Up
 - ii. Rotate Down
 - iii. Rotate Left
 - iv. Rotate Right
- b. Pressing one of these buttons will have the same effect as rotating the cube 90 degrees in that direction.

6. Face rotation

- a. The faces will be addressed using rubik's cube notation, consisting of 6 letters for a clockwise rotation and the same letters followed by an apostrophe for anticlockwise rotation
 - i. R R'
 - ii. L L'
 - iii. U U'
 - iv. D D'
 - v. F F'
 - vi. B B'
- b. Rotating on one of these faces will be a rotation of 90 degrees clockwise, or anticlockwise when on a prime move, on their corresponding face.
- c. The user will also have an M / M' move which will rotate the middle slice of the cube

7. Scramble

- a. The scramble button will generate 20 moves
 - i. A move will be a primary face rotations or their prime (anticlockwise) counterpart
 - ii. Two consecutive moves should not undo each other. I.e. an R should not be followed by an R' since that is the same as no rotation.
- b. These moves will be automatically applied to the digital cube
- c. The moves will be outputted to a text box for the user to read

8. Timer

- a. A timer button will start a stopwatch to time the users solve speed
- b. After solving the cube, the stopwatch will stop
 - i. The time to solve will be recorded into a table in the database along with the username
- c. The current time should be constantly displayed and updated regularly
- d. The timer should not start if the cube is already solved

9. Save / load

- a. The current position of all tiles will be stored into a text file
- b. A user should only be able to access their own saved cube
 - i. The file will be named the username of the current user when saved to differentiate them from other users cubes
 - ii. Each user will be able to store 1 cube state
- c. Loading the cube will overwrite the currently displayed cube with what the user has stored
 - i. If the user has no stored cube, an error will be displayed.

10. New cube

- a. Pressing the new cube button will generate a new cube
 - i. The new cube will be in the solved state
- b. The previous cube will be destroyed upon creation of a new cube

11. Tile recolouring

- a. Every tile on the rubik's cube will be able to be recoloured

Rubik's Cube Solver & Trainer

- i. The centres of each face will not be able to be recoloured since they are crucial to the structure of the cube
- b. The colour selector will be a button that cycles through the 6 colours upon clicking
- c. A small button grid will be displayed with each button corresponding to a tile on the cube
 - i. The grid will be a 2D cube net
 - ii. Upon clicking a tile, the tile will be recoloured to the current colour selected
 - iii. The cube and button grid will update to reflect the change
- d. The recolouring will only be available in the playground
- e. If the user enters an invalid cube configuration and attempts to enter the solver, they will not be able to.

12. Trainer

- a. Upon entering the training mode, 4 dropdowns will be displayed along with a play and pause button
- b. The user can select any algorithm from the following CFOP steps and a custom section
 - i. F2L
 - ii. OLL
 - iii. PLL
 - iv. Custom
- c. When selecting an algorithm it should give the correct output:
 - i. A new cube that requires the selected algorithm will be displayed
 - ii. The moves are outputted to a text box
- d. Pressing the play button will sequentially do the moves of the algorithm and so solve the case
 - i. There will be a 1 second delay between moves
 - ii. The current move will be highlighted in the text box
- e. There will be an add button that will get the user to input the moves of an algorithm and give it a name.
 - i. Algorithms must have more than 3 moves
 - ii. All moves should be valid moves that can be performed by the program
 - iii. Any custom algorithms will appear under the custom dropdown
- f. All algorithms will be stored in a table in the database

13. Solver

- a. The solver section will solve the current rubik's cube
 - i. The solver will only solve by the CFOP method
- b. Each of the following steps will be there own button to be pressed
 - i. White Cross
 - ii. F2L
 - iii. OLL 1st Look
 - iv. OLL 2nd Look
 - v. PLL 1st Look
 - vi. PLL 2nd Look
- c. The stages will not work until all previous steps have been completed
- d. Each algorithm will be stored in a database and queried when needed
- e. After each stage, the moves required will be outputted to a text box

14. White Cross

- a. The white cross will use a shortest path algorithm.
- b. The algorithm will be a heuristic algorithm in order to optimise for speed rather than move count
- c. An output should be shown to the user when selecting the white cross stage so that they know the algorithm is processing

Rubik's Cube Solver & Trainer

15. F2L

- a. The F2L solver will be ran for each of the 4 pairs
- b. For each pair, there will be some preprocessing and then the primary algorithm
 - i. If the edge piece is not in the correct column, the edge will be placed into the top layer above the correct middle layer
 - ii. The corner piece will be placed into the top face if it is not already
 - iii. The corner piece will be moved to the correct column
 - 1. This will take precedence over the edge piece positioning
- c. It will iterate over each of the F2L algorithms, queried from a table, until it finds the correct one.
- d. Once the correct algorithm has been located, it will be added to a list and the next pair will begin searching.
- e. The output string will consist of 4 sets of moves, separated by a newline.

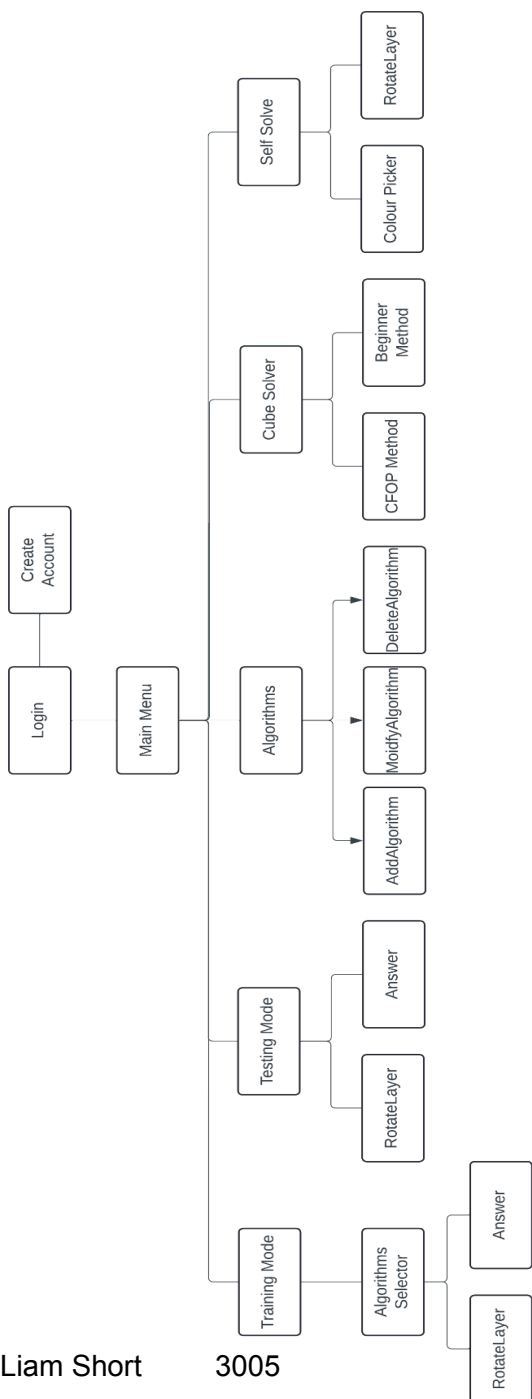
16. OLL + PLL

- a. For each of the OLL and PLL stages, detection logic will select which algorithm to use
- b. After each algorithm, that move set will be outputted to the text box

Design

In designing the Rubik's cube solver and trainer system, careful attention is given to the structure and flow of both the user interface and the underlying algorithms. The project will be divided into several core components, each designed with specific functionality and user interaction in mind. The user interface will feature a 3D projection of the Rubik's cube, enabling users to input configurations, practice solving, and receive step-by-step guidance in their respective modes. The system will utilise object-oriented design principles to model the cube, manage user interactions, and provide efficient algorithms for cube solving. Additionally, the design will incorporate modularity to allow for future scalability, such as adding new algorithms or training modes. Data storage will also be structured to save user progress and performance metrics, which will inform personalised training sessions.

High-Level Overview

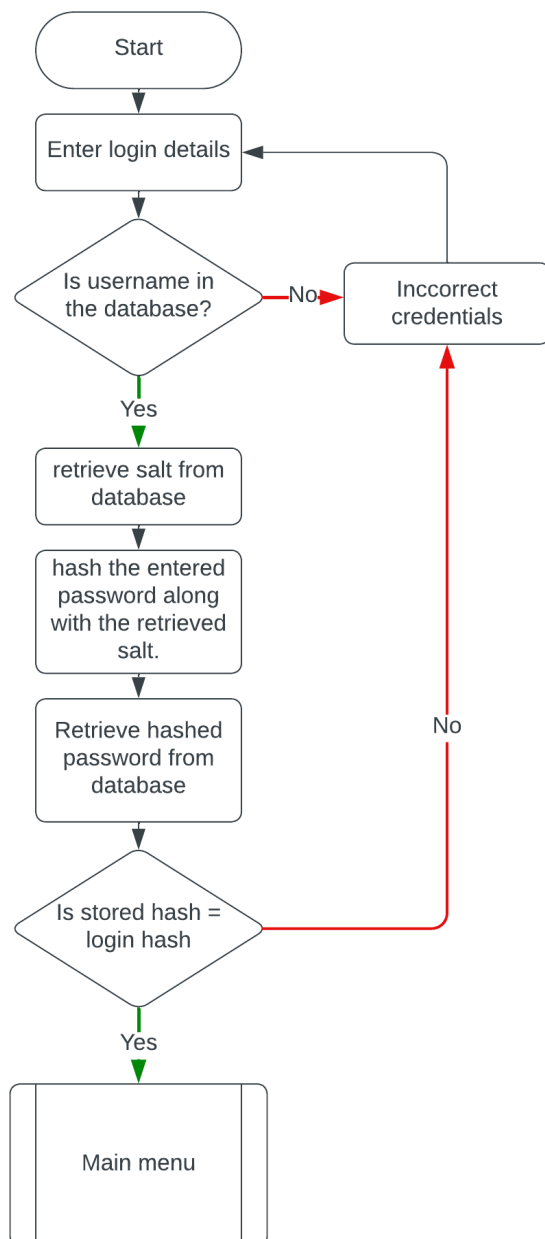


Overall Design

- Hierarchy Chart

This is a modular structure diagram to illustrate the possible user workflows. It shows how each module links to each other and outlines the different interfaces a user will see throughout the software. After a user has successfully logged into the program they will be able to interact with a solved cube. They may wish to save/load a cube, enter testing/training/solving mode or interact with the cube and set its stickers colours.

Rubik's Cube Solver & Trainer



Login

This flowchart shows a high level view into what will happen when a user attempts to login to the program. If a user exists with the entered username, it will compare the hashed passwords of the stored user. If the two passwords are the same, the user will be allowed access into the system.

Form structure

For there to only be one application window, there will be one main form. Between different modules of the application, the relevant UI will be made visible / hidden as required. To achieve this, each module's objects will be tagged with the modules name. This grants the ability to set all objects with tag X to visible and any object without tag X to invisible.

Rubik's Cube Solver & Trainer

Playground

The playground is the name for the module where a user can freely play with the cube. This means they can set the colours for each sticker and apply moves to the cube.

The playground is the base module that the program will launch into after successfully logging in to the application. Most of the objects from the playground will be visible in the other modules since they all require the ability to view the cube and make rotations on the cube. Only the paint tool will be hidden unless required.

Features:

Cube Display

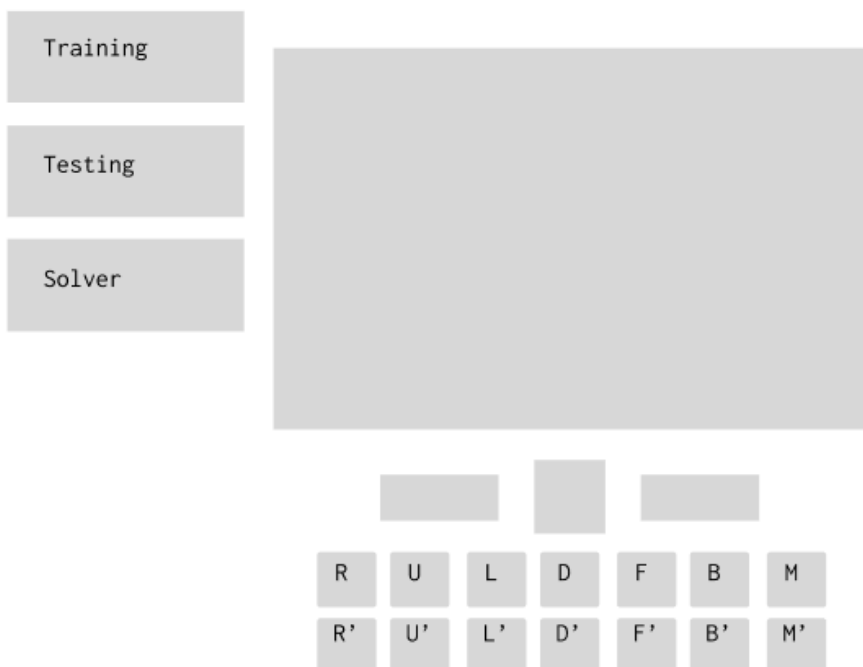
Rotation Buttons

Move buttons

Paint Tool

Navigation to other modules

User Interface Design:



Rubik's Cube Solver & Trainer

Training

In the training module, a user will be shown example cases from either the CFOP method. These will be selectable from a drop down menu that is dynamically loaded from the Algorithms database. Any user created algorithms will be automatically stored in a category called custom in order for the dropdowns to not extend past the boundaries of the form.

Pseudocode:

Retrieve cube state data, algorithm solution, category. From Training table

Display cube loaded from state data

Playing = true

While Playing:

if no more moves left & playerconfig.cycle is true:

 reload cube state data

else if there are more moves:

 step to next move

wait time interval from playerconfig.interval

PLAY button pressed:

 playing = true

PAUSE button pressed:

 Playing = false

STEP FORWARD button pressed:

 display next move

STEP BACKWARD button pressed:

 display currentMove prime

Features:

Cube Display

Rotation Buttons

Move Display

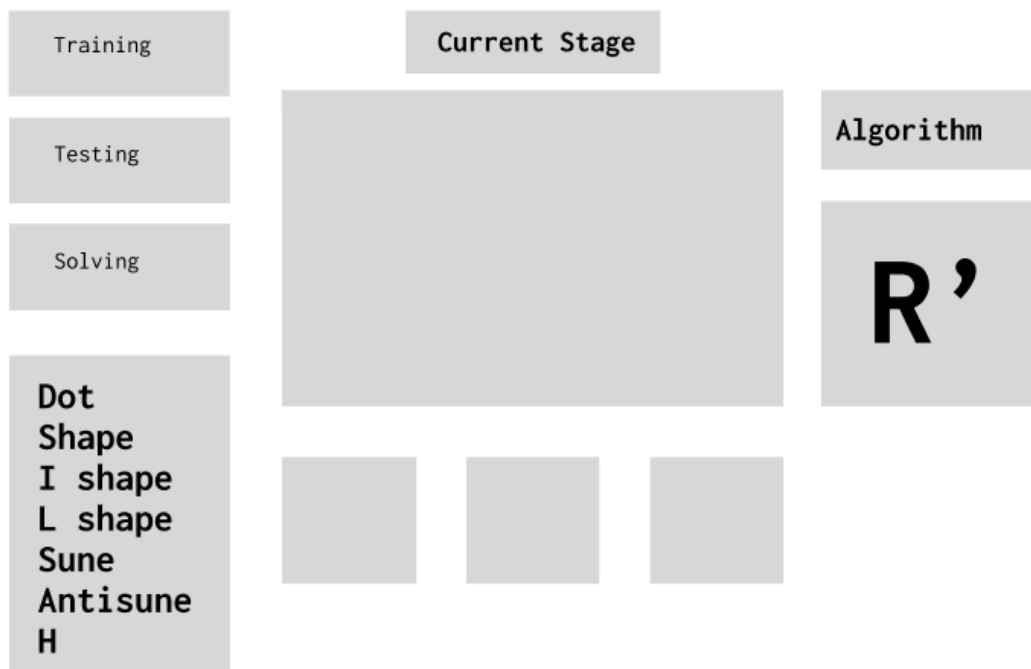
Algorithm Display

Navigation to other modules

Algorithm Navigation I.e play/pause

Rubik's Cube Solver & Trainer

User Interface Design:

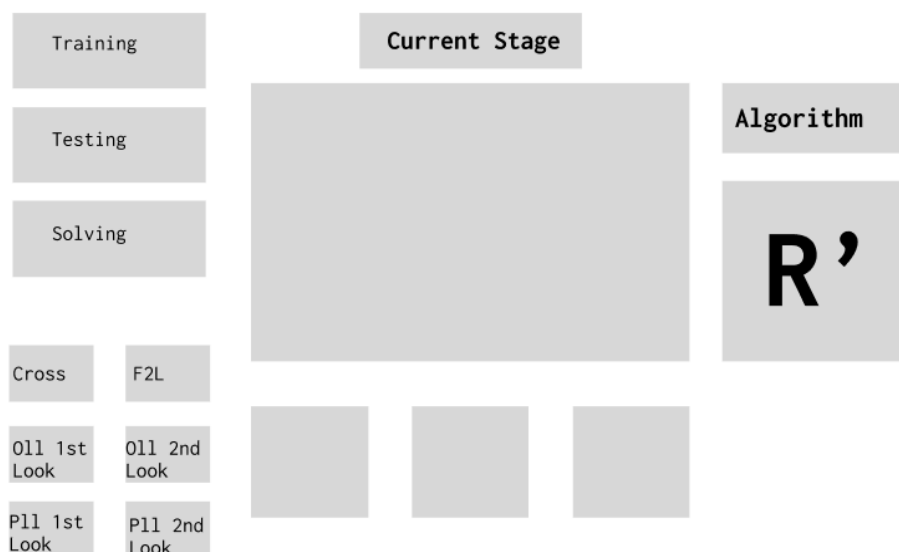


Dot, shape, I shape, L shape... Are placeholders for a dynamic button table for users to select algorithms to analyse. It will be a tiered menu. The surface will have the different stages. For CFOP: F2l, OLL, PLL. For Beginner: Second Layer, Yellow Cross, Yellow edges, permute corners, orient corners.

Pressing any of these buttons will then clear the table and set a new grid of buttons for each algorithm in those categories. The ones given in the design are some of the CFOP OLL algorithms

Solving

In the solving module, 6 buttons are added to the bottom left corner of the form. Each button corresponds to a stage of the CFOP method. An example work flow would be scrambling the cube, white cross, f2l, oll 1st look, oll 2nd look, pll 1st look, pll 2nd look



Rubik's Cube Solver & Trainer

Description of Algorithms

The Markov-Chain Algorithm

The Markov-Chain Algorithm will be implemented as a scrambling algorithm for the rubiks cube. The algorithm works for scrambling a rubik's cube because it ensures that the next move doesn't directly undo the previous one (e.g., avoid **R** followed by **R'**).

Input: The number of moves to generate.

Move Iteration:

1. **Randomly** choose a legal move from a predefined array
2. **Compare** this move to the previously generated move
3. If this move is the **counterpart** of the previous move, **skip** this generation
4. Else **apply** the **move** and add it to the scramble sequence

Output: Return an array of the scramble moves in the order they were performed.

Implementation of the Markov-Chain algorithm in pseudo code

```
FUNCTION markovChainScramble(num_moves):
moves ← ['U', 'U'', 'U2', 'R', 'R'', 'R2', 'F', 'F'', 'F2', 'L',
'L'', 'L2', 'D', 'D'', 'D2', 'B', 'B'', 'B2']
cube ← Cube()
scramble_sequence ← []

current_move ← random.choice(moves)
scramble_sequence.append(current_move)
cube.apply_move(current_move)

for i ← 1 to num_moves:
    next_move = random.choice(moves)
    IF are_opposite(current_move, next_move) THEN
        i = i - 1
    ELSE
        cube.apply_move(next_move)
        scramble_sequence.append(next_move)
        current_move = next_move
ENDFOR

PRINT(scramble_sequence)
```

This algorithm has a time complexity of $O(n)$, where n is the number of moves to attempt to generate.

Check if two moves are inverse

In order to form an effective scrambling algorithm, A check is required to see whether a move will undo the previous one. I am using numbers 0-5 to index the faces on which rotations occur and 6-11 for the anticlockwise variant. This means that, to check if two moves are opposite, get the first move and add 6 to it, check if this is equal the second move. As well as getting the first move and removing 6 from it, check if this is equal to the second move. If either of this statements are true, the moves are opposite each other.

```
areOpposite = false
IF move1 - 6 = move2 OR move1 + 6 = move2
    THEN areOpposite = true
return areOpposite
```

Rubik's Cube Solver & Trainer

Draw Cube

To form a visually appealing cube that is easy to understand, I will be creating a 2D projection of a 2D cube. This will be an isometric view of a cube as illustrated below.

$$y = 0.55x \quad \{0 < x < 9\}$$

$$y = 0.55x + 10 \quad \{-9 < x < 0\}$$

$$y = 0.55x - 10 \quad \{0 < x < 9\}$$

$$y = -0.55x \quad \{-9 < x < 0\}$$

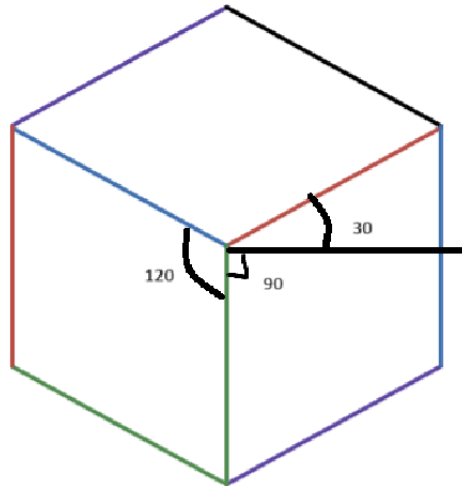
$$y = -0.55x + 10 \quad \{0 < x < 9\}$$

$$y = -0.55x - 10 \quad \{-9 < x < 0\}$$

$$x = 0 \quad \{-10 < y < 0\}$$

$$x = -9 \quad \{-5 < y < 5\}$$

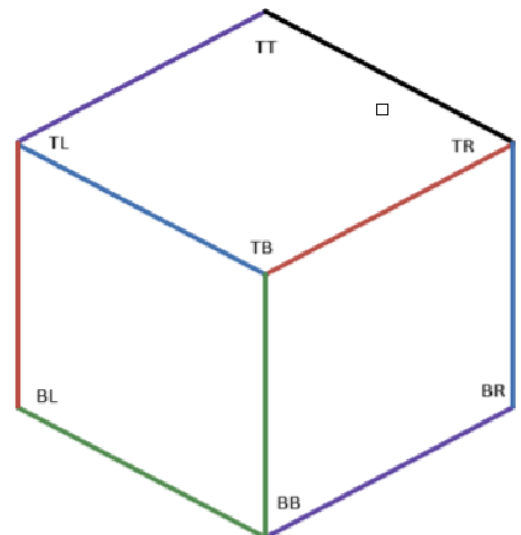
$$x = 9 \quad \{-5 < y < 5\}$$



To find the gradients of the lines I have used trigonometry. Since $\sin(x) = \text{opposite/hypotenuse}$, and $\sin(30) = \frac{1}{2}$, it is shown that the ratio of opposite/hypotenuse is $\frac{1}{2}$. Since gradient is the rate of change of y with respect to x , the gradient of these lines should be $\frac{1}{2}$. However, after discussing this with the end-user, we both agreed that it appeared off when mathematically correct. I came to the conclusion that adding a small factor of 0.05 to the gradient helped make the cube look more natural despite being incorrect.

Using these equations, it is possible to compute the coordinates of points of interest.

```
Gradient = 0.55
Size = 129
TL = 0, size
TT = TL.x + size, TL.y - (size
*gradient)
TR = TT.x + size, TT.y - (size
*gradient)
TB = TL.x + size, TL.y - (size
*gradient)
BL = TL.x, TL.y + size
BB = TB.x, TB.y + size
BR = TR.x, TR.y + size
```



Rubik's Cube Solver & Trainer

Drawing Cube Stickers

Using the primary coordinates calculated previously, I can iterate through 3 times to find the subordinates of each sticker. Each step through this recursion will calculate the 4 vertices of each sticker for the three faces displayed to the user.

```
FOR i -> 3
  FOR j -> 3
    leftFaceTL = (TL.x + j * size, TL.y + j * size *
gradient + i * size)
    leftFaceTR = (leftFaceTL.x + size, leftFaceTL.y + size *
gradient)
    leftFaceBL = (leftFaceTL.x, leftFaceTL.y + size)
    leftFaceBR = (leftFaceTR.x, leftFaceTR.y + size)

    rightFaceTL = (TB.x + (j * size), TB.y - (j * size *
gradient) + (i * size))
    rightFaceTR = (rightFaceTL.x + size, rightFaceTL.y -
(size * gradient))
    rightFaceBL = (rightFaceTL.x, rightFaceTL.y + size)
    rightFaceBR = (rightFaceTR.x, rightFaceTR.y + size)

    topFaceT = (TT.x + (j * size) - (i * size), TT.y + (j *
size * gradient) + (i * size * gradient))
    topFaceL = (topFaceT.x - size, topFaceT.y + (size *
gradient))
    topFaceB = (topFaceT.x, topFaceT.y + (1 * size) + 4)
    topFaceR = (topFaceL.x + (2 * size), topFaceL.y)

Brush = getColourOfPiece(pieceLeft[x,y])
fillPolygon(brush,
  leftFaceTL.x, leftFaceTL.y
  leftFaceTR.x, leftFaceTR.y
  leftFaceBR.x, leftFaceBR.y
  leftFaceBL.x, leftFaceBL.y)
Brush = getColourOfPiece(pieceRight[x,y])
fillPolygon(brush,
  rightFaceTL.x, rightFaceTL.y
  rightFaceTR.x, rightFaceTR.y
  rightFaceBR.x, rightFaceBR.y
  rightFaceBL.x, rightFaceBL.y)
Brush = getColourOfPiece(pieceTop[x,y])
fillPolygon(brush,
  topFaceTL.x, topFaceTL.y
  topFaceTR.x, topFaceTR.y
  topFaceBR.x, topFaceBR.y
  topFaceBL.x, topFaceBL.y)
  ENDFOR
ENDFOR
```

Rubik's Cube Solver & Trainer

Drawing Cube Edges

Using the coordinates found previously, lines can be drawn iteratively between the points in a certain order to correctly draw the edges.

```
FOR i = 0, i < 4, i++
  DrawLine(Black, TL.x + (i * size), TL.y + (i * size * gradient),
    TT.x + (i * size), TT.y + (i * size * gradient))

  DrawLine(Black, TL.x + (i * size), TL.y - (i * size * gradient),
    TB.x + (i * size), TB.y - (i * size * gradient))

  DrawLine(Black, TL.x + (i * size), TL.y + (i * size * gradient),
    BL.x + (i * size), BL.y + (i * size * gradient))

  DrawLine(Black, TB.x + (i * size), TB.y - (i * size * gradient),
    BB.x + (i * size), BB.y - (i * size * gradient))

  DrawLine(Black, TL.x, TL.y + (i * size), TB.x, TB.y + (i * size))
  DrawLine(Black, TB.x, TB.y + (i * size), TR.x, TR.y + (i * size))
ENDFOR
```

Rotating Cube Faces

In order to rotate a face, two 2D arrays will store 1. The indices of adjacent faces 2. The indices of the affected tiles on each face. A face rotation will consist of two steps. Rotating the face's tiles, and rotating the adjacent edges

To rotate the faces' tiles, a piece will move to the next index of the same type of piece.

When rotating clockwise: 0 -> 2, 2 -> 8, 6 -> 0, 8 -> 6
1 -> 3, 3 -> 7, 5 -> 1, 7 -> 5

And the same occurs but in reverse for anticlockwise

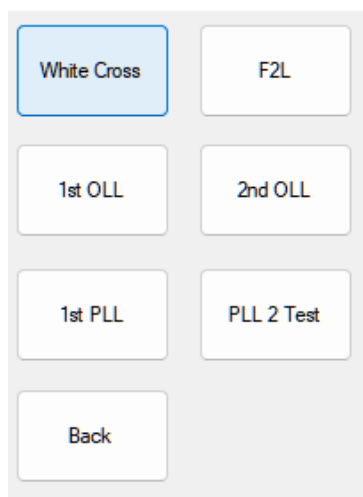
```
FUNCTION RotateFaceTiles(move, anticlockwise)
  newFace = ARRAY[9]
  FOR(i -> 9)
    int x = i % 3
    int y = i / 3
    IF(Anticlockwise)
      THEN newX = y
        newY = 2 - x
    ELSE
      THEN newX = 2 - y
        newY = x
    ENDIF
    newFace[newX + 3 * newY] = faces[faceIndex, i]
  ENDFOR
  FOR(i -> 9)
    faces[faceIndex, i] = newFace[i]
  ENDFOR
ENDFUNCTION
```

To rotate the adjacent edges, the face index will be used to get the array of adjacent faces from the 2D array. The face index will also be used to get the array of affected edge tile indices. These will be iterated over and rotated as by defined in those arrays

```
FUNCTION RotateAdjacentEdges(faceIndex, AntiClockwise)
```

Rubik's Cube Solver & Trainer

```
adjFaces = adjacentFaces[faceIndex]
adjEdges = edgeIndexes[faceIndex]
tile[] temp = new tile[3];
FOR(int i = 0; i < 3; i++)
    temp[i] = faces[adjFaces[IF(Anticlockwise) THEN 0 ELSE 3]],
                adjEdges[(IF(Anticlockwise) THEN 0 ELSE 9) + i]]
ENDFOR
IF (AntiClockwise)
    THEN
        FOR(i -> 3)
            FOR(j -> 3)
                faces[adjFaces[i], adjEdges[i * 3 + j]] =
                    faces[adjFaces[i + 1], adjEdges[(i + 1) * 3 + j]]
            ENDFOR
        ENDFOR
    ELSE
        THEN
            FOR(i = 3 -> 0)
                FOR(j -> 3)
                    faces[adjFaces[i], adjEdges[i * 3 + j]] =
                        faces[adjFaces[i - 1], adjEdges[(i - 1) * 3 + j]]
                ENDFOR
            ENDFOR
        FOR(i -> 3)
            faces[adjFaces[IF(anticlockwise) THEN 3 ELSE 0],
                adjEdges[(IF(anticlockwise) THEN 9 ELSE 0) + i]]
                = temp[i]
        ENDFOR
    ENDFUNCTION
```



Determining human steps of solution

The steps will be divided into the respective CFOP steps, having 1 button for each stage.

This makes it easier for the user to navigate through the different stages rather than guessing which one they are on.

This will require the user to step through the stages, stage by stage, otherwise the solver will not be able to find a solution

IDA* Algorithm (white cross)

When applying IDA* to solve the white cross on the Rubik's Cube, we are trying to find the minimal sequence of moves that places the white cross in the correct configuration. Here's how IDA* would be used:

```
FUNCTION IDAStar(startCube):
    threshold = startCube.Heuristic()
    solutionPath = new List(int)
    WHILE(true):
        Temp = DepthLimitedSearch(startCube, 0, threshold)
        IF temp == maxValue THEN RETURN null
        IF temp == -1 THEN RETURN solutionPath
        threshold = temp
    ENDWHILE
```

- **Initial State:** The starting state is the Rubik's Cube with the white cross pieces not yet correctly placed. The solver's goal is to position the white edge pieces around the white centre and consequently form a white cross.
- **Goal State:** The goal state is when the white cross is solved, meaning:
 - The white pieces are on the white face.
 - Each white edge is correctly aligned with the centre of the adjacent coloured faces.
- **Heuristic:** The heuristic function estimates the number of moves or steps required to solve the white cross from the current configuration, guiding the search toward the solution. The heuristic for the white cross would be:
 - **Piece Positioning:** How far the white edges are from their correct positions.
 - **Piece Orientation:** Whether the white edges are oriented correctly (matching the adjacent centres).
 - Since it has been proven for any white cross to be solved in 8 moves or less, my heuristic function will include this fact.
 - It will find how many white pieces are in the right position and subtract this from 8.
 - Whilst this isn't the best estimate for the minimum number of moves, due to their only being a maximum of 8 required, even in the worst case the algorithm will take no more than a couple of seconds to calculate.
 - ```
FUNCTION Heuristic()
 CorrectPieces = 0
 FOR i = 0, i < 9 i++
 IF faces[whiteFace, i].colour == W
 THEN correctPieces++
 ENDIF
 ENDFOR
 Return 8 - CorrectPieces
```

# Rubik's Cube Solver & Trainer

- **Search Process:**

- The **depth-limited search** starts by exploring the cube from its initial state. The search is limited by the heuristic (a number of moves), which is the upper bound for how many moves the algorithm will consider.
- It keeps exploring deeper configurations, pruning invalid paths that exceed the current threshold.
- As it deepens, the threshold increases based on the heuristic, until the white cross is solved.
- Since this algorithm is essentially finding the shortest path between a root and a node on a tree, it makes sense to code it using a recursive structure.
  - Recursion naturally fits the task of tracing a path along a tree and makes keeping track of variables such as the current amount of moves much simpler
  - The main issue with using a recursive method is that it breaks if the call stack fills. However, since the most amount of moves the algorithm finds should be around 8 moves, only 8 elements will be added to the call stack and this should not be enough to cause any issues.
- ```
FUNCTION DepthLimitedSearch(node, currentCost, threshold):
    estimatedCost = currentCost + node.Heuristic()
    IF estimatedCost > threshold THEN RETURN estimatedCost
    IF node.WhiteCrossSolved == true THEN RETURN -1
    Minimum = maxValue
    FOREACH move in node.PossibleMoves
        Child = ApplyMove(node, move)
        SolutionPath.Add(move)
        temp = DepthLimitedSearch(child, currentCost + 1, threshold)
        IF temp == -1 THEN RETURN -1
        IF temp < minimum THEN minimum = temp
        solutionPath.RemoveLastMove()
    ENDFOREACH
    RETURN minimum
```
- ```
FUNCTION ApplyMove(node, move):
 nextState = node.Clone()
 Anticlockwise = false
 IF move > 6
 THEN move = move - 6 Anticlockwise = true
 ENDIF
 nextState.RotateFace(move, Anticlockwise)
 RETURN nextState
```

- **Incremental Search:**

- Initially, the search might focus on correcting just one or two pieces of the white cross.
- If it doesn't reach the goal, the threshold increases, and the search considers deeper or more complex moves (which might involve multiple pieces).
- Each iteration of the search looks for a shorter path or more efficient way to reach the goal, gradually improving its solution.

# Rubik's Cube Solver & Trainer

- **Solution Found:** The algorithm terminates when it finds a valid solution (a series of moves) that positions all the white edges correctly, or when it exhausts all possibilities (which theoretically shouldn't happen, since IDA\* is guaranteed to find a solution if one exists).

## First 2 Layers

The first 2 layers inserts the white corner pieces into their correct positions as well as filling in the edge pieces on the second layer. There are 41 cases that each of the 4 pairs can be in after pre processing.

### Pre-Processing

For a pair to be in one of the 41 cases, both the edge and corner pieces must be in a certain set of places. The corner piece must be in the column that it will be placed in after solving. The edge piece must be either in that column or on one of the two adjacent faces, where the front facing edge is the same colour as that face.

If the edge is not in the top layer or column a right-hand move algorithm, more commonly known as a sexy move, will get the edge out of the wrong column and into the top layer.

### Main F2L

Due to the large number of algorithms compared to the other stages, this solver will use a brute forcing method to figure out the correct case and algorithm. Foreach of the pairs, pre processing will be applied. Then every f2l algorithm will be applied sequentially to a clone of the cube. If the final move results in the pair being solved, the brute force algorithm will return the required moves and move on to the next pair.

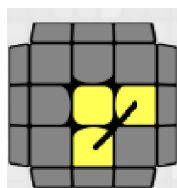
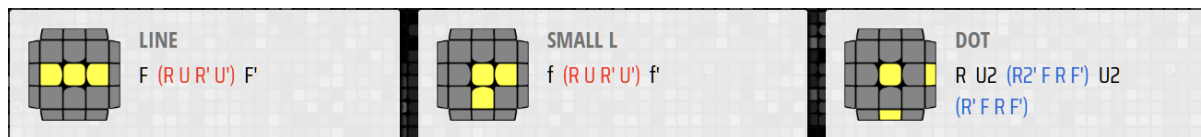
```
FUNCTION F2LBruteForcee(Cube, pairnumber)
Solution = List<int>
Cube = getEdgeInPosition(cube, pairnumber, ref solution)
Cube = getCornerInPosition(cube, pairnumber, ref solution)

Clone = cube.Clone()
FOREACH(moves IN F2LAlgorithms)
 IF(Clone.F2LPairSolved(pairnumber))
 THEN RETURN solution
 ENDIF
 Clone = Cube.Clone()
 FOREACH(Move IN Moves)
 cube.ApplyMove(Move)
 solution.Add(Move)
 ENDFOREACH
 IF(Clone.F2LPairSolved(pairnumber))
 RETURN SOLUTION
ELSE
 RETURN null
```

## First Look OLL

The first look of OLL turns the yellow face into a yellow cross. There are 4 states the cube can be in at this stage: dot, line, small L, and solved.

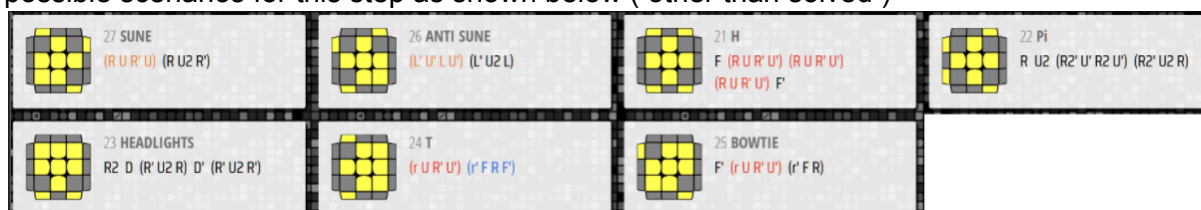
# Rubik's Cube Solver & Trainer



First the system will try to find any diagonal pairs such as the case depicted in the image. If any of these are found ( a diagonal pair but not in the same orientation ) the algorithm will rotate the top layer until it looks the same as 'Small L' at which point it performs the algorithm as shown. If no diagonal pairs are found, we check for a yellow line and orient this correctly if found. If neither of these are found, it must be in the dot case and so the dot algorithm is performed.

## Second Look OLL

The second look of OLL turns the yellow cross into a completed yellow face. There are 8 possible scenarios for this step as shown below ( other than solved )

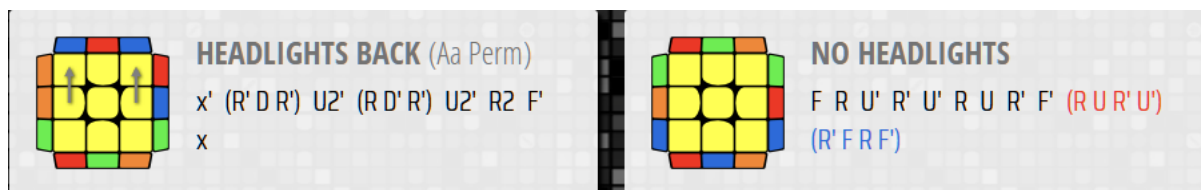


The algorithm first counts how many yellow corners are on the yellow face. If there are 0, it must be H or Pi. At this point it determines whether it is the H case by checking if the 2 opposite faces have yellow corners in their 0 and 2 index.

If there is 1 corner in the yellow face, it must be Sune or Antisune. It goes through and checks if 2 stickers on index 2 are yellow. If any more than 1 is yellow, it must be Sune, else Antisune.

Finally, if 2 stickers are yellow, it could be one of three cases: Bowtie, T or headlights. First check if Bowtie by checking for a yellow diagonal of 3. If there is a diagonal it must be Bowtie, Else it is T or headlights. It checks whether stickers in index 0 and 2 on any face (apart from yellow) are both yellow, if so it is headlights, else it is T.

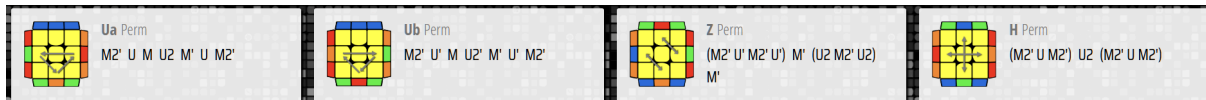
## First Look PLL



There are only 2 cases for the first look of PLL. Using the same algorithm as the second stage of OLL, it can search for how many headlights are found. If none, then use the no headlights algorithm. If 1, use the headlights back algorithm (after rotating to correct orientation) if 4, first look is already solved.

# Rubik's Cube Solver & Trainer

## Second Look PLL



There are now 4 cases for the second look of PLL. The algorithm checks for whether it is a U permutation. It does this by checking how many rows of solid colour are on each of the 4 edge faces. For example, in Ua, there is 1 row of colour on the back face. If there is only 1 correct row, it is a U Perm. At this point the correct row is oriented to the back face, and then it checks whether the middle edge on the front face is the same colour as a side edge on the left face. If so it is Ua, else it is Ub.

If there are no rows of colour, the algorithm checks if the middle edge on two opposite faces are the same colour. If so it is a Z perm, else it is H perm

If there are 4 solid rows of colour, the final stage is complete and just needs rotating to match the side colours.]

## Save Cube / Load Cube

Each of the 6 faces on the cube are comprised of 9 tiles, foreach tile on the cube, the colour will be written into a text file. At the end of each face, the writer will continue to the next line resulting in 6 lines of 9 colours. A default cube on the default orientation will appear like this:

```
rrrrrrrrr
ooooooooo
ggggggggg
bbbbbbbbb
wwwwwwwww
yyyyyyyyy
```

```
FUNCTION Save (Filename):
 File = Open(Filename)
 FOR(x = 0 -> 5):
 FOR(y = 0 -> 8):
 Write(faces[x,y].colour)
 ENDFOR
 File.WriteLine
 ENDFOR
FUNCTION Load (Filename):
 Cube = new Cube
 File = Open(Filename)
 FOR(x = 0 -> 5):
 FOR(y = 0 -> 8):
 Colour = Read()
 IF(Colour = /n OR Colour = /r)
 Colour = Read()
 Faces[x,y].colour = colour
 ENDFOR
 ENDFOR
 RETURN cube
```

## Trainer Algorithms

Upon entering the training mode, after an algorithm has been selected from the dropdowns, a new trainer will be instantiated with a cube and the algorithm id selected. Along with this, a timer will be created with a 1000 millisecond interval. The trainer will query the Algorithms

# Rubik's Cube Solver & Trainer

database for the charMoves field where the AlgID is the same as the AlgID passed into the constructor. It will then reverse the moves and apply them to a solved cube. This will put the cube into a case where that algorithm would be required to be used as seen in the diagram below.

After pressing the play button, the timer will begin. With the 1000ms interval, it will call for the next move of the algorithm to be played out. This will continue until all moves are played out or the user presses pause.

```
Timer.Interval = 1000 ms
```

```
EVENT dropdownChanged:
Cube = new Cube
Cube.RotateDisplayDown()
Cube.RotateDisplayDown()
AlgID = SelectedIndex
Trainer = new Trainer(AlgID, Cube)
Trainer.DisplayAlgorithm()
Timer.pause()
HeaderText = Trainer.Name
Display.Invalidate
ENDEVENT

FUNCTION ReverseMovesOrder(moveList)
reverseMovesStack = new stack(moveList.size)
FOREACH move IN moveList:
 reverseMoves.Push(move)
ENDFOREACH
reversedMovesList = new list()
WHILE NOT reverseMovesStack.IsEmpty():
 Move = reverseMovesStack.Pop()
 ReversedMoves.Add(Move)
ENDWHILE
ENDFUNCTION

FUNCTION InvertMoves(ReversedMovesList):
InvertedMoves = new list
FOREACH move in ReversedMovesList:
 IF move > 17 THEN move = move - 6
 ELSE IF move > 11 THEN move = move + 6
 ELSE IF move < 6 THEN move = move + 6
 ELSE THEN move = move - 6
 ENDIF
 InvertedMoves.Add(move)
ENDFOR
RETURN InvertedMoves
ENDFUNCTION
```

**Log in** [NIST SHA-256 publication](#)

```
username = txtUsername.Text
password = txtPassword.TExt
```

# Rubik's Cube Solver & Trainer

```
IF username.Length < 4 OR password.Length < 6
 THEN MessageBox.Show("Username must be at least 4 characters
 \nPassword must be at least 6
characters")
ELSE
 THEN
 # Get password and salt from the database
 _sSqlString = "SELECT UserPassword, salt " +
 "FROM Users " +
 $"WHERE Username = '{username}';"
 dt = ExecuteSqlQuery(_sSqlString)
 UserPassword = dt.rows[0][0]
 UserSalt = dt.rows[0][1]
 IF UserPassword == null
 THEN
 IF (MessageBox.Show("Username was not found in the
 database, would you like to create
 an account?",
 "Create account",
 MessageBoxButtons.YesNo,
 MessageBoxIcon.Question)
 == DialogResult.Yes)
 THEN # user agrees to create account
 salt = GenerateSalt()
 hashed = Sha256Hasher.ComputeHash(password +
salt)
 _sSqlString = "INSERT INTO Users" +
"(Username, UserPassword, Salt)" +
"VALUES" +
$"('{username}', '{hashed}', '{salt}');"
 ExecuteSqlQuery(_sSqlString) # create user
 ENDIF
 ENDIF
 hashedPassword = ComputeHash(password + UserSalt)
 IF hashedPassword = UserPassword
 THEN
 MessageBox.Show("Log in Successful!")
 CurrentUser = username
 FOREACH control in Controls :
 IF control.Tag != null && control.Tag.ToString()
 == "MainItems"
 THEN
 control.Visible = true
 ENDIF
 IF control.Tag != null && control.Tag.ToString()
 == "Log In"
 THEN
 control.Visible = false
 ENDIF
 Size = new System.Drawing.Size(953, 463)
 btnBack_Click(sender, e)
 ELSE MessageBox.Show("Incorrect Username or Password")
 ENDIF
 ENDIF
 ENDIF
```

# Rubik's Cube Solver & Trainer

## Salt Generator

A random set of 16 characters is generated for every new user as an added protection against brute force attacks using a precomputed hash table.

```
FUNCTION GenerateSalt():
CONSTANT validCharacters =
"abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789"
salt = ""
random = new Random()
FOR i -> 16
 salt += validCharacters[random.Next(validCharacters.Length)]
ENDFOR
RETURN salt
ENDFUNCTION
```

## Preprocessing

As said in the NIST publication, the parsed data must be first padded to ensure the message is a multiple of 512 bits. More specifically, for SHA-256:

At the end of the message, append the bit '1' followed by k lots of '0' bits.

K is the smallest non negative solution to the equation  $k = 447 - \text{bit length of data}$

Then append the 64 bit binary representation of the length of the data.

For example:

Let the data be an 8 bit ASCII string "123"

This has length  $3 * 8 = 24$ .

Therefore, the message is padded with a '1' then  $447 - 24 = 423$ , '0' bits.

Then the message length 24 in 64 bit binary is 00... 011000

So the padded message becomes

|          |          |          |   |        |            |
|----------|----------|----------|---|--------|------------|
| 00110001 | 00110010 | 00110011 | 1 | 00..00 | 00..011000 |
| "1"      | "2"      | "3"      |   | 423    | 24         |

The string "123" is now padded to a length of 512 bits

```
FUNCTION PadMessage(data, bitLength):
padLength = 64 - (data.Length % 64) # get padding length
IF padLength < 9 THEN padLength += 64

BYTE ARRAY paddedData = BYTE ARRAY[data.Length + padLength]
paddedData = data.copy()
paddedData[data.Length] = 0x80

Append the original bit length as a 64-bit integer.
FOR i = paddedData.Length - 8 -> paddedData.Length:
 paddedData[i] = (bitLength >> ((paddedData.Length - 1 - i)
 * 8) & 0xff)
ENDFOR

RETURN paddedData
ENDFUNCTION
```

# Rubik's Cube Solver & Trainer

## SHA-256

The initial Hash Values 'h' are eight, 32-bit words that represent the initial hash values for the algorithm

The Round Constants 'k' are 64 predefined 32-bit constants used in every round of the compression function.

After preprocessing, the next step is to iterate through each 512-bit block, for this password system there will often only be one 512-bit block.

Each iteration:

- Create a message schedule array
- Initialise the first 16 words with the 512 bit block:
  - The first 32-bit word is constructed from the first 4 bytes of the data
  - The second word from the second set of 4 bytes (5,6,7,8)
- For the remaining 48 words of the message schedule:
  - The first word (word17) is constructed via the following
  - $\text{Sigma0} = \text{ROTR } 7(\text{word1}) \text{ XOR } \text{ROTR } 18(\text{word1}) \text{ XOR } \text{SHR } 3(\text{word1})$
  - $\text{Sigma1} = \text{ROTR } 17(\text{word1}) \text{ XOR } \text{ROTR } 19(\text{word1}) \text{ XOR } \text{SHR } 10(\text{word1})$
  - $\text{Word17} = \text{word1} + \text{Sigma0} + \text{word9} + \text{Sigma1}$
  - Repeat incrementing all words by 1, until the 64th word has been defined
- Initialise the work variables with the current hash values
  - $a = h[0]$
  - $b = h[1]$
  - $c = h[2]$
  - $d = h[3]$
  - $e = h[4]$
  - $f = h[5]$
  - $g = h[6]$
  - $hh = h[7]$
- Main Compression Loop:
  - For each of the 64 words:
  - $\text{SIGMA1} = \text{ROTR } 6(\text{word}) \text{ XOR } \text{ROTR } 11(\text{word}) \text{ XOR } \text{ROTR } 25(\text{word})$
  - $\text{Ch}(e, f, g) = (e \text{ AND } f) \text{ XOR } ((\text{NOT } e) \text{ AND } g)$
  - $\text{Temp1} = hh + \text{SIGMA1} + \text{Ch} + k[\text{wordNum}] + \text{word}$
  - $\text{SIGMA0} = \text{ROTR } 2(\text{word}) \text{ XOR } \text{ROTR } 13(\text{word}) \text{ XOR } \text{ROTR } 22(\text{word})$
  - $\text{Maj}(a, b, c) = (a \text{ AND } b) \text{ XOR } (a \text{ AND } c) \text{ XOR } (b \text{ AND } c)$
  - $\text{Temp2} = \text{SIGMA0} + \text{Maj}$
  - $hh = g$
  - $g = f$
  - $f = e$
  - $e = d + \text{temp1}$
  - $d = c$
  - $c = b$
  - $b = a$
  - $a = \text{temp1} + \text{temp2}$
- Add the compressed set to the current hashes
  - $h[0] += a$
  - $h[1] += b$
  - $h[2] += c$
  - $h[3] += d$
  - $h[4] += e$
  - $h[5] += f$
  - $h[6] += g$
  - $h[7] += hh$
- Return the hash values as a hexadecimal string representation.

# Rubik's Cube Solver & Trainer

```
FUNCTION ComputeHASH(password):
UINT ARRAY h =
{
 0x6a09e667, 0xbb67ae85, 0x3c6ef372, 0xa54ff53a,
 0x510e527f, 0x9b05688c, 0x1f83d9ab, 0x5be0cd19
}

UINT ARRAY k =
{
 0x428a2f98, 0x71374491, 0xb5c0fbcf, 0xe9b5dba5, 0x3956c25b,
 0x59f111f1, 0x923f82a4, 0xab1c5ed5,
 0xd807aa98, 0x12835b01, 0x243185be, 0x550c7dc3, 0x72be5d74,
 0x80deb1fe, 0x9bdc06a7, 0xc19bf174,
 0xe49b69c1, 0xefbe4786, 0x0fc19dc6, 0x240ca1cc, 0x2de92c6f,
 0x4a7484aa, 0x5cb0a9dc, 0x76f988da,
 0x983e5152, 0xa831c66d, 0xb00327c8, 0xbf597fc7, 0xc6e00bf3,
 0xd5a79147, 0x06ca6351, 0x14292967,
 0x27b70a85, 0x2e1b2138, 0x4d2c6dfc, 0x53380d13, 0x650a7354,
 0x766a0abb, 0x81c2c92e, 0x92722c85,
 0xa2bfe8a1, 0xa81a664b, 0xc24b8b70, 0xc76c51a3, 0xd192e819,
 0xd6990624, 0xf40e3585, 0x106aa070,
 0x19a4c116, 0x1e376c08, 0x2748774c, 0x34b0bcb5, 0x391c0cb3,
 0x4ed8aa4a, 0x5b9cca4f, 0x682e6fff3,
 0x748f82ee, 0x78a5636f, 0x84c87814, 0x8cc70208, 0x90befffa,
 0xa4506ceb, 0xbef9a3f7, 0xc67178f2
}

BYTE ARRAY data = ConvertStringToBytes(password)
bitLength = data.Length * 8
Data = PadString(data, bitLength)

FOR i -> data.Length i = i + 64:

 UINT ARRAY w = new UINT ARRAY(64)

initialise first 16 words
 FOR j -> 16:
 w[j] = (data[i + 4 * j] << 24)
 | (data[i + 4 * j + 1] << 16)
 | (data[i + 4 * j + 2] << 8)
 | data[i + 4 * j + 3])
 ENDFOR

extend the 16 words into the other 48 words of the schedule
array
 FOR j = 16 -> 64:
 UINT s0 = RightRotate(w[j - 15], 7)
 ^ RightRotate(w[j - 15], 18)
 ^ (w[j - 15] >> 3)
 UINT s1 = RightRotate(w[j - 2], 17)
 ^ RightRotate(w[j - 2], 19)
 ^ (w[j - 2] >> 10)
 w[j] = w[j - 16] + s0 + w[j - 7] + s1
 ENDFOR

initialise working variables with current hash values
```

# Rubik's Cube Solver & Trainer

```
UNIT a = h[0], b = h[1], c = h[2], d = h[3], e = h[4],
 f = h[5], g = h[6], hh = h[7]

compression loop
FOR j -> 64:
 UINT S1 = RightRotate(e, 6)
 ^ RightRotate(e, 11)
 ^ RightRotate(e, 25)
 UINT ch = (e & f) ^ ((~e) & g)
 UINT temp1 = hh + S1 + ch + k[j] + w[j]
 UINT S0 = RightRotate(a, 2)
 ^ RightRotate(a, 13)
 ^ RightRotate(a, 22)
 UINT maj = (a & b) ^ (a & c) ^ (b & c)
 UINT temp2 = S0 + maj

 hh = g g = f f = e e = d + temp1
 d = c c = b b = a a = temp1 + temp2
ENDFOR

add compressed chunk to hash values
h[0] += a h[1] += b h[2] += c h[3] += d
h[4] += e h[5] += f h[6] += g h[7] += hh
ENDFOR
Convert hash values to hexadecimal string.
RETURN ConvertToHexArrayToString(h)
ENDFUNCTION
```

## Solvability

There are 3 checks to determine whether a cube state is solvable. If any of these checks fail, the cube is in some malformed state that should not be possible.

```
FUNCTION IsCubeSolvable(cube):
IF(!CheckColorCount(cube))
 THEN RETURN false
ENDIF
IF(!CheckCornerOrientation(cube))
 THEN RETURN false
ENDIF
IF(!CheckEdgeOrientation(cube))
 THEN RETURN false
ENDIF
IF(!CheckPermutationParity(cube))
 THEN RETURN false
ENDIF
RETURN true
ENDFUNCTION
```

## Sticker count

If there are more ( or less ) than 9 of a colour sticker, that is not a possible cube state since one piece would not exist in real life.

```
FUNCTION CheckColorCount(cube)
```

# Rubik's Cube Solver & Trainer

```
redCount = 0
orangeCount = 0
greenCount = 0
blueCount = 0
whiteCount = 0
yellowCount = 0
FOREACH(tile tile in cube.faces)
 SWITCH(tile.getColour())
 case 'r':
 redCount++
 case 'o':
 orangeCount++
 case 'g':
 greenCount++
 case 'b':
 blueCount++
 case 'w':
 whiteCount++
 case 'y':
 yellowCount++
 ENDSWITCH
ENDFOREACH
IF (redCount > 9 OR orangeCount > 9 OR greenCount > 9 OR
 blueCount > 9 OR whiteCount > 9 OR yellowCount > 9)
 THEN RETURN false
ELSE RETURN true
```

## Piece Orientation

Each of the 8 corner pieces in a Rubik's Cube has 3 possible orientations (labelled 0, 1, or 2). The number of orientations is constrained because the cube is a 3D puzzle where pieces are interconnected, and turning one face affects multiple pieces.

- **0:** The piece is oriented correctly.
- **1:** The piece is twisted 120 degrees clockwise.
- **2:** The piece is twisted 240 degrees clockwise

For any valid Rubik's Cube configuration, the sum of all corner orientations modulo 3 must be zero. This rule arises because of how pieces are rotated: twisting a corner piece affects its orientation relative to others, and the total twist across the entire cube must balance out to an even number of 120-degree twists (or multiples of 360 degrees).

Algorithmically, for a valid cube: the sum of corner orientations MOD 3 = 0

A similar approach is done for the edge pieces of a cube, but since an edge only has 2 sides: **0:** The edge is oriented correctly, **1:** the edge is flipped.

The formula for edges is also slightly different being the sum of edge orientations MOD 2 = 0

Corner Orientation logic:

```
FUNCTION CheckCornerOrientation(Cube cube)
 cornerOrientations = ARRAY [8]
 index = 0
 cornerIndexes = 0, 2, 6, 8
```

# Rubik's Cube Solver & Trainer

```
 FOREACH(i IN cornerIndexes):
 (face2, pos2, face3, pos3) = CornerEdgeLookup[(0, i)]
 color1 = cube.faces[0, i].getColour()
 color2 = cube.faces[face2, pos2].getColour()
 color3 = cube.faces[face3, pos3].getColour()
 cornerOrientations[index++] =
 DetermineCornerOrientation(color1, color2, color3)
 ENDFOREACH
 FOREACH(i IN cornerIndexes)
 (face2, pos2, face3, pos3) = CornerEdgeLookup[(1, i)]
 color1 = cube.faces[1, i].getColour()
 color2 = cube.faces[face2, pos2].getColour()
 color3 = cube.faces[face3, pos3].getColour()
 cornerOrientations[index++] =
 DetermineCornerOrientation(color1, color2, color3)
 ENDFOREACH
 cornerOrientationSum = 0
 FOR (i -> 8)
 cornerOrientationSum += cornerOrientations[i]
 ENDFOR
 IF (cornerOrientationSum % 3 != 0)
 THEN RETURN false
 ELSE RETURN true
 ENDIF
ENDFUNCTION
```

```
FUNCTION DetermineCornerOrientation(color1, color2, color3):
 IF (Color1 = r OR Color1 = o)
 THEN RETURN 0
 ELSE IF (color2 = r OR color2 = o)
 THEN RETURN 1
 ELSE RETURN 2
 ENDIF
ENDFUNCTION
```

```
FUNCTION DetermineEdgeOrientation(color1)
 IF (color1 == 'r' OR color1 == 'o')
 THEN RETURN 0
 ELSE RETURN 1
 ENDFUNCTION
FUNCTION DetermineAlternateEdgeIndex(color1)
 IF(color1 == 'w' OR color1 == 'y')
 THEN RETURN 0
 ELSE RETURN 1
 ENDFUNCTION
```

```
FUNCTION CheckEdgeOrientation(Cube cube)
 edgeOrientations = ARRAY[12]
 index = 0
 edgeIndexes = 1, 3, 5, 7
 extraEdgeIndexes = 3, 5
 FOREACH(i IN edgeIndexes) # red face
 color1 = cube.faces[0, i].getColour()
 edgeOrientations[index++] =
```

# Rubik's Cube Solver & Trainer

```
 DetermineEdgeOrientation(color1)
 ENDFOREACH
 REPEAT FOR orange, white, yellow faces
 edgeOrientationSum = 0
 FOR (i -> 12)
 edgeOrientationSum += edgeOrientations[i]
 ENDFOR
 IF(edgeOrientationSum % 2 != 0)
 THEN RETURN false // Unsolvable configuration
 ELSE RETURN true
 ENDIF
ENDFUNCTION
```

## Piece Permutation

A single move on a Rubik's cube rotates three edges and three corners. This preserves the even parity since 6 pieces are affected and this will never change.

On a valid cube, it is impossible to flip one edge alone or swap just two pieces without violating the cube's parity.

To test this validity, the position of each edge piece is stored into an array

An example array could look like: 0,5,6,2,3,1,4,7.

Then, using a cycle decomposition algorithm, the minimum number of swaps is calculated.

This is then repeated for the corner pieces.

If the total corner swaps are even, and the total edge swaps are even, then parity is preserved

If the total corner swaps are odd, and the total edge swaps are odd, then parity is preserved  
Otherwise, parity is broken

```
FUNCTION CountCycleSwaps(permutation)
 visited = new ARRAY[permutation.Length]
 swaps = 0
 FOR (i -> permutation.Length)
 IF(!visited[i])
 current = i
 int cycleSize = 0
 ENDIF
 WHILE (!visited[current])
 visited[current] = true
 current = permutation[current]
 cycleSize++
 ENDWHILE
 IF(cycleSize > 1)
 swaps += cycleSize - 1
 ENDIF
 ENDFOR
 RETURN swaps
ENDFUNCTION
```

```
FUNCTION CheckPermutationParity(cube)
 cornerPermutation = GetCornerPermutation(cube)
 edgePermutation = GetEdgePermutation(cube)
 cornerSwaps = CountCycleSwaps(cornerPermutation)
 edgeSwaps = CountCycleSwaps(edgePermutation)
```

# Rubik's Cube Solver & Trainer

```
cornerParity = (cornerSwaps % 2 == 0)
edgeParity = (edgeSwaps % 2 == 0)
RETURN cornerParity == edgeParity
ENDFUNCTION

FUNCTION GetEdgePermutation(cube)
 edgePermutation = ARRAY[12]
 index = 0
 edgeIndexes = 1, 3, 5, 7
 extraEdgeIndexes = 3, 5
 FOREACH(i IN edgeIndexes) // red face
 face2, pos2 = AdjacentEdgeLookup[(0, i)]
 color1 = cube.faces[0, i].getColour()
 color2 = cube.faces[face2, pos2].getColour()
 edgePermutation[index++] = DetermineEdgeIndex(color1,
 color2)
 ENDFOREACH
 REPEAT FOR orange, yellow, white faces
 RETURN edgePermutation
ENDFUNCTION

FUNCTION DetermineEdgeIndex(color1, color2)
 Dictionary<(char, char), int> edgeMapping = ...
 sortedColours = color1, color2
 RETURN edgeMapping[ortedColors]
ENDFUNCTION

FUNCTION GetCornerPermutation(cube)
 cornerPermutation = ARRAY[8]
 index = 0
 cornerIndexes = 0, 2, 6, 8
 FOREACH(i IN cornerIndexes)
 face2, pos2, face3, pos3 = CornerEdgeLookup[(0, i)]
 color1 = cube.faces[0, i].getColour()
 color2 = cube.faces[face2, pos2].getColour()
 color3 = cube.faces[face3, pos3].getColour()
 cornerPermutation[index++] =
 DetermineCornerIndex(color1, color2, color3)
 ENDFOREACH
 Repeat on orange face
 RETURN cornerPermutation
ENDFUNCTION

FUNCTION DetermineCornerIndex(color1, color2, color3)
 Dictionary<(char, char, char), int> cornerMapping = ...
 sortedColors = color1, color2, color3
 Array.Sort(sortedColors)
 RETURN cornerMapping[sortedColors[0], sortedColors[1],
 sortedColors[2]]
ENDFUNCTION
```

# Rubik's Cube Solver & Trainer

## Description of Data Structures

### 1D and 2D arrays

Cube = Tile[1-6, 1-9] Where 1-6 are the faces of the cube, and 1-9 for the Tiles.

A two dimensional array is used to keep track of a cube. One dimension will be used for the faces on the cube, the second dimension for the Tiles on each face. This will be used when a cube is instantiated and will keep track of the cube throughout its permutations.

### Records and Fields

### Dictionary

A use case for a dictionary will be when solving the last three stages of CFOP (FOP). During F2L solving, an edge piece and its corner pair will both need to be found. The program will find all the f2l edges and store their positions in an array, it will then go through and find the matching corner piece and store them in a different array, but using the same positions as the edge pieces. I.e if the blue-red edge is stored in position 1, the red-blue-white corner will be stored in position 1 also. Using a dictionary to store these values is the simplest way to have them. The dictionary will take a position of a sticker, and output the positions of the adjacent sticker[s]. To find whether it is the piece wanted, an algorithm will compare the colours of the retrieved positions and check whether that matches as required.

|   |   |   |
|---|---|---|
| 0 | 1 | 2 |
| 3 | 4 | 5 |
| 6 | 7 | 8 |

This algorithm is used to locate all pieces with a certain colour and position on the cube. By position, it is referring to whether a piece is an edge or a corner piece.

An edge piece would be a piece with index 1,3,5, or 7.

A corner piece would be a piece with index 0,2,6, or 8.

### Lists

Lists will be used when the item count to be stored is unknown until the end of computation. It will be used in places such as storing moves from the IDA\* searching algorithm or keeping track of the current moves to be applied to the cube.

```
FUNCTION List():
 elements = ARRAY[initialCapacity]
ENDFUNCTION

FUNCTION this[index]:
 GET
 IF(index < 0 || index >= count)
 THEN ERROR ("Index is out of bounds.")
 ENDIF
```

# Rubik's Cube Solver & Trainer

```
 RETURN elements[index]
 ENDGET
 SET
 IF (index < 0 || index >= count)
 THEN ERROR ("Index is out of bounds.")
 ENDIF
 elements[index] = value
 ENDSET
ENDFUNCTION

FUNCTION Add(element):
 IF(count == elements.Length)
 Array.Resize(ref elements, elements.Length * 2)
 ENDIF
 elements[count++] = element
ENDFUNCTION

FUNCTION Remove(element):
 index = Array.IndexOf(elements, element)
 IF(index != -1)
 FOR (i = index -> count - 1)
 elements[i] = elements[i + 1]
 ENDFOR
 Count-
 ENDIF
ENDFUNCTION

FUNCTION RemoveAt(index):
 FOR (i = index -> count - 1)
 elements[i] = elements[i + 1]
 ENDFOR
 count--
ENDFUNCTION

FUNCTION Reverse():
 Stack<T> reverseStack = new Stack<T>(count)
 FOR (i -> count)
 reverseStack.Push(elements[i])
 ENDFOR
 FOR (I -> Count)
 elements[i] = reverseStack.Pop()
 ENDFOR
ENDFUNCTION

FUNCTION Insert(index, element):
 IF(count == elements.Length)
 Array.Resize(ref elements, elements.Length * 2)
 ENDIF

 FOR(i = count -> index i-)
 elements[i] = elements[i - 1]
 ENDFOR

 elements[index] = element
 count++
ENDFUNCTION
```

# Rubik's Cube Solver & Trainer

```
FUNCTION AddRange(elementsToAdd):
 FOREACH(element IN elementsToAdd)
 Add(element)
 ENDFOREACH
ENDFUNCTION
```

```
FUNCTION IsEmpty():
 return count == 0
ENDFUNCTION
```

```
FUNCTION Count():
 return count
ENDFUNCTION
```

## Stacks

A stack will primarily be used to reverse the order of arrays or lists.

Pseudocode:

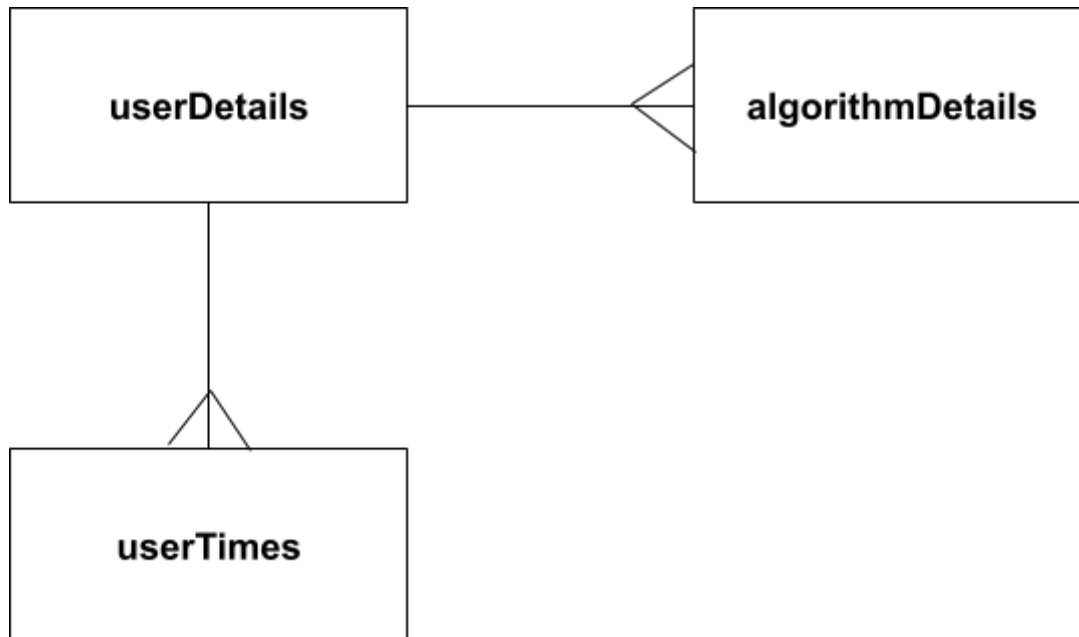
```
FUNCTION Stack(size):
 maxSize = size
 Elements = array[maxsize]
 topPointer = -1
ENDFUNCTION
FUNCTION IsEmpty():
 RETURN topPointer == -1
ENDFUNCTION
FUNCTION IsFull():
 RETURN topPointer == maxSize - 1
ENDFUNCTION
FUNCTION Push(element):
 IF(IsFull())
 THEN ERROR("Stack Overflow Error")
 ELSE
 THEN elements[topPointer++] = element
 ENDIF
ENDFUNCTION
FUNCTION Pop():
 IF(IsEmpty())
 THEN ERROR("Stack Underflow Error")
 ELSE
 THEN RETURN elements[topPointer--]
 ENDIF
ENDFUNCTION
```

## Database Design

This revised Entity-Relationship Diagram removes the need of a table for storing the users fastest time - this new design removes data redundancy. I have removed the bestTime table and converted userAverages to userTimes. Having all the users' times stored in a table

# Rubik's Cube Solver & Trainer

means the best time is queryable. By selecting all times from a user and sorting by time ascending, the first result will be the user's best time.



## Entities:

**algorithmDetails**(AlgorithmID, UserID, Title, Category, Moves)

**Users**(UserID, Username, Password, Salt)

**userTimes**(UserID, Time)

## Create statements:

### Users

```
CREATE TABLE Users (
 UserID AUTOINCREMENT NOT NULL ,
 Username VARCHAR(20) NOT NULL,
 UserPassword VARCHAR(64) NOT NULL,
 Salt VARCHAR(16) NOT NULL,
 PRIMARY KEY (UserID)
);
```

### Algorithms

```
CREATE TABLE Algorithms(
 AlgName VARCHAR(30) NOT NULL,
 AlgCat VARCHAR(15) NOT NULL,
 CharMoves VARCHAR(100) NOT NULL,
 AlgID NUMERIC(10) NOT NULL UNIQUE,
 PRIMARY KEY(AlgID)
);
```

### UserTimes

```
CREATE TABLE UserTimes(
 UserID NUMERIC(7) NOT NULL,
```

# Rubik's Cube Solver & Trainer

```
Milliseconds LONG NOT NULL
);
```

## Queries used by the program:

Get the times the user has set sorted by time ascending for the leaderboard

```
SELECT Time
FROM userTimes
WHERE UserID = UserID
ORDER BY time ASC
```

Add new time to user times when user solves cube with a timer

```
INSERT INTO UserTimes (UserID, Milliseconds)
VALUES({userID}, {millisecondUserTime});
```

Get all names of algorithms for trainer dropdowns

```
SELECT AlgName FROM Algorithms WHERE AlgCat = 'F2L';
SELECT AlgName FROM Algorithms WHERE AlgCat = 'OLL';
SELECT AlgName FROM Algorithms WHERE AlgCat = 'PLL';
SELECT AlgName FROM Algorithms WHERE AlgCat = 'Custom';
```

Get user password and salt for login

```
SELECT UserPassword, salt
FROM Users
WHERE Username = '{username}';
```

Add username, password and salt to users for account creation

```
INSERT INTO Users(Username, UserPassword, Salt)
VALUES('{username}', '{hashed}', '{salt}');
```

Get the moves for every algorithm in each CFOP category

```
SELECT CharMoves
FROM Algorithms
WHERE AlgCat = 'F2L'
ORDER BY AlgID ASC;
```

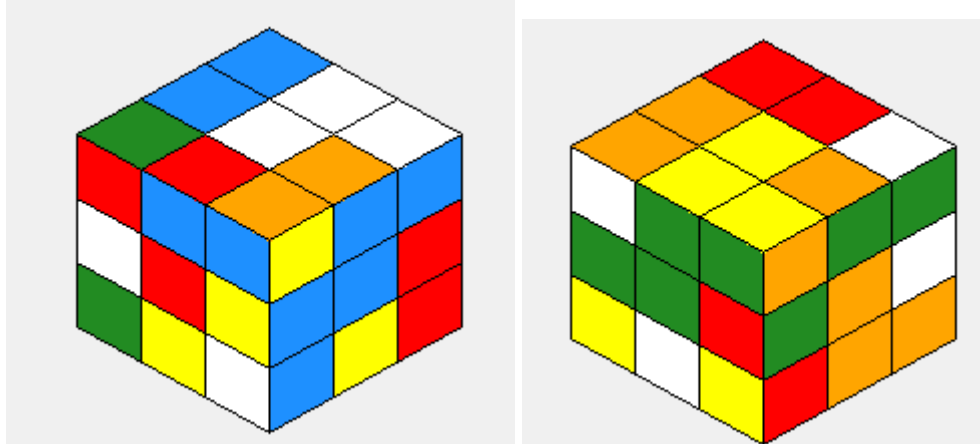
```
SELECT CharMoves
FROM Algorithms
WHERE AlgCat = 'OLL'
ORDER BY AlgID ASC;
```

```
SELECT CharMoves
FROM Algorithms
WHERE AlgCat = 'PLL'
ORDER BY AlgID ASC;
```

# Rubik's Cube Solver & Trainer

## File Design

Files will be utilised in order to save the state of a cube between different instances of the program. Each user will be able to save 1 cube that can only be accessed by that particular user. The file will consist of 6 rows; 1 row for each face of the cube. Each of these rows will contain 9 characters representing the colour of the tile on each index of the face.



In this example, the file would look like this:

```
wggggrwyw
ybbbbrbyr
wygyrwbbr
ogggowroo
rrwoyooyy
gbbrwwoww
```

For demonstration purposes, the character in bold represents the piece at the centre of each face, this makes it easier to cross reference between the image and the file example. This will not be reflected in the real file and will just consist of plaintext characters.

|                                                                                                                                                                                                    |                                                                                                                                                                                                                                              |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Input</b> <ul style="list-style-type: none"><li>• Tile colours</li><li>• Rotations</li><li>• Algorithm Addition</li><li>• Username + password</li><li>• Training selection</li><li>• </li></ul> | <b>Process</b> <ul style="list-style-type: none"><li>• Change tile colour to new colour</li><li>• Rotate face and adjacent tiles</li><li>• Add new algorithm record to the database</li></ul>                                                |
| <b>Storage</b> <ul style="list-style-type: none"><li>• Cube configuration as a file</li><li>• Algorithms in a database table</li><li>• Users in a database table</li><li>• </li></ul>              | <b>Output</b> <ul style="list-style-type: none"><li>• Update display to show cube after rotation or after recolouring.</li><li>• Login or show incorrect details / ask user to register</li><li>• Direct the user to CFOP training</li></ul> |

## Technical Solution

---

Cube.cs

---

```
using System.Drawing;
using System;
using System.Windows.Forms;
using System.IO;

namespace Rubiks_Cube_Solver_And_Trainer
{
 public class Cube
 {
 public tile[,] faces = new tile[6, 9]; // 6 faces, each with
 9 tiles
 private int[] MoveArray = new int[6] { 0, 1, 2, 3, 4, 5 }; //
 f, b, l, r, u, d
 private int[] DisplayArray = new int[6] { 0, 1, 2, 3, 4, 5 };
 // Initial order in which the Faces are displayed
 public int[] PossibleMoves = new int[12] { 0, 1, 2, 3, 4, 5,
 6, 7, 8, 9, 10, 11 }; // all 6 basic moves + their anticlockwise
 variants.
 private int[,] displayFaces = new int[6, 9]
 {
 { 0,1,2,3,4,5,6,7,8},
 { 0,1,2,3,4,5,6,7,8},
 { 0,1,2,3,4,5,6,7,8},
 { 0,1,2,3,4,5,6,7,8},
 { 0,1,2,3,4,5,6,7,8},
 { 0,1,2,3,4,5,6,7,8}
 };
 };
}
```

## Rubik's Cube Solver & Trainer

```
// Define the adjacent faces for each main face of the cube.
int[][] adjacentFaces = new int[][]
{
 new int[] {4, 3, 5, 2}, // Front face affects top,
 right, bottom, left (F)
 new int[] {5, 3, 4, 2}, // Back face (B)
 new int[] {4, 0, 5, 1}, // Left face (L)
 new int[] {4, 1, 5, 0}, // Right face (R)
 new int[] {1, 3, 0, 2}, // Top face (U)
 new int[] {0, 3, 1, 2}, // Bottom face (D)
};

// Define the specific edge indices for each adjacent face to
be rotated.
int[][] edgeIndexes = new int[][]
{
 new int[] {6, 7, 8, 0, 3, 6, 2, 1, 0, 8, 5, 2}, //F
 new int[] {6, 7, 8, 8, 5, 2, 2, 1, 0, 0, 3, 6}, //B
 new int[] {0, 3, 6, 0, 3, 6, 0, 3, 6, 0, 3, 6}, //L
 new int[] {8, 5, 2, 8, 5, 2, 8, 5, 2, 8, 5, 2}, //R
 new int[] {6, 7, 8, 2, 1, 0, 2, 1, 0, 2, 1, 0}, //U
 new int[] {6, 7, 8, 6, 7, 8, 2, 1, 0, 6, 7, 8}, //D
};

};
public Cube()
{
 InitializeCube();
}

public Cube Clone(Cube cube)
{
 Cube clone = new Cube();
 clone.faces = (tile[,])cube.faces.Clone();
 return clone;
}

private void InitializeCube()
{
 // Initialize the cube with default colors
 char[] colors = { 'r', 'o', 'g', 'b', 'w', 'y' }; //
 Front, Back, Left, Right, Top, Bottom
 //char[] colors = { 'g', 'b', 'o', 'r', 'w', 'y' };
 for (int face = 0; face < 6; face++)
 {
 for (int i = 0; i < 9; i++)
 {
 faces[face, i] = new tile(colors[face]);
 }
 }
}

public void rotateDisplayLeft()
{
 RotateHorizontalMiddleLayer();
 RotateFace(MoveArray[4], false);
}
```

## Rubik's Cube Solver & Trainer

```
 RotateFace(MoveArray[5], true);
 }
 public void rotateDisplayRight()
 {
 RotateHorizontalMiddleLayer();
 RotateHorizontalMiddleLayer();
 RotateHorizontalMiddleLayer();
 RotateFace(MoveArray[4], true);
 RotateFace(MoveArray[5], false);
 }
 public void rotateDisplayUp()
 {
 RotateVerticalMiddleLayer();
 RotateFace(MoveArray[2], true);
 RotateFace(MoveArray[3], false);
 }
 public void rotateDisplayDown()
 {
 RotateVerticalMiddleLayer();
 RotateVerticalMiddleLayer();
 RotateVerticalMiddleLayer();
 RotateFace(MoveArray[2], false);
 RotateFace(MoveArray[3], true);
 }

 public void RotateFace(int move, bool AntiClockwise)
 {
 RotateFaceTiles(MoveArray[move], AntiClockwise);
 RotateAdjacentEdges(MoveArray[move], AntiClockwise);
 }

 private void RotateFaceTiles(int faceIndex, bool
AntiClockwise)
 {
 // Rotate the 3x3 grid of the face
 tile[] newFace = new tile[9];

 for (int i = 0; i < 9; i++)
 {
 int x = i % 3;
 int y = i / 3;
 int newX = AntiClockwise ? y : 2 - y;
 int newY = AntiClockwise ? 2 - x : x;

 newFace[newX + 3 * newY] = faces[faceIndex, i];
 }

 // Copy newFace back into the original face
 for (int i = 0; i < 9; i++)
 {
 faces[faceIndex, i] = newFace[i];
 }
 }
}
```

## Rubik's Cube Solver & Trainer

```
private void RotateAdjacentEdges(int faceIndex, bool
AntiClockwise)
{
 int[] adjFaces = adjacentFaces[faceIndex];
 int[] adjEdges = edgeIndexes[faceIndex];

 // Temporary storage for edges that will be overwritten
 tile[] temp = new tile[3];
 for (int i = 0; i < 3; i++)
 {
 temp[i] = faces[adjFaces[AntiClockwise ? 0 : 3],
adjEdges[(AntiClockwise ? 0 : 9) + i]];
 }

 // Rotate edges
 if (AntiClockwise)
 {
 for (int i = 0; i < 3; i++)
 {
 for (int j = 0; j < 3; j++)
 {
 faces[adjFaces[i], adjEdges[i * 3 + j]] =
faces[adjFaces[i + 1], adjEdges[(i + 1) * 3 + j]];
 }
 }
 }
 else
 {
 for (int i = 3; i > 0; i--)
 {
 for (int j = 0; j < 3; j++)
 {
 faces[adjFaces[i], adjEdges[i * 3 + j]] =
faces[adjFaces[i - 1], adjEdges[(i - 1) * 3 + j]];
 }
 }
 }

 // Restore the overwritten edges from temp storage.
 for (int i = 0; i < 3; i++)
 {
 faces[adjFaces[AntiClockwise ? 3 : 0],
adjEdges[(AntiClockwise ? 9 : 0) + i]] = temp[i];
 }
}

public void RotateHorizontalMiddleLayer()
{
 tile[,] tempCube = (tile[,])faces.Clone();
 faces[0, 3] = tempCube[3, 3];
 faces[0, 4] = tempCube[3, 4];
 faces[0, 5] = tempCube[3, 5];
}
```

## Rubik's Cube Solver & Trainer

```
 faces[2, 3] = tempCube[0, 3];
 faces[2, 4] = tempCube[0, 4];
 faces[2, 5] = tempCube[0, 5];

 faces[1, 5] = tempCube[2, 3];
 faces[1, 4] = tempCube[2, 4];
 faces[1, 3] = tempCube[2, 5];

 faces[3, 3] = tempCube[1, 5];
 faces[3, 4] = tempCube[1, 4];
 faces[3, 5] = tempCube[1, 3];
 }

 public void RotateVerticalMiddleLayer()
 {
 tile[,] tempCube = (tile[,])faces.Clone();
 faces[0, 1] = tempCube[5, 1];
 faces[0, 4] = tempCube[5, 4];
 faces[0, 7] = tempCube[5, 7];

 faces[5, 1] = tempCube[1, 1];
 faces[5, 4] = tempCube[1, 4];
 faces[5, 7] = tempCube[1, 7];

 faces[1, 1] = tempCube[4, 1];
 faces[1, 4] = tempCube[4, 4];
 faces[1, 7] = tempCube[4, 7];

 faces[4, 1] = tempCube[0, 1];
 faces[4, 4] = tempCube[0, 4];
 faces[4, 7] = tempCube[0, 7];
 }

 private SolidBrush GetBrush(char color)
 {
 switch (color)
 {
 case 'w': return new SolidBrush(Color.White);
 case 'r': return new SolidBrush(Color.Red);
 case 'b': return new SolidBrush(Color.DodgerBlue);
 case 'y': return new SolidBrush(Color.Yellow);
 case 'o': return new SolidBrush(Color.Orange);
 case 'g': return new SolidBrush(Color.ForestGreen);
 default: return new SolidBrush(Color.Black);
 }
 }

 public char GetPieceColour(int faceIndex, int pieceIndex)
 {
 return faces[faceIndex, pieceIndex].getColour();
 }
}
```

## Rubik's Cube Solver & Trainer

```
public void drawCube(object sender, PaintEventArgs e, coord
TL, coord TT, coord TR, coord TB, coord BL, coord BB, coord BR, int
size, float gradient)
{
 Graphics g = e.Graphics;
 SolidBrush brush;
 int k = 0;

 // Iterate over the rows and columns of the Rubik's cube
 for (int i = 0; i < 3; i++)
 {
 for (int j = 0; j < 3; j++)
 {
 // Calculate coordinates for each face
 coord leftFaceTL = new coord(TL.x + j * size, TL.y
+ j * size * gradient + i * size);
 coord leftFaceTR = new coord(leftFaceTL.x + size,
leftFaceTL.y + size * gradient);
 coord leftFaceBL = new coord(leftFaceTL.x,
leftFaceTL.y + size);
 coord leftFaceBR = new coord(leftFaceTR.x,
leftFaceTR.y + size);

 coord rightFaceTL = new coord(TB.x + (j * size),
TB.y - (j * size * gradient) + (i * size));
 coord rightFaceTR = new coord(rightFaceTL.x +
size, rightFaceTL.y - (size * gradient));
 coord rightFaceBL = new coord(rightFaceTL.x,
rightFaceTL.y + size);
 coord rightFaceBR = new coord(rightFaceTR.x,
rightFaceTR.y + size);

 coord topFaceT = new coord(TT.x + (j * size) - (i
* size), TT.y + (j * size * gradient) + (i * size * gradient));
 coord topFaceL = new coord(topFaceT.x - size,
topFaceT.y + (size * gradient));
 coord topFaceB = new coord(topFaceT.x, topFaceT.y
+ (1 * size) + 4);
 coord topFaceR = new coord(topFaceL.x + (2 *
size), topFaceL.y);

 // Set the color for the left face (front face)
 brush = GetBrush(GetPieceColour(DisplayArray[0],
displayFaces[DisplayArray[0], k]));
 g.FillPolygon(brush, new PointF[4]
 {
 new PointF(leftFaceTL.x, leftFaceTL.y),
 new PointF(leftFaceTR.x, leftFaceTR.y),
 new PointF(leftFaceBR.x, leftFaceBR.y),
 new PointF(leftFaceBL.x, leftFaceBL.y)
 });
 // Set the color for the right face (side face)
 brush = GetBrush(GetPieceColour(DisplayArray[3],
displayFaces[DisplayArray[3], k]));
```

## Rubik's Cube Solver & Trainer

```
 g.FillPolygon(brush, new PointF[4]
 {
 new PointF(rightFaceTL.x, rightFaceTL.y),
 new PointF(rightFaceTR.x, rightFaceTR.y),
 new PointF(rightFaceBR.x, rightFaceBR.y),
 new PointF(rightFaceBL.x, rightFaceBL.y)
 });

 // Set the color for the top face
 brush = GetBrush(GetPieceColour(DisplayArray[4],
displayFaces[DisplayArray[4], k]));
 g.FillPolygon(brush, new PointF[4]
 {
 new PointF(topFaceL.x, topFaceL.y),
 new PointF(topFaceT.x, topFaceT.y),
 new PointF(topFaceR.x, topFaceR.y),
 new PointF(topFaceB.x, topFaceB.y)
 });
 k++;
 }
}

// Draw the outlines of the cube
DrawCubeEdges(g, TL, TT, TR, TB, BL, BB, BR, size,
gradient);
}

private void DrawCubeEdges(Graphics g, coord TL, coord TT,
coord TR, coord TB, coord BL, coord BB, coord BR, int size, float
gradient)
{
 // Drawing the cube edges
 for (int i = 0; i < 4; i++)
 {
 g.DrawLine(Pens.Black, TL.x + (i * size), TL.y + (i *
size * gradient), TT.x + (i * size), TT.y + (i * size * gradient));
 g.DrawLine(Pens.Black, TL.x + (i * size), TL.y - (i *
size * gradient), TB.x + (i * size), TB.y - (i * size * gradient));
 g.DrawLine(Pens.Black, TL.x + (i * size), TL.y + (i *
size * gradient), BL.x + (i * size), BL.y + (i * size * gradient));
 g.DrawLine(Pens.Black, TB.x + (i * size), TB.y - (i *
size * gradient), BB.x + (i * size), BB.y - (i * size * gradient));
 g.DrawLine(Pens.Black, TL.x, TL.y + (i * size), TB.x,
TB.y + (i * size));
 g.DrawLine(Pens.Black, TB.x, TB.y + (i * size), TR.x,
TR.y + (i * size));
 }
}

public List<int> ScrambleCube()
{
 List<int> moves = new List<int>();
 Random random = new Random();
}
```

## Rubik's Cube Solver & Trainer

```
int nextMove;
int currentMove = random.Next(0, 12);
moves.Add(currentMove);

for (int i = 0; i < 19; i++) // 1 + 19 moves in a standard
scramble
{
 nextMove = random.Next(0, 12); // random move on face
0-5, with 6-11 implying anticlockwise
 if (AreOpposite(currentMove, nextMove))
 {
 i--; // move is opposite, so regenerate a new
random move
 }
 else
 {
 moves.Add(nextMove);
 currentMove = nextMove;
 }
}
return moves;
}

public bool AreOpposite(int move1, int move2)
{
 bool areOpposite = false;
 if (move1 - 6 == move2 || move1 + 6 == move2) // Adding
or removing 6 from the move face will result in the same move but in
the opposite direction
 {
 areOpposite = true;
 }
 return areOpposite;
}

//Solving Cube
//White Cross

public int whiteFaceIndex()
{
 int whiteFace = -1;
 int i = 0;
 while (whiteFace == -1)
 {
 if (faces[i, 4].getColour() == 'w')
 {
 whiteFace = i;
 }
 i++;
 }
 return whiteFace;
}
```

## Rubik's Cube Solver & Trainer

```
 }

 public int AdjacencyCheck(int whiteFace)
 {
 int[] adjacentWhiteFaces = adjacentFaces[whiteFace];
 int[] adjacentIndexes = new int[4] {
edgeIndexes[whiteFace][1], edgeIndexes[whiteFace][4],
edgeIndexes[whiteFace][7], edgeIndexes[whiteFace][10] };
 int correctAdjacents = 0;
 for (int i = 0; i < 4; i++)
 {
 if (faces[adjacentWhiteFaces[i], 4].getColour() ==
faces[adjacentWhiteFaces[i], adjacentIndexes[i]].getColour())
 {
 correctAdjacents++;
 }
 }
 return correctAdjacents;
 }

 public bool WhiteCrossSolved()
 {
 int whiteFace = whiteFaceIndex();
 bool solved = false;
 if ((faces[whiteFace, 1].getColour() == 'w') &&
(faces[whiteFace, 3].getColour() == 'w') && (faces[whiteFace,
5].getColour() == 'w') && (faces[whiteFace, 7].getColour() == 'w'))
 {
 if (AdjacencyCheck(whiteFace) == 4)
 {
 solved = true;
 }
 }
 return solved;
 }

 public int Heuristic()
 {
 int whiteFace = whiteFaceIndex();
 int CorrectPieces = 0;
 for (int i = 0; i < 9; i++)
 {
 if (faces[whiteFace, i].getColour() == 'w')
 {
 CorrectPieces++;
 }
 }

 return 8 - CorrectPieces;
 }
}
```

## Rubik's Cube Solver & Trainer

```
// F2L

public char[,] edgeComposition = new char[4, 2]
{
 {'b', 'r'},
 {'r', 'g'},
 {'g', 'o'},
 {'o', 'b'},
};

public bool F2LPairSolved(int pairNumber) // check if given
pair has been solved
{
 if (faces[0, 5].getColour() == edgeComposition[pairNumber,
0])
 {
 if (faces[3, 3].getColour() ==
edgeComposition[pairNumber, 1])
 {
 if (faces[5, 2].getColour() == 'w')
 {
 if (faces[0, 8].getColour() ==
edgeComposition[pairNumber, 0])
 {
 if (faces[3, 6].getColour() ==
edgeComposition[pairNumber, 1])
 {
 return true;
 }
 }
 }
 }
 }
 return false;
}

public void Save(string filename)
{
 StreamWriter SW = new StreamWriter(File.Create(filename));
 for(int x = 0; x < 6; x++)
 {
 for(int y = 0; y < 9; y++)
 {
 SW.Write(faces[x, y].getColour());
 }
 SW.WriteLine();
 }
 SW.Close();
}

public Cube Load(string filename)
{
 char colour;
 Cube cube = new Cube();
}
```

## Rubik's Cube Solver & Trainer

```
try {
 StreamReader SR = new StreamReader(filename);
 for (int x = 0; x < 6; x++)
 {
 for (int y = 0; y < 9; y++)
 {
 colour = Convert.ToChar(SR.Read());
 if (colour == 13 || colour == 10)
 {
 SR.Read();
 colour = Convert.ToChar(SR.Read());
 }
 cube.faces[x, y].setColour(colour);
 }
 }
 SR.Close();
}
catch (Exception ex)
{
 MessageBox.Show("This user has no saved cubes.");
}

return cube;
}

public bool solved()
{
 for (int i = 0; i < 6; i++)
 {
 for (int j = 0; j < 8; j++)
 {
 if (faces[i, j].getColour() != faces[i, j +
1].getColour())
 {
 return false;
 }
 }
 }
 return true;
}

}

public class tile
{
 private char colour;

 public tile(char colour)
 {
 this.colour = colour;
 }
 public void setColour(char colour)
 {
 this.colour = colour;
 }
}
```

# Rubik's Cube Solver & Trainer

```
 public char getColour()
 {
 return this.colour;
 }
 }
}
```

---

Custom data structures.cs

---

```
using System;
using System.Collections;
using System.Collections.Generic;

namespace Rubiks_Cube_Solver_And_Trainer
{
 public struct coord
 {
 public float x;
 public float y;
 public coord(float x, float y)
 {
 this.x = x;
 this.y = y;
 }
 }

 public struct edge
 {
 public int face;
 public int position;
 public edge(int face, int position)
 {
 this.face = face;
 this.position = position;
 }
 }

 public class List<T> : IEnumerable<T> // T is a type parameter
 {
 private const int initialCapacity = 5;
 private T[] elements;
 private int count = 0;

 public List()
 {
 elements = new T[initialCapacity];
 }
 public T this[int index] // from microsoft documentation
 {
 get
 {
 if (index < 0 || index >= count)
 {

```

## Rubik's Cube Solver & Trainer

```
 throw new
ArgumentOutOfRangeException(nameof(index), "Index is out of bounds.");
 }
 return elements[index];
}
set
{
 if (index < 0 || index >= count)
 {
 throw new
ArgumentOutOfRangeException(nameof(index), "Index is out of bounds.");
 }
 elements[index] = value;
}
}

public void Add(T element)
{
 if (count == elements.Length)
 {
 Array.Resize(ref elements, elements.Length * 2);
 }
 elements[count++] = element;
}

public void Remove(T element)
{
 int index = Array.IndexOf(elements, element);
 if (index != -1)
 {
 // Shift elements to close the gap
 for (int i = index; i < count - 1; i++)
 {
 elements[i] = elements[i + 1];
 }
 count--;
 }
}

public void RemoveAt(int index)
{
 for (int i = index; i < count - 1; i++)
 {
 elements[i] = elements[i + 1];
 }
 count--;
}

public void Reverse()
{
 Stack<T> reverseStack = new Stack<T>(count);
 for (int i = 0; i < count; i++)
 {
 reverseStack.Push(elements[i]);
 }
}
```

## Rubik's Cube Solver & Trainer

```
 }
 for (int i = 0; i < count; i++)
 {
 elements[i] = reverseStack.Pop();
 }
 }

 public void Insert(int index, T element)
 {
 if (count == elements.Length)
 {
 Array.Resize(ref elements, elements.Length * 2);
 }

 for (int i = count; i > index; i--)
 {
 elements[i] = elements[i - 1];
 }

 elements[index] = element;
 count++;
 }

 public void AddRange(IEnumerable<T> elementsToAdd)
 {
 foreach (T element in elementsToAdd)
 {
 Add(element);
 }
 }

 public bool IsEmpty()
 {
 return count == 0;
 }

 public int Count()
 {
 return count;
 }

 // Implement GetEnumerator for IEnumerable<T> <- found from
 microsoft documentation
 public IEnumerator<T> GetEnumerator()
 {
 for (int i = 0; i < count; i++)
 {
 yield return elements[i];
 }
 }
 IEnumerator IEnumerable.GetEnumerator()
 {
 return GetEnumerator();
 }
```

## Rubik's Cube Solver & Trainer

```
 }
}
public class Stack<T>
{
 private int maxSize;
 private T[] elements;
 private int topPointer = -1;

 public Stack(int size)
 {
 maxSize = size;
 elements = new T[maxSize];
 }

 public bool IsEmpty()
 {
 return topPointer == -1;
 }

 public bool IsFull()
 {
 return topPointer == maxSize - 1;
 }

 public void Push(T element)
 {
 if (IsFull())
 {
 Console.WriteLine("Stack overflow error");
 }
 else
 {
 elements[++topPointer] = element;
 }
 }

 public T Pop()
 {
 if (IsEmpty())
 {
 Console.WriteLine("Stack underflow error");
 return default(T); // Return default value for the
type T
 }
 return elements[topPointer--];
 }

 public void Print()
 {
 Console.WriteLine("Top of stack");
 for (int i = topPointer; i >= 0; i--)
 {
 Console.WriteLine(elements[i]);
 }
 }
}
```

## Rubik's Cube Solver & Trainer

```
 }
 }

 public T Peek()
 {
 if (!IsEmpty())
 {
 return elements[topPointer];
 }
 else
 {
 Console.WriteLine("Stack contains no items");
 return default(T); // Return default value for the
type T
 }
 }
}
```

---

DatabaseInitialiser.cs

---

```
namespace Rubiks_Cube_Solver_And_Trainer
{
 static class DatabaseInitialiser
 {
 public static void PopulateDatabase()
 {
 PopulateAlgorithms();
 }
 public static void PopulateAlgorithms()
 {
 // F2L
 string _sqlstring;
 _sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Case 1', 'F2L', 'URur', 0);";
 DatabaseUtilities.ExecuteSqlNonQuery(_sqlstring);
 _sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Case 2', 'F2L', 'uFUF', 1);";
 DatabaseUtilities.ExecuteSqlNonQuery(_sqlstring);
 _sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Case 3', 'F2L', 'fuF', 2);";
 DatabaseUtilities.ExecuteSqlNonQuery(_sqlstring);
 _sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Case 4', 'F2L', 'RUr', 3);";
 DatabaseUtilities.ExecuteSqlNonQuery(_sqlstring);
 }
 }
}
```

## Rubik's Cube Solver & Trainer

```
 _sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Case 5', 'F2L', 'uRUruRUUr', 4);";
corner asd
DatabaseUtilities.ExecuteNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Case 6', 'F2L', 'UfuFUfUUF', 5);";
DatabaseUtilities.ExecuteNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Case 7', 'F2L', 'uRUUruRUUr', 6);";
DatabaseUtilities.ExecuteNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Case 8', 'F2L', 'UfUUFUfUUF', 7);";
DatabaseUtilities.ExecuteNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Case 9', 'F2L', 'UfuFufuF', 8);";
DatabaseUtilities.ExecuteNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Case 10', 'F2L', 'uRUrURUr', 9);";
DatabaseUtilities.ExecuteNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Case 11', 'F2L', 'uRUUrUfuF', 10);";
DatabaseUtilities.ExecuteNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Case 12', 'F2L', 'UfUUFuRUr', 11);";
DatabaseUtilities.ExecuteNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Case 13', 'F2L', 'UfUFufuF', 12);";
DatabaseUtilities.ExecuteNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Case 14', 'F2L', 'uRurURUr', 13);";
DatabaseUtilities.ExecuteNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
```

## Rubik's Cube Solver & Trainer

```
 "('Case 15', 'F2L', 'fUFUURUr', 14);";
DatabaseUtilities.ExecuteSqlNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Case 16', 'F2L', 'RurUUfuF', 15);";
DatabaseUtilities.ExecuteSqlNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Case 17', 'F2L', 'RUUruRUr', 16);"; //
Corner
DatabaseUtilities.ExecuteSqlNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Case 18', 'F2L', 'fUUFUfuF', 17);";
DatabaseUtilities.ExecuteSqlNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Case 19', 'F2L', 'URUUrURur', 18);";
DatabaseUtilities.ExecuteSqlNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Case 20', 'F2L', 'ufUUFufUF', 19);";
DatabaseUtilities.ExecuteSqlNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Case 21', 'F2L', 'UURUrURur', 20);";
DatabaseUtilities.ExecuteSqlNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Case 22', 'F2L', 'UUfuFufUF', 21);";
DatabaseUtilities.ExecuteSqlNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Case 23', 'F2L', 'RUruuRUruRUr', 22);";
DatabaseUtilities.ExecuteSqlNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Case 24', 'F2L', 'yruRUUruRUruRY', 23);";
DatabaseUtilities.ExecuteSqlNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" + // Corner Bottom &
Edge Top start
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Case 25', 'F2L', 'ufUFURur', 24);";
```

## Rubik's Cube Solver & Trainer

```
DatabaseUtilities.ExecuteNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Case 26', 'F2L', 'URurufUF', 25);";
DatabaseUtilities.ExecuteNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Case 27', 'F2L', 'RurURur', 26);";
DatabaseUtilities.ExecuteNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Case 28', 'F2L', 'fUFufUF', 27);";
DatabaseUtilities.ExecuteNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Case 29', 'F2L', 'fuFUfuF', 28);";
DatabaseUtilities.ExecuteNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Case 30', 'F2L', 'RUruRUr', 29);";
DatabaseUtilities.ExecuteNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" + // Corner Top &
Edge Middle Start
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Case 31', 'F2L', 'RurUfUF', 30);";
DatabaseUtilities.ExecuteNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Case 32', 'F2L', 'RUruRUruRUr', 31);";
DatabaseUtilities.ExecuteNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Case 33', 'F2L', 'uRuruRUUr', 32);";
DatabaseUtilities.ExecuteNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Case 34', 'F2L', 'UfUFUfUUF', 33);";
DatabaseUtilities.ExecuteNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Case 35', 'F2L', 'uRUrUfuF', 34);";
DatabaseUtilities.ExecuteNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
```

## Rubik's Cube Solver & Trainer

```
 "VALUES" +
 "('Case 36', 'F2L', 'UfuFuRUr', 35);";
DatabaseUtilities.ExecuteSqlNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Case 37', 'F2L', 'RurUfUUFUfUUF', 36);";
DatabaseUtilities.ExecuteSqlNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Case 38', 'F2L', 'RuruRUruRUUr', 37);";
DatabaseUtilities.ExecuteSqlNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Case 39', 'F2L', 'RurURUUrURur', 38);";
DatabaseUtilities.ExecuteSqlNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Case 40', 'F2L', 'RurUfuFufuF', 39);";
DatabaseUtilities.ExecuteSqlNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Case 41', 'F2L', 'RUruRurUUfuF', 40);";
DatabaseUtilities.ExecuteSqlNonQuery(_sqlstring);

// OLL
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Sune', 'OLL', 'RUrURUUr', 41);";
DatabaseUtilities.ExecuteSqlNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Antisune', 'OLL', 'luLuLUUL', 42);";
DatabaseUtilities.ExecuteSqlNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('H', 'OLL', 'FRUruRUruRUruf', 43);";
DatabaseUtilities.ExecuteSqlNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Pi', 'OLL', 'RUURRuRRuRRUUR', 44);";
DatabaseUtilities.ExecuteSqlNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Headlights', 'OLL', 'RRDrUURdrUUr', 45);";
```

## Rubik's Cube Solver & Trainer

```
DatabaseUtilities.ExecuteNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('T', 'OLL', 'MRUrumrFRf', 46);";
DatabaseUtilities.ExecuteNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Bowtie', 'OLL', 'fMRUrumrFR', 47);";
DatabaseUtilities.ExecuteNonQuery(_sqlstring);

// PLL
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Headlights Back', 'PLL', 'rDrUURdrUURRf',
48);";

DatabaseUtilities.ExecuteNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('No Headlights', 'PLL', 'FRuruRUrFRUurFRf',
49);";

DatabaseUtilities.ExecuteNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Ua Perm', 'PLL', 'MMUMUUmUMM', 50);";
DatabaseUtilities.ExecuteNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Ub Perm', 'PLL', 'MMuMUUmuMM', 51);";
DatabaseUtilities.ExecuteNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('Z Perm', 'PLL', 'MMuMMumUUMMUUm', 52);";
DatabaseUtilities.ExecuteNonQuery(_sqlstring);
_sqlstring = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves, AlgID)" +
 "VALUES" +
 "('H Perm', 'PLL', 'MMUMMUUMMUMM', 53);";
DatabaseUtilities.ExecuteNonQuery(_sqlstring);
 }
}
}
```

---

DatabaseUtilities.cs

---

```
using ADOX;
using System;
```

# Rubik's Cube Solver & Trainer

```
using System.IO;
using System.Windows.Forms;
using System.Data.OleDb;
using System.Data;

namespace Rubiks_Cube_Solver_And_Trainer
{
 static class DatabaseUtilities
 {
 private const string EXAMPLEDB = "RubikCubeDatabase.mdb";
 private const string CONNECTION_STRING = @"Provider=Microsoft
Jet 4.0 OLE DB Provider;Data Source = " + EXAMPLEDB + ";";
 public static void CreateNewDatabase()
 {
 try
 {
 // Check if the database exists
 if (!File.Exists(EXAMPLEDB))
 {
 // Create a catalog object
 CatalogClass cat = new CatalogClass();

 // Create the database
 cat.Create(CONNECTION_STRING);
 MessageBox.Show("Database Created Successfully",
"Rubiks Cube Trainer", MessageBoxButtons.OK,
MessageBoxIcon.Information);

 // create database tables in the database
 CreateTable();
 DatabaseInitialiser.PopulateAlgorithms();

 //cleanup
 cat = null;
 }
 else
 {
 MessageBox.Show("The database already exists",
"Rubiks Cube Trainer", MessageBoxButtons.OK,
MessageBoxIcon.Information);
 }
 }
 catch (Exception ex)
 {
 MessageBox.Show(ex.Message, "Rubiks Cube Trainer",
MessageBoxButtons.OK, MessageBoxIcon.Exclamation);
 }
 }
 public static DataTable ExecuteSqlQuery(String sSqlString)
 {
 OleDbConnection cnn = new
OleDbConnection(CONNECTION_STRING); // Create a new connection to the
database
```

## Rubik's Cube Solver & Trainer

```
OleDbCommand cmd = new OleDbCommand(sSqlString, cnn); //
sSqlString string contains a SQL statement to run on the database
OleDbDataAdapter da = new OleDbDataAdapter(cmd);
DataTable dt = new DataTable();

try
{
 cnn.Open();
 da.Fill(dt);
 cnn.Close();
}
catch (Exception ex)
{
 MessageBox.Show(ex.Message, "Rubiks Cube Trainer",
 MessageBoxButtons.OK, MessageBoxIcon.Exclamation);
}
return dt; // Return the data retrieved by the query
}

public static void ExecuteSqlNonQuery(String sSqlString)
{
 try
 {
 OleDbConnection cnn = new
OleDbConnection(CONNECTION_STRING);
 cnn.Open();
 OleDbCommand cmd = new OleDbCommand(sSqlString, cnn);
 cmd.ExecuteNonQuery();
 cnn.Close();
 }
 catch (Exception ex)
 {
 MessageBox.Show(ex.Message, "Library Administration
System", MessageBoxButtons.OK, MessageBoxIcon.Exclamation);
 }
}

private static void CreateTables()
{
 string _SqlString;
 _SqlString = "CREATE TABLE Users (" +
 "UserID AUTOINCREMENT NOT NULL , " +
 "Username VARCHAR(20) NOT NULL, " +
 "UserPassword VARCHAR(64) NOT NULL, " +
 "Salt VARCHAR(16) NOT NULL, " +
 "PRIMARY KEY (UserID) " +
 ");";
 ExecuteSqlNonQuery(_SqlString);

 _SqlString = "CREATE TABLE Algorithms(" +
 "AlgName VARCHAR(30) NOT NULL," +
 "AlgCat VARCHAR(15) NOT NULL," +
 "CharMoves VARCHAR(100) NOT NULL," +
 "AlgID NUMERIC(10) NOT NULL UNIQUE," +
```

## Rubik's Cube Solver & Trainer

```
 "PRIMARY KEY(AlgID)" +
 ");";
 ExecuteSqlNonQuery(_SqlString);

 _SqlString = "CREATE TABLE UserTimes(" +
 "UserID NUMERIC(7) NOT NULL," +
 "Milliseconds LONG NOT NULL " +
 ");";
 ExecuteSqlNonQuery(_SqlString);

 }

}
```

---

Form1.cs

---

```
using System.Windows.Forms;
using System;
using System.Data;
using System.Diagnostics;
using System.Drawing;
using System.Linq;

namespace Rubiks_Cube_Solver_And_Trainer
{
 public partial class frmMainScreen : Form
 {
 public Cube cube;
 public IDAStarWhiteCross crossSolver;
 public Solver solver;
 public Trainer trainer;
 public Timer timer;
 private Timer displayUpdater;
 private Stopwatch userTimer;
 private string CurrentUser;
 private int colourSelelctionIndex;
 private Color[] colours = {Color.Red, Color.Orange,
Color.ForestGreen, Color.DodgerBlue, Color.White, Color.Yellow };
 public frmMainScreen()
 {
 DatabaseUtilities.CreateNewDatabase();
 InitializeComponent();
 cube = new Cube();
 crossSolver = new IDAStarWhiteCross();
 solver = new Solver();
 colourSelelctionIndex = 0;
 btnColourCycle.BackColor = colours[colourSelelctionIndex];
 btnF2l.Visible = false;
 BtnWhiteCross.Visible = false;
 btnOLL1.Visible = false;
 btnOLL2.Visible = false;
 btnPLL1.Visible = false;
 BtnPLL2.Visible = false;
 }
 }
}
```

## Rubik's Cube Solver & Trainer

```
 btnBack.Visible = false;
 ddF2L.Visible = false;
 ddOLL.Visible = false;
 ddPLL.Visible = false;
 btnPlay.Visible = false;
 btnPause.Visible = false;
 txtAlgHeader.Visible = false;
 foreach (Control control in Controls)
 {
 if (control.Tag != null && (control.Tag.ToString() ==
"MainItems" || control.Tag.ToString() == "ColourSelection"))
 {
 control.Visible = false;
 }
 }
 Size = new Size(300, 500);
 }
 private void pnlCubeDisplay_Paint(object sender,
PaintEventArgs e)
 {
 const float gradient = 0.55f;
 const int size = 96;
 coord TL = new coord(0, size);
 coord TT = new coord(TL.x + size, TL.y - size * gradient);
 coord TR = new coord(TT.x + size, TT.y + size * gradient);
 coord TB = new coord(TL.x + size, TL.y + size * gradient);
 coord BL = new coord(TL.x, TL.y + size);
 coord BB = new coord(TB.x, TB.y + size);
 coord BR = new coord(TR.x, TR.y + size);
 cube.drawCube(sender, e, TL, TT, TR, TB, BL, BB, BR,
Convert.ToInt32(size / 3), gradient);
 }

 private void btnR_Click(object sender, EventArgs e)
 {
 cube.RotateFace(3, false);
 pnlCubeDisplay.Invalidate(); //Redraw Cube
 initialiseColourButtons();
 }

 private void btnRprime_Click(object sender, EventArgs e)
 {
 cube.RotateFace(3, true);
 pnlCubeDisplay.Invalidate();
 initialiseColourButtons();
 }

 private void btnU_Click(object sender, EventArgs e)
 {
 cube.RotateFace(4, false);
 pnlCubeDisplay.Invalidate();
 initialiseColourButtons();
 }
}
```

## Rubik's Cube Solver & Trainer

```
private void btnUprime_Click(object sender, EventArgs e)
{
 cube.RotateFace(4, true);
 pnlCubeDisplay.Invalidate();
 initialiseColourButtons();
}

private void btnL_Click(object sender, EventArgs e)
{
 cube.RotateFace(2, false);
 pnlCubeDisplay.Invalidate();
 initialiseColourButtons();
}

private void btnLprime_Click(object sender, EventArgs e)
{
 cube.RotateFace(2, true);
 pnlCubeDisplay.Invalidate();
 initialiseColourButtons();
}

private void btnD_Click(object sender, EventArgs e)
{
 cube.RotateFace(5, false);
 pnlCubeDisplay.Invalidate();
 initialiseColourButtons();
}

private void btnDprime_Click(object sender, EventArgs e)
{
 cube.RotateFace(5, true);
 pnlCubeDisplay.Invalidate();
 initialiseColourButtons();
}

private void btnF_Click(object sender, EventArgs e)
{
 cube.RotateFace(0, false);
 pnlCubeDisplay.Invalidate();
 initialiseColourButtons();
}

private void btnFprime_Click(object sender, EventArgs e)
{
 cube.RotateFace(0, true);
 pnlCubeDisplay.Invalidate();
 initialiseColourButtons();
}

private void btnB_Click(object sender, EventArgs e)
{
 cube.RotateFace(1, false);
 pnlCubeDisplay.Invalidate();
}
```

## Rubik's Cube Solver & Trainer

```
 initialiseColourButtons();
 }

 private void btnBprime_Click(object sender, EventArgs e)
 {
 cube.RotateFace(1, true);
 pnlCubeDisplay.Invalidate();
 initialiseColourButtons();
 }

 private void btnM_Click(object sender, EventArgs e)
 {
 cube.RotateVerticalMiddleLayer();
 pnlCubeDisplay.Invalidate();
 initialiseColourButtons();
 }

 private void btnMprime_Click(object sender, EventArgs e)
 {
 cube.RotateVerticalMiddleLayer();
 cube.RotateVerticalMiddleLayer();
 cube.RotateVerticalMiddleLayer();
 pnlCubeDisplay.Invalidate();
 initialiseColourButtons();
 }

 private void btnRotateUp_Click(object sender, EventArgs e)
 {
 cube.rotateDisplayUp();
 pnlCubeDisplay.Invalidate();
 initialiseColourButtons();
 initialiseColourButtons();
 }

 private void btnRotateDown_Click(object sender, EventArgs e)
 {
 cube.rotateDisplayDown();
 pnlCubeDisplay.Invalidate();
 initialiseColourButtons();
 }

 private void btnRotateLeft_Click(object sender, EventArgs e)
 {
 cube.rotateDisplayLeft();
 pnlCubeDisplay.Invalidate();
 initialiseColourButtons();
 }

 private void BtnRotateRight_Click(object sender, EventArgs e)
 {
 cube.rotateDisplayRight();
 pnlCubeDisplay.Invalidate();
 initialiseColourButtons();
 }
}
```

## Rubik's Cube Solver & Trainer

```
private void BtnSolver_Click(object sender, EventArgs e)
{
 List<int> moves = crossSolver.IDAStar(cube);
 bool Anticlockwise = false;
 int decodedMove;
 string stringMoves = "";
 foreach (int move in moves)
 {
 if (move > 5)
 {
 decodedMove = move - 6;
 Anticlockwise = true;
 }
 else
 {
 decodedMove = move;
 Anticlockwise = false;
 }
 cube.RotateFace(decodedMove, Anticlockwise);
 switch (decodedMove)
 {
 case 0:
 stringMoves += 'F';
 break;
 case 1:
 stringMoves += 'B';
 break;
 case 2:
 stringMoves += 'L';
 break;
 case 3:
 stringMoves += 'R';
 break;
 case 4:
 stringMoves += 'U';
 break;
 case 5:
 stringMoves += 'D';
 break;
 }
 if (Anticlockwise) { stringMoves += '\'; }
 stringMoves += ' ';
 }
 txtSolveMoves.Text = stringMoves;
 pnlCubeDisplay.Invalidate();
}

private void btnScramble_Click(object sender, EventArgs e)
{
 List<int> moves = cube.ScrambleCube();
 bool Anticlockwise = false;
```

## Rubik's Cube Solver & Trainer

```
int decodedMove;
string stringMoves = "";
foreach (int move in moves)
{
 if (move > 5)
 {
 decodedMove = move - 6;
 Anticlockwise = true;
 }
 else
 {
 decodedMove = move;
 Anticlockwise = false;
 }

 switch (decodedMove)
 {
 case 0:
 stringMoves += 'F';
 break;
 case 1:
 stringMoves += 'B';
 break;
 case 2:
 stringMoves += 'L';
 break;
 case 3:
 stringMoves += 'R';
 break;
 case 4:
 stringMoves += 'U';
 break;
 case 5:
 stringMoves += 'D';
 break;
 }
 if (Anticlockwise) { stringMoves += '\'; }
 stringMoves += ' ';
 cube.RotateFace(decodedMove, Anticlockwise);
}
txtSolveMoves.Text = stringMoves;
pnlCubeDisplay.Invalidate();
initialiseColourButtons();
}

private void btnF2lTest_Click(object sender, EventArgs e)
{
 if (solver.WhiteCrossSolved(cube))
 {
 char[,] edgeComposition = new char[4, 2]
{
```

## Rubik's Cube Solver & Trainer

```
{'b', 'r'},
{'r', 'g'},
{'g', 'o'},
{'o', 'b'},
};
string stringSolution = "";
for (int i = 0; i < 4; i++)
{
 if (cube.faces[2, 4].getColour() == 'y' ||
cube.faces[3, 4].getColour() == 'y') //Orient cube correctly
 {
 cube.rotateDisplayLeft();
 stringSolution += "Y ";
 }
 while (cube.faces[4, 4].getColour() != 'y')
 {
 cube.rotateDisplayDown();
 stringSolution += "X' ";
 }
 while (cube.faces[0, 4].getColour() !=
edgeComposition[i, 0])
 {
 cube.rotateDisplayLeft();
 stringSolution += "Y ";
 }
 (List<int> moves, List<int> orientationMoves) =
solver.f2lbruteforce(cube, i);
 bool Anticlockwise = false;
 int decodedMove = -1;
 string stringMoves = "";
 foreach (int move in orientationMoves)
 {
 if (move > 5)
 {
 if (move == 17)
 {
 decodedMove = move;
 Anticlockwise = false;
 }
 else
 {
 decodedMove = move - 6;
 Anticlockwise = true;
 }
 }
 else
 {
 decodedMove = move;
 Anticlockwise = false;
 }
 switch (decodedMove)
 {
 case 0:
 stringMoves += 'F';
```

## Rubik's Cube Solver & Trainer

```
 break;
 case 1:
 stringMoves += 'B';
 break;
 case 2:
 stringMoves += 'L';
 break;
 case 3:
 stringMoves += 'R';
 break;
 case 4:
 stringMoves += 'U';
 break;
 case 5:
 stringMoves += 'D';
 break;
 case 17:
 stringMoves += 'Y';
 break;
 }
 if (Anticlockwise) { stringMoves += '\''; }
 stringMoves += ' ';
}

stringSolution += stringMoves;
stringMoves = "";
foreach (int move in moves)
{
 if (move > 11)
 {
 switch (move)
 {
 case 16:
 Anticlockwise = false;
 cube.RotateFace(5, false);
 cube.RotateHorizontalMiddleLayer();
 break;
 case 17:
 Anticlockwise = false;
 cube.RotateFace(5, false);
 cube.RotateHorizontalMiddleLayer();
 cube.RotateFace(4, false);
 break;
 case 26:
 Anticlockwise = true;
 cube.RotateFace(5, true);
 cube.RotateHorizontalMiddleLayer();
 cube.RotateHorizontalMiddleLayer();
 cube.RotateHorizontalMiddleLayer();
 break;
 case 27:
 Anticlockwise = true;
```

## Rubik's Cube Solver & Trainer

```

 cube.RotateFace(5, true);
 cube.RotateHorizontalMiddleLayer();
 cube.RotateHorizontalMiddleLayer();
 cube.RotateHorizontalMiddleLayer();
 cube.RotateFace(4, true);
 break;
 }
}
else if (move > 5)
{
 decodedMove = move - 6;
 Anticlockwise = true;
 cube.RotateFace(decodedMove,
Anticlockwise);
}
else
{
 decodedMove = move;
 Anticlockwise = false;
 cube.RotateFace(decodedMove,
Anticlockwise);
}
switch (decodedMove)
{
 case 0:
 stringMoves += 'F';
 break;
 case 1:
 stringMoves += 'B';
 break;
 case 2:
 stringMoves += 'L';
 break;
 case 3:
 stringMoves += 'R';
 break;
 case 4:
 stringMoves += 'U';
 break;
 case 5:
 stringMoves += 'D';
 break;
 case 17:
 stringMoves += 'Y';
 break;
}
if (Anticlockwise) { stringMoves += '\'; }
stringMoves += ' ';
}
stringSolution += stringMoves;
stringSolution += "\n\n";
}
txtSolveMoves.Text = stringSolution;
```

## Rubik's Cube Solver & Trainer

```
 pnlCubeDisplay.Invalidate();
 }
 else
 {
 MessageBox.Show("Previous stage not complete");
 }
}

private void btnOLL_Click(object sender, EventArgs e)
{
 if (solver.WhiteCrossSolved(cube) &&
 solver.F2LSolved(cube))
 {
 if (cube.faces[2, 4].getColour() == 'y' ||
 cube.faces[3, 4].getColour() == 'y') // Orient Cube correctly
 {
 cube.rotateDisplayLeft();
 }
 while (cube.faces[4, 4].getColour() != 'y')
 {
 cube.rotateDisplayDown();
 }
 List<int> solution = solver.SolveOLL(cube.Clone(cube));
 bool Anticlockwise = false;
 int decodedMove;
 string stringMoves = "";
 foreach (int move in solution)
 {
 if (move > 5)
 {
 decodedMove = move - 6;
 Anticlockwise = true;
 }
 else
 {
 decodedMove = move;
 Anticlockwise = false;
 }
 cube.RotateFace(decodedMove, Anticlockwise);
 switch (decodedMove)
 {
 case 0:
 stringMoves += 'F';
 break;
 case 1:
 stringMoves += 'B';
 break;
 case 2:
 stringMoves += 'L';
 break;
 case 3:
```

## Rubik's Cube Solver & Trainer

```
 stringMoves += 'R';
 break;
 case 4:
 stringMoves += 'U';
 break;
 case 5:
 stringMoves += 'D';
 break;
 }
 if (Anticlockwise) { stringMoves += '\''; }
 stringMoves += ' ';
}
txtSolveMoves.Text = stringMoves;
pnlCubeDisplay.Invalidate();
}
else
{
 MessageBox.Show("Previous stages not complete");
}
}

private void btnOLL2Look_Click(object sender, EventArgs e)
{
 if (solver.WhiteCrossSolved(cube) &&
 solver.F2LSolved(cube) && solver.Oll1Solved(cube))
 {
 List<int> solution =
 solver.OLLSecondLook(cube.Clone(cube));
 bool Anticlockwise = false;
 int decodedMove;
 string stringMoves = "";
 foreach (int move in solution)
 {
 if (move == 12)
 {
 decodedMove = 8;
 Anticlockwise = false;
 cube.RotateVerticalMiddleLayer();
 }
 else if (move == 18)
 {
 decodedMove = 8;
 Anticlockwise = true;
 cube.RotateVerticalMiddleLayer();
 cube.RotateVerticalMiddleLayer();
 cube.RotateVerticalMiddleLayer();
 }
 else if (move > 5)
 {
 decodedMove = move - 6;
 Anticlockwise = true;
 cube.RotateFace(decodedMove, Anticlockwise);
 }
 }
 }
}
```

## Rubik's Cube Solver & Trainer

```
 else
 {
 decodedMove = move;
 Anticlockwise = false;
 cube.RotateFace(decodedMove, Anticlockwise);
 }
 switch (decodedMove)
 {
 case 0:
 stringMoves += 'F';
 break;
 case 1:
 stringMoves += 'B';
 break;
 case 2:
 stringMoves += 'L';
 break;
 case 3:
 stringMoves += 'R';
 break;
 case 4:
 stringMoves += 'U';
 break;
 case 5:
 stringMoves += 'D';
 break;
 case 8:
 stringMoves += 'M';
 break;

 }
 if (Anticlockwise) { stringMoves += '\''; }
 stringMoves += ' ';
 }
 txtSolveMoves.Text = stringMoves;
 pnlCubeDisplay.Invalidate();
}
else
{
 MessageBox.Show("Previous stages not complete");
}
}

private void btnPLL1Test_Click(object sender, EventArgs e)
{
 if (solver.WhiteCrossSolved(cube) &&
 solver.F2LSolved(cube) && solver.Oll1Solved(cube) &&
 solver.Oll2Solved(cube))
 {
 string stringMoves = "";
 (List<int> solution, int Case) =
 solver.PLLFirstLook(cube.Clone(cube));
 if (Case == 1)
 {

```

## Rubik's Cube Solver & Trainer

```
switch (solver.GetTopLayerHeadlights(cube))
{
 case 0:
 cube.RotateFace(4, false);
 cube.RotateFace(4, false);
 stringMoves += "U U ";
 break;
 case 3:
 cube.RotateFace(4, true);
 stringMoves += "U' ";
 break;
 case 2:
 cube.RotateFace(4, false);
 stringMoves += "U ";
 break;
}
if (cube.faces[2, 4].getColour() == 'y' ||
cube.faces[3, 4].getColour() == 'y') // Orient Cube correctly
{
 cube.rotateDisplayLeft();
}
while (cube.faces[0, 4].getColour() != 'y')
{
 cube.rotateDisplayDown();
}
stringMoves += "X' ";
}
bool Anticlockwise = false;
int decodedMove;
foreach (int move in solution)
{
 if (move == 12)
 {
 decodedMove = 8;
 cube.RotateFace(3, true);
 cube.RotateFace(2, false);
 }
 else if (move == 18)
 {
 decodedMove = 8;
 Anticlockwise = true;
 cube.RotateFace(3, false);
 cube.RotateFace(2, true);
 }
 else if (move > 5)
 {
 decodedMove = move - 6;
 Anticlockwise = true;
 cube.RotateFace(decodedMove, Anticlockwise);
 }
 else
 {
 decodedMove = move;
 Anticlockwise = false;
 }
}
```

## Rubik's Cube Solver & Trainer

```
 cube.RotateFace(decodedMove, Anticlockwise);
 }
 switch (decodedMove)
 {
 case 0:
 stringMoves += 'F';
 break;
 case 1:
 stringMoves += 'B';
 break;
 case 2:
 stringMoves += 'L';
 break;
 case 3:
 stringMoves += 'R';
 break;
 case 4:
 stringMoves += 'U';
 break;
 case 5:
 stringMoves += 'D';
 break;
 case 8:
 stringMoves += 'M';
 break;
 }
 if (Anticlockwise) { stringMoves += '\''; }
 stringMoves += ' ';
}
if (Case == 1)
{
 cube.rotateDisplayUp();
 stringMoves += "X ";
}
txtSolveMoves.Text = stringMoves;
pnlCubeDisplay.Invalidate();
}
else
{
 MessageBox.Show("Previous stages not complete");
}
}

private void BtnPLCTest2_Click(object sender, EventArgs e)
{
 if (solver.WhiteCrossSolved(cube) &&
 solver.F2LSolved(cube) && solver.Oll1Solved(cube) &&
 solver.Oll2Solved(cube) && solver.Pll1Solved(cube))
 {
 string stringMoves = "";
 int decodedMove;
 bool Anticlockwise = false;
```

## Rubik's Cube Solver & Trainer

```
 if (cube.faces[2, 4].getColour() == 'y' ||
cube.faces[3, 4].getColour() == 'y') // Orient Cube correctly
 {
 cube.rotateDisplayLeft();
 }
 while (cube.faces[4, 4].getColour() != 'y')
 {
 cube.rotateDisplayDown();
 stringMoves += "X' ";
 }
 for (int i = 0; i < 2; i++) // Some cases require 2
algorithms
 {
 List<int> solution =
solver.PLLSecondLook(cube.Clone(cube));
 foreach (int move in solution)
 {
 if (move == 12)
 {
 decodedMove = 8;
 cube.RotateVerticalMiddleLayer();
 }
 else if (move == 18)
 {
 decodedMove = 8;
 Anticlockwise = true;
 cube.RotateVerticalMiddleLayer();
 cube.RotateVerticalMiddleLayer();
 cube.RotateVerticalMiddleLayer();
 }
 else if (move > 5)
 {
 decodedMove = move - 6;
 Anticlockwise = true;
 cube.RotateFace(decodedMove,
Anticlockwise);
 }
 else
 {
 decodedMove = move;
 Anticlockwise = false;
 cube.RotateFace(decodedMove,
Anticlockwise);
 }
 switch (decodedMove)
 {
 case 0:
 stringMoves += 'F';
 break;
 case 1:
 stringMoves += 'B';
 break;
 case 2:
```

## Rubik's Cube Solver & Trainer

```
 stringMoves += 'L';
 break;
 case 3:
 stringMoves += 'R';
 break;
 case 4:
 stringMoves += 'U';
 break;
 case 5:
 stringMoves += 'D';
 break;
 case 8:
 stringMoves += 'M';
 break;
 }
 if (Anticlockwise) { stringMoves += '\'; }
 stringMoves += ' ';
}
}
if (stringMoves == "X' X ")
{
 stringMoves = "";
}
int uCount = 0;
while (cube.faces[0, 4].getColour() != cube.faces[0,
1].getColour())
{
 uCount++;
 cube.RotateFace(4, false);
 stringMoves += "U ";
 if (uCount == 3)
 {
 stringMoves =
stringMoves.Remove(stringMoves.Length - 6);
 stringMoves += "U' ";
 }
}
txtSolveMoves.Text = stringMoves;
pnlCubeDisplay.Invalidate();
}
else
{
 MessageBox.Show("Previous stages not complete");
}
}

private void btnCFOPSolver_Click(object sender, EventArgs e)
{
 if (!solver.IsCubeSolvable(cube))
 {
 MessageBox.Show("Invalid Cube Configuration");
 }
}
```

## Rubik's Cube Solver & Trainer

```
else
{
 btnScramble.Visible = false;
 btnCFOPSolver.Visible = false;
 BtnTrainer.Visible = false;
 btnF2l.Visible = true;
 BtnWhiteCross.Visible = true;
 btnOLL1.Visible = true;
 btnOLL2.Visible = true;
 btnPLL1.Visible = true;
 BtnPLL2.Visible = true;
 btnBack.Visible = true;
 btnStartTimer.Visible = false;
 txtTimer.Visible = false;
 btnNewCube.Visible = false;

 foreach (Control control in Controls)
 {
 if (control.Tag != null && control.Tag.ToString()
== "ColourSelection")
 {
 control.Visible = false;
 }
 }
}

private void btnBack_Click(object sender, EventArgs e)
{
 btnScramble.Visible = true;
 btnCFOPSolver.Visible = true;
 BtnTrainer.Visible = true;
 btnF2l.Visible = false;
 BtnWhiteCross.Visible = false;
 btnOLL1.Visible = false;
 btnOLL2.Visible = false;
 btnPLL1.Visible = false;
 BtnPLL2.Visible = false;
 btnBack.Visible = false;
 ddF2L.Visible = false;
 ddOLL.Visible = false;
 ddPLL.Visible = false;
 ddCustom.Visible = false;
 btnBack.Location = new System.Drawing.Point(26, 250);
 btnSaveCube.Visible = true;
 btnLoadCube.Visible = true;
 btnPlay.Visible = false;
 btnPause.Visible = false;
 btnRotateDown.Visible = true;
 btnRotateUp.Visible = true;
 btnRotateLeft.Visible = true;
 BtnRotateRight.Visible = true;
 btnStartTimer.Visible = true;
}
```

## Rubik's Cube Solver & Trainer

```
txtTimer.Visible = true;
txtSolveMoves.Text = "";
btnNewCube.Visible = true;
txtAlgHeader.Visible = false;
txtAlgMoves.Visible = false;
txtAlgMovesInput.Visible = false;
txtAlgName.Visible = false;
txtAlgNameInput.Visible = false;
btnAddAlgorithm.Visible = false;

foreach (Control control in Controls)
{
 if (control.Tag != null && control.Tag.ToString() ==
"ColourSelection")
 {
 control.Visible = true;
 }
}
initialiseColourButtons();
}

private void BtnTrainer_Click(object sender, EventArgs e)
{
 btnScramble.Visible = false;
 btnCFOPSolver.Visible = false;
 BtnTrainer.Visible = false;
 btnStartTimer.Visible = false;
 txtTimer.Visible = false;
 txtAlgMoves.Visible = true;
 txtAlgMovesInput.Visible = true;
 txtAlgName.Visible = true;
 txtAlgNameInput.Visible = true;
 btnAddAlgorithm.Visible = true;
 btnBack.Visible = true;
 ddF2L.Visible = true;
 ddOLL.Visible = true;
 ddPLL.Visible = true;
 ddCustom.Visible = true;
 btnBack.Location = new Point(26, 110);
 btnSaveCube.Visible = false;
 btnLoadCube.Visible = false;
 btnPlay.Visible = true;
 btnPause.Visible = true;
 txtAlgHeader.Visible = true;
 btnRotateDown.Visible = false;
 btnRotateUp.Visible = false;
 btnRotateLeft.Visible = false;
 BtnRotateRight.Visible = false;
 btnNewCube.Visible = false;

 foreach (Control control in Controls)
 {
```

## Rubik's Cube Solver & Trainer

```
 if (control.Tag != null && control.Tag.ToString() ==
"ColourSelection")
 {
 control.Visible = false;
 }
 }
 DisplayAlgorithms();
}
private void DisplayAlgorithms()
{
 string _sqlstring;
 _sqlstring = "SELECT AlgName FROM Algorithms WHERE AlgCat =
'F2L'";
 DataTable _dt =
DatabaseUtilities.ExecuteSqlQuery(_sqlstring);
 string[] algNames = new string[_dt.Rows.Count];
 int i = 0;
 foreach(DataRow row in _dt.Rows)
 {
 algNames[i++] = Convert.ToString(row[0]);
 }
 i = 0;
 ddF2L.DataSource = algNames;
 _sqlstring = "SELECT AlgName FROM Algorithms WHERE AlgCat =
'OLL'";
 _dt = DatabaseUtilities.ExecuteSqlQuery(_sqlstring);
 algNames = new string[_dt.Rows.Count];
 foreach (DataRow row in _dt.Rows)
 {
 algNames[i++] = Convert.ToString(row[0]);
 }
 i = 0;
 ddOLL.DataSource = algNames;
 _sqlstring = "SELECT AlgName FROM Algorithms WHERE AlgCat =
'PLL'";
 _dt = DatabaseUtilities.ExecuteSqlQuery(_sqlstring);
 algNames = new string[_dt.Rows.Count];
 foreach (DataRow row in _dt.Rows)
 {
 algNames[i++] = Convert.ToString(row[0]);
 }
 i = 0;
 ddPLL.DataSource = algNames;
 _sqlstring = "SELECT AlgName FROM Algorithms WHERE AlgCat =
'Custom'";
 _dt = DatabaseUtilities.ExecuteSqlQuery(_sqlstring);
 algNames = new string[_dt.Rows.Count];
 foreach (DataRow row in _dt.Rows)
 {
 algNames[i++] = Convert.ToString(row[0]);
 }
 ddCustom.DataSource = algNames;
}
```

## Rubik's Cube Solver & Trainer

```
private void btnSaveCube_Click(object sender, EventArgs e)
{
 cube.Save(CurrentUser);
}

private void btnLoadCube_Click(object sender, EventArgs e)
{
 cube = cube.Load(CurrentUser);
 pnlCubeDisplay.Invalidate();
 initialiseColourButtons();
}

private void ddF2L_SelectedIndexChanged(object sender,
EventArgs e)
{
 cube = new Cube(); // Set the cube to a solved state
 cube.rotateDisplayDown();
 cube.rotateDisplayDown(); ///; get yellow face on top
 int algID = ddF2L.SelectedIndex;
 trainer = new Trainer(algID, cube);
 trainer.DisplayAlgorithm();
 if (timer != null)
 {
 timer.Dispose();
 }
 timer = new Timer();
 timer.Stop();

 timer.Interval = 1000;
 timer.Tick += OnTick;
 txtAlgHeader.Text = ddF2L.GetItemText(ddF2L.SelectedItem);
 txtAlgHeader.SelectionAlignment =
HorizontalAlignment.Center;
 pnlCubeDisplay.Invalidate();
}

private void OnTick(object sender, EventArgs e)
{
 if (trainer.currentMoveIndex > trainer.moves.Count() - 1)
 {
 timer.Stop();
 return;
 }
 trainer.DisplayMoves();
 pnlCubeDisplay.Invalidate();
}

private void btnPlay_Click(object sender, EventArgs e)
{
 timer.Start();
}

private void btnPause_Click(object sender, EventArgs e)
```

## Rubik's Cube Solver & Trainer

```
{
 timer.Stop();
}

private void ddOLL_SelectedIndexChanged(object sender,
EventArgs e)
{
 cube = new Cube(); // Set the cube to a solved state
 cube.rotateDisplayDown();
 cube.rotateDisplayDown(); //; get yellow face on top
 int algID = ddOLL.SelectedIndex + 41;
 trainer = new Trainer(algID, cube);
 trainer.DisplayAlgorithm();
 if (timer != null)
 {
 timer.Dispose();
 }
 timer = new Timer();
 timer.Stop();
 timer.Interval = 1000;
 timer.Tick += OnTick;
 txtAlgHeader.Text = ddOLL.GetItemText(ddOLL.SelectedItem);
 txtAlgHeader.SelectionAlignment =
HorizontalAlignment.Center;
 pnlCubeDisplay.Invalidate();
}

private void ddPLL_SelectedIndexChanged(object sender,
EventArgs e)
{
 cube = new Cube(); // Set the cube to a solved state
 cube.rotateDisplayDown();
 cube.rotateDisplayDown(); //; get yellow face on top
 int algID = ddPLL.SelectedIndex + 48;
 if (algID == 48)
 {
 cube.rotateDisplayDown();
 }
 trainer = new Trainer(algID, cube);
 trainer.DisplayAlgorithm();
 if (timer != null)
 {
 timer.Dispose();
 }
 timer = new Timer();
 timer.Stop();
 timer.Interval = 1000;
 timer.Tick += OnTick;
 txtAlgHeader.Text = ddPLL.GetItemText(ddPLL.SelectedItem);
 txtAlgHeader.SelectionAlignment =
HorizontalAlignment.Center;
 pnlCubeDisplay.Invalidate();
}
```

## Rubik's Cube Solver & Trainer

```
private void ddCustom_SelectedIndexChanged(object sender,
EventArgs e)
{
 cube = new Cube(); // Set the cube to a solved state
 cube.rotateDisplayDown();
 cube.rotateDisplayDown(); //; get yellow face on top
 int algID = ddCustom.SelectedIndex + 54;
 trainer = new Trainer(algID, cube);
 trainer.DisplayAlgorithm();
 if (timer != null)
 {
 timer.Dispose();
 }
 timer = new Timer();
 timer.Stop();
 timer.Interval = 1000;
 timer.Tick += OnTick;
 txtAlgHeader.Text =
ddCustom.GetItemText(ddCustom.SelectedItem);
 txtAlgHeader.SelectionAlignment =
HorizontalAlignment.Center;
 pnlCubeDisplay.Invalidate();
}

private void btnLogIn_Click(object sender, EventArgs e)
{
 string username = txtUsername.Text;
 string password = txtPassword.Text;
 if (username.Length < 4 || password.Length < 6)
 {
 MessageBox.Show("Username must be at least 4
characters \nPassword must be at least 6 characters");
 }
 else
 {
 string _sSqlString = "SELECT UserPassword, salt " +
 "FROM Users " +
 $"WHERE Username =
'{username}';";
 DataTable dt =
DatabaseUtilities.ExecuteSqlQuery(_sSqlString); // get password + salt
from db
 string UserPassword = null;
 string UserSalt = null;
 try { UserPassword = Convert.ToString(dt.Rows[0][0]);
 } catch (Exception ex) { }
 try { UserSalt = Convert.ToString(dt.Rows[0][1]); }
 catch (Exception ex) { }
 if (UserPassword == null) // check if user exists in
db
 {
 if (MessageBox.Show("Username was not found in the
database, would you like to create an account?", "Create account",
MessageBoxButtons.YesNo, MessageBoxIcon.Question) == DialogResult.Yes)
```

## Rubik's Cube Solver & Trainer

```
 {
 string salt = GenerateSalt();
 string hashed =
Sha256Hasher.ComputeHash(password + salt);
 _sSqlString = "INSERT INTO Users" +
 "(Username, UserPassword, Salt)"
+
 "VALUES" +
 $"('{username}', '{hashed}',
'{salt}')";
DatabaseUtilities.ExecuteSqlQuery(_sSqlString); // create user
 }

 string hashedPassword =
Sha256Hasher.ComputeHash(password + UserSalt);
 if (hashedPassword == UserPassword)
 {
 MessageBox.Show("Log in Successful!");
 CurrentUser = username;
 foreach (Control control in Controls)
 {
 if (control.Tag != null &&
(control.Tag.ToString() == "MainItems" || control.Tag.ToString() ==
"ColourSelection"))
 {
 control.Visible = true;
 }
 if (control.Tag != null &&
control.Tag.ToString() == "Log In")
 {
 control.Visible = false;
 }
 }
 Size = new Size(953, 463);
 initialiseColourButtons();
 btnBack_Click(sender, e);
 }
 else if (UserPassword != null)
 {
 MessageBox.Show("Incorrect Username or Password");
 }
 }

 private string GenerateSalt()
 {
 const string validCharacters =
"abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789";
 string salt = "";
 Random random = new Random();
 for (int i = 0; i < 16; i++)
```

## Rubik's Cube Solver & Trainer

```
 {
 salt +=
validCharacters[random.Next(validCharacters.Length)];
 }
 return salt;
 }

 private void btnStartTimer_Click(object sender, EventArgs e)
 {
 if (!cube.solved())
 {
 userTimer = new Stopwatch();
 displayUpdater = new Timer();
 displayUpdater.Interval = 200;
 displayUpdater.Tick += updateTimerDisplay;
 userTimer.Start();
 displayUpdater.Start();
 }
 else
 {
 MessageBox.Show("Cube must be in an unsolved state to
start the timer!");
 }
 }

 private void updateTimerDisplay(object sender, EventArgs e)
 {
 TimeSpan currentElapsed = userTimer.Elapsed;
 txtTimer.Text =
 $"{currentElapsed.Minutes}:{currentElapsed.Seconds}:{currentElapsed.Mi
lliseconds}";
 if (cube.solved())
 {
 int millisecondUserTime = (currentElapsed.Minutes * 60
* 1000) + (currentElapsed.Seconds * 60) + currentElapsed.Milliseconds;
 userTimer.Stop();
 displayUpdater.Stop();
 if (CurrentUser != null)
 {
 string _sSqlQuery = $"SELECT userID FROM Users
WHERE Username = '{CurrentUser}'";
 int userID =
 (int)DatabaseUtilities.ExecuteSqlQuery(_sSqlQuery).Rows[0][0];
 _sSqlQuery = "INSERT INTO UserTimes (UserID,
Milliseconds)" +
 $"VALUES({userID},
{millisecondUserTime})";
 DatabaseUtilities.ExecuteSqlQuery(_sSqlQuery);
 }
 }
 }

 private void initialiseColourButtons()
```

## Rubik's Cube Solver & Trainer

```
{
 btn00.BackColor = getColourFromChar(cube.GetPieceColour(0,
0));
 btn01.BackColor = getColourFromChar(cube.GetPieceColour(0,
1));
 btn02.BackColor = getColourFromChar(cube.GetPieceColour(0,
2));
 btn03.BackColor = getColourFromChar(cube.GetPieceColour(0,
3));
 btn04.BackColor = getColourFromChar(cube.GetPieceColour(0,
4));
 btn05.BackColor = getColourFromChar(cube.GetPieceColour(0,
5));
 btn06.BackColor = getColourFromChar(cube.GetPieceColour(0,
6));
 btn07.BackColor = getColourFromChar(cube.GetPieceColour(0,
7));
 btn08.BackColor = getColourFromChar(cube.GetPieceColour(0,
8));

 btn10.BackColor = getColourFromChar(cube.GetPieceColour(1,
0));
 btn11.BackColor = getColourFromChar(cube.GetPieceColour(1,
1));
 btn12.BackColor = getColourFromChar(cube.GetPieceColour(1,
2));
 btn13.BackColor = getColourFromChar(cube.GetPieceColour(1,
3));
 btn14.BackColor = getColourFromChar(cube.GetPieceColour(1,
4));
 btn15.BackColor = getColourFromChar(cube.GetPieceColour(1,
5));
 btn16.BackColor = getColourFromChar(cube.GetPieceColour(1,
6));
 btn17.BackColor = getColourFromChar(cube.GetPieceColour(1,
7));
 btn18.BackColor = getColourFromChar(cube.GetPieceColour(1,
8));

 btn20.BackColor = getColourFromChar(cube.GetPieceColour(2,
0));
 btn21.BackColor = getColourFromChar(cube.GetPieceColour(2,
1));
 btn22.BackColor = getColourFromChar(cube.GetPieceColour(2,
2));
 btn23.BackColor = getColourFromChar(cube.GetPieceColour(2,
3));
 btn24.BackColor = getColourFromChar(cube.GetPieceColour(2,
4));
 btn25.BackColor = getColourFromChar(cube.GetPieceColour(2,
5));
 btn26.BackColor = getColourFromChar(cube.GetPieceColour(2,
6));
```

## Rubik's Cube Solver & Trainer

```
btn27.BackColor = getColourFromChar(cube.GetPieceColour(2,
7));
btn28.BackColor = getColourFromChar(cube.GetPieceColour(2,
8));

btn30.BackColor = getColourFromChar(cube.GetPieceColour(3,
0));
btn31.BackColor = getColourFromChar(cube.GetPieceColour(3,
1));
btn32.BackColor = getColourFromChar(cube.GetPieceColour(3,
2));
btn33.BackColor = getColourFromChar(cube.GetPieceColour(3,
3));
btn34.BackColor = getColourFromChar(cube.GetPieceColour(3,
4));
btn35.BackColor = getColourFromChar(cube.GetPieceColour(3,
5));
btn36.BackColor = getColourFromChar(cube.GetPieceColour(3,
6));
btn37.BackColor = getColourFromChar(cube.GetPieceColour(3,
7));
btn38.BackColor = getColourFromChar(cube.GetPieceColour(3,
8));

btn40.BackColor = getColourFromChar(cube.GetPieceColour(4,
0));
btn41.BackColor = getColourFromChar(cube.GetPieceColour(4,
1));
btn42.BackColor = getColourFromChar(cube.GetPieceColour(4,
2));
btn43.BackColor = getColourFromChar(cube.GetPieceColour(4,
3));
btn44.BackColor = getColourFromChar(cube.GetPieceColour(4,
4));
btn45.BackColor = getColourFromChar(cube.GetPieceColour(4,
5));
btn46.BackColor = getColourFromChar(cube.GetPieceColour(4,
6));
btn47.BackColor = getColourFromChar(cube.GetPieceColour(4,
7));
btn48.BackColor = getColourFromChar(cube.GetPieceColour(4,
8));

btn50.BackColor = getColourFromChar(cube.GetPieceColour(5,
0));
btn51.BackColor = getColourFromChar(cube.GetPieceColour(5,
1));
btn52.BackColor = getColourFromChar(cube.GetPieceColour(5,
2));
btn53.BackColor = getColourFromChar(cube.GetPieceColour(5,
3));
btn54.BackColor = getColourFromChar(cube.GetPieceColour(5,
4));
```

## Rubik's Cube Solver & Trainer

```
 btn55.BackColor = getColourFromChar(cube.GetPieceColour(5,
5));
 btn56.BackColor = getColourFromChar(cube.GetPieceColour(5,
6));
 btn57.BackColor = getColourFromChar(cube.GetPieceColour(5,
7));
 btn58.BackColor = getColourFromChar(cube.GetPieceColour(5,
8));
 }

 private Color getColourFromChar(char colour)
 {
 switch (colour)
 {
 case 'w': return Color.White;
 case 'r': return Color.Red;
 case 'b': return Color.DodgerBlue;
 case 'y': return Color.Yellow;
 case 'o': return Color.Orange;
 case 'g': return Color.ForestGreen;
 default: return Color.Black;
 }
 }

 private char getColourFromIndex()
 {
 char colour = 'p';
 switch(colourSeletionIndex)
 {
 case 0:
 colour = 'r';
 break;
 case 1:
 colour = 'o';
 break;
 case 2:
 colour = 'g';
 break;
 case 3:
 colour = 'b';
 break;
 case 4:
 colour = 'w';
 break;
 case 5:
 colour = 'y';
 break;
 }
 return colour;
 }

 private void btnColourCycle_Click(object sender, EventArgs e)
 {
 colourSeletionIndex++;
 colourSeletionIndex = colourSeletionIndex % 6;
 }
}
```

## Rubik's Cube Solver & Trainer

```
 btnColourCycle.BackColor = colours[colourSeletionIndex];
 pnlCubeDisplay.Invalidate();
 }
 private void btn00_Click(object sender, EventArgs e)
 {
 cube.faces[0, 0].setColour(getColourFromIndex());
 btn00.BackColor = colours[colourSeletionIndex];
 pnlCubeDisplay.Invalidate();
 }

 private void btn01_Click(object sender, EventArgs e)
 {
 cube.faces[0, 1].setColour(getColourFromIndex());
 btn01.BackColor = colours[colourSeletionIndex];
 pnlCubeDisplay.Invalidate();
 }

 private void btn02_Click(object sender, EventArgs e)
 {
 cube.faces[0, 2].setColour(getColourFromIndex());
 btn02.BackColor = colours[colourSeletionIndex];
 pnlCubeDisplay.Invalidate();
 }

 private void btn03_Click(object sender, EventArgs e)
 {
 cube.faces[0, 3].setColour(getColourFromIndex());
 btn03.BackColor = colours[colourSeletionIndex];
 pnlCubeDisplay.Invalidate();
 }

 private void btn05_Click(object sender, EventArgs e)
 {
 cube.faces[0, 5].setColour(getColourFromIndex());
 btn05.BackColor = colours[colourSeletionIndex];
 pnlCubeDisplay.Invalidate();
 }

 private void btn06_Click(object sender, EventArgs e)
 {
 cube.faces[0, 6].setColour(getColourFromIndex());
 btn06.BackColor = colours[colourSeletionIndex];
 pnlCubeDisplay.Invalidate();
 }

 private void btn07_Click(object sender, EventArgs e)
 {
 cube.faces[0, 7].setColour(getColourFromIndex());
 btn07.BackColor = colours[colourSeletionIndex];
 pnlCubeDisplay.Invalidate();
 }

 private void btn08_Click(object sender, EventArgs e)
 {
```

## Rubik's Cube Solver & Trainer

```
 cube.faces[0, 8].setColour(getColourFromIndex());
 btn08.BackColor = colours[colourSelelctionIndex];
 pnlCubeDisplay.Invalidate();
 }

 private void btn10_Click(object sender, EventArgs e)
 {
 cube.faces[1, 0].setColour(getColourFromIndex());
 btn10.BackColor = colours[colourSelelctionIndex];
 pnlCubeDisplay.Invalidate();
 }

 private void btn11_Click(object sender, EventArgs e)
 {
 cube.faces[1, 1].setColour(getColourFromIndex());
 btn11.BackColor = colours[colourSelelctionIndex];
 pnlCubeDisplay.Invalidate();
 }

 private void btn12_Click(object sender, EventArgs e)
 {
 cube.faces[1, 2].setColour(getColourFromIndex());
 btn12.BackColor = colours[colourSelelctionIndex];
 pnlCubeDisplay.Invalidate();
 }

 private void btn13_Click(object sender, EventArgs e)
 {
 cube.faces[1, 3].setColour(getColourFromIndex());
 btn13.BackColor = colours[colourSelelctionIndex];
 pnlCubeDisplay.Invalidate();
 }

 private void btn15_Click(object sender, EventArgs e)
 {
 cube.faces[1, 5].setColour(getColourFromIndex());
 btn15.BackColor = colours[colourSelelctionIndex];
 pnlCubeDisplay.Invalidate();
 }

 private void btn16_Click(object sender, EventArgs e)
 {
 cube.faces[1, 6].setColour(getColourFromIndex());
 btn16.BackColor = colours[colourSelelctionIndex];
 pnlCubeDisplay.Invalidate();
 }

 private void btn17_Click(object sender, EventArgs e)
 {
 cube.faces[1, 7].setColour(getColourFromIndex());
 btn17.BackColor = colours[colourSelelctionIndex];
 pnlCubeDisplay.Invalidate();
 }
}
```

## Rubik's Cube Solver & Trainer

```
private void btn18_Click(object sender, EventArgs e)
{
 cube.faces[1, 8].setColour(getColourFromIndex());
 btn18.BackColor = colours[colourSelelctionIndex];
 pnlCubeDisplay.Invalidate();
}

private void btn20_Click(object sender, EventArgs e)
{
 cube.faces[2, 0].setColour(getColourFromIndex());
 btn20.BackColor = colours[colourSelelctionIndex];
 pnlCubeDisplay.Invalidate();
}

private void btn21_Click(object sender, EventArgs e)
{
 cube.faces[2, 1].setColour(getColourFromIndex());
 btn21.BackColor = colours[colourSelelctionIndex];
 pnlCubeDisplay.Invalidate();
}

private void btn22_Click(object sender, EventArgs e)
{
 cube.faces[2, 2].setColour(getColourFromIndex());
 btn22.BackColor = colours[colourSelelctionIndex];
 pnlCubeDisplay.Invalidate();
}

private void btn23_Click(object sender, EventArgs e)
{
 cube.faces[2, 3].setColour(getColourFromIndex());
 btn23.BackColor = colours[colourSelelctionIndex];
 pnlCubeDisplay.Invalidate();
}

private void btn25_Click(object sender, EventArgs e)
{
 cube.faces[2, 5].setColour(getColourFromIndex());
 btn25.BackColor = colours[colourSelelctionIndex];
 pnlCubeDisplay.Invalidate();
}

private void btn26_Click(object sender, EventArgs e)
{
 cube.faces[2, 6].setColour(getColourFromIndex());
 btn26.BackColor = colours[colourSelelctionIndex];
 pnlCubeDisplay.Invalidate();
}

private void btn27_Click(object sender, EventArgs e)
{
 cube.faces[2, 7].setColour(getColourFromIndex());
 btn27.BackColor = colours[colourSelelctionIndex];
 pnlCubeDisplay.Invalidate();
}
```

## Rubik's Cube Solver & Trainer

```
private void btn28_Click(object sender, EventArgs e)
{
 cube.faces[2, 8].setColour(getColourFromIndex());
 btn28.BackColor = colours[colourSelelctionIndex];
 pnlCubeDisplay.Invalidate();
}

private void btn30_Click(object sender, EventArgs e)
{
 cube.faces[3, 0].setColour(getColourFromIndex());
 btn30.BackColor = colours[colourSelelctionIndex];
 pnlCubeDisplay.Invalidate();
}

private void btn31_Click(object sender, EventArgs e)
{
 cube.faces[3, 1].setColour(getColourFromIndex());
 btn31.BackColor = colours[colourSelelctionIndex];
 pnlCubeDisplay.Invalidate();
}

private void btn32_Click(object sender, EventArgs e)
{
 cube.faces[3, 2].setColour(getColourFromIndex());
 btn32.BackColor = colours[colourSelelctionIndex];
 pnlCubeDisplay.Invalidate();
}

private void btn33_Click(object sender, EventArgs e)
{
 cube.faces[3, 3].setColour(getColourFromIndex());
 btn33.BackColor = colours[colourSelelctionIndex];
 pnlCubeDisplay.Invalidate();
}

private void btn35_Click(object sender, EventArgs e)
{
 cube.faces[3, 5].setColour(getColourFromIndex());
 btn35.BackColor = colours[colourSelelctionIndex];
 pnlCubeDisplay.Invalidate();
}

private void btn36_Click(object sender, EventArgs e)
{
 cube.faces[3, 6].setColour(getColourFromIndex());
 btn36.BackColor = colours[colourSelelctionIndex];
 pnlCubeDisplay.Invalidate();
}

private void btn37_Click(object sender, EventArgs e)
{
 cube.faces[3, 7].setColour(getColourFromIndex());
 btn37.BackColor = colours[colourSelelctionIndex];
}
```

## Rubik's Cube Solver & Trainer

```
 pnlCubeDisplay.Invalidate();
 }

 private void btn38_Click(object sender, EventArgs e)
 {
 cube.faces[3, 8].setColour(getColourFromIndex());
 btn38.BackColor = colours[colourSeletionIndex];
 pnlCubeDisplay.Invalidate();
 }

 private void btn40_Click(object sender, EventArgs e)
 {
 cube.faces[4, 0].setColour(getColourFromIndex());
 btn40.BackColor = colours[colourSeletionIndex];
 pnlCubeDisplay.Invalidate();
 }

 private void btn41_Click(object sender, EventArgs e)
 {
 cube.faces[4, 1].setColour(getColourFromIndex());
 btn41.BackColor = colours[colourSeletionIndex];
 pnlCubeDisplay.Invalidate();
 }

 private void btn42_Click(object sender, EventArgs e)
 {
 cube.faces[4, 2].setColour(getColourFromIndex());
 btn42.BackColor = colours[colourSeletionIndex];
 pnlCubeDisplay.Invalidate();
 }

 private void btn43_Click(object sender, EventArgs e)
 {
 cube.faces[4, 3].setColour(getColourFromIndex());
 btn43.BackColor = colours[colourSeletionIndex];
 pnlCubeDisplay.Invalidate();
 }

 private void btn45_Click(object sender, EventArgs e)
 {
 cube.faces[4, 5].setColour(getColourFromIndex());
 btn45.BackColor = colours[colourSeletionIndex];
 pnlCubeDisplay.Invalidate();
 }

 private void btn46_Click(object sender, EventArgs e)
 {
 cube.faces[4, 6].setColour(getColourFromIndex());
 btn46.BackColor = colours[colourSeletionIndex];
 pnlCubeDisplay.Invalidate();
 }

 private void btn47_Click(object sender, EventArgs e)
 {
```

## Rubik's Cube Solver & Trainer

```
 cube.faces[4, 7].setColour(getColourFromIndex());
 btn47.BackColor = colours[colourSelelctionIndex];
 pnlCubeDisplay.Invalidate();
 }

 private void btn48_Click(object sender, EventArgs e)
 {
 cube.faces[4, 8].setColour(getColourFromIndex());
 btn48.BackColor = colours[colourSelelctionIndex];
 pnlCubeDisplay.Invalidate();
 }

 private void btn50_Click(object sender, EventArgs e)
 {
 cube.faces[5, 0].setColour(getColourFromIndex());
 btn50.BackColor = colours[colourSelelctionIndex];
 pnlCubeDisplay.Invalidate();
 }

 private void btn51_Click(object sender, EventArgs e)
 {
 cube.faces[5, 1].setColour(getColourFromIndex());
 btn51.BackColor = colours[colourSelelctionIndex];
 pnlCubeDisplay.Invalidate();
 }

 private void btn52_Click(object sender, EventArgs e)
 {
 cube.faces[5, 2].setColour(getColourFromIndex());
 btn52.BackColor = colours[colourSelelctionIndex];
 pnlCubeDisplay.Invalidate();
 }

 private void btn53_Click(object sender, EventArgs e)
 {
 cube.faces[5, 3].setColour(getColourFromIndex());
 btn53.BackColor = colours[colourSelelctionIndex];
 pnlCubeDisplay.Invalidate();
 }

 private void btn55_Click(object sender, EventArgs e)
 {
 cube.faces[5, 5].setColour(getColourFromIndex());
 btn55.BackColor = colours[colourSelelctionIndex];
 pnlCubeDisplay.Invalidate();
 }

 private void btn56_Click(object sender, EventArgs e)
 {
 cube.faces[5, 6].setColour(getColourFromIndex());
 btn56.BackColor = colours[colourSelelctionIndex];
 pnlCubeDisplay.Invalidate();
 }
}
```

## Rubik's Cube Solver & Trainer

```
private void btn57_Click(object sender, EventArgs e)
{
 cube.faces[5, 7].setColour(getColourFromIndex());
 btn57.BackColor = colours[colourSeletionIndex];
 pnlCubeDisplay.Invalidate();
}

private void btn58_Click(object sender, EventArgs e)
{
 cube.faces[5, 8].setColour(getColourFromIndex());
 btn58.BackColor = colours[colourSeletionIndex];
 pnlCubeDisplay.Invalidate();
}

private void btnNewCube_Click(object sender, EventArgs e)
{
 cube = new Cube();
 pnlCubeDisplay.Invalidate();
 initialiseColourButtons();
}

private void btnAddAlgorithm_Click(object sender, EventArgs e)
{
 string algorithmName = txtAlgNameInput.Text;
 string algorithmMoves = txtAlgMovesInput.Text;

 if(algorithmName.Length == 0)
 {
 MessageBox.Show("Algorithm must have a name");
 }
 else if(algorithmMoves.Length < 3)
 {
 MessageBox.Show("Algorithm must have at least 3
moves");
 }
 else if(!checkMovesAreValid(algorithmMoves))
 {
 MessageBox.Show("Algorithm must consist of the
following moves: r, l, u, d, f, b, m \nClockwise moves should be upper
case, anticlockwise moves to be lowercase");
 }
 else
 {
 int id = getCurrentAlgId();
 string _sSqlString = "INSERT INTO Algorithms" +
 "(AlgName, AlgCat, CharMoves,
AlgID)" +
 "VALUES" +
 $"('{algorithmName}', 'Custom',
'{algorithmMoves}', {id});";
 DatabaseUtilities.ExecuteSqlNonQuery(_sSqlString);
 MessageBox.Show("Algorithm Successfully added");
 }
}
```

## Rubik's Cube Solver & Trainer

```
 DisplayAlgorithms();
 }

 private bool checkMovesAreValid(string moves)
 {
 char[] validMoves = { 'r', 'l', 'u', 'd', 'f', 'b', 'm' };
 foreach(char c in moves.ToLower())
 {
 if (!validMoves.Contains(c))
 {
 return false;
 }
 }
 return true;
 }

 private int getCurrentAlgId()
 {
 string _sSqlString = "SELECT AlgID FROM Algorithms " +
 "ORDER BY AlgID DESC;";

 DataTable results =
 DatabaseUtilities.ExecuteSqlQuery(_sSqlString);
 return Convert.ToInt32(results.Rows[0][0]) + 1;
 }
}
```

---

Program.cs

---

```
using System;
using System.Windows.Forms;

namespace Rubiks_Cube_Solver_And_Trainer
{
 internal static class Program
 {
 /// <summary>
 /// The main entry point for the application.
 /// </summary>
 [STAThread]
 static void Main()
 {
 Application.EnableVisualStyles();
 Application.SetCompatibleTextRenderingDefault(false);
 Application.Run(new frmMainScreen());
 }
 }
}
```

---

Sha256Hasher.cs

---

```
using System;
using System.Text;
```

# Rubik's Cube Solver & Trainer

```
namespace Rubiks_Cube_Solver_And_Trainer
{
 public static class Sha256Hasher
 {
 public static string ComputeHash(string rawData)
 {
 // Initial hash values (h) defined by the SHA-256 standard
 (NIST).
 uint[] h =
 {
 0x6a09e667, 0xbb67ae85, 0x3c6ef372, 0xa54ff53a,
 0x510e527f, 0x9b05688c, 0x1f83d9ab, 0x5be0cd19
 };

 // Round constants (k) defined by the SHA-256 standard.
 uint[] k =
 {
 0x428a2f98, 0x71374491, 0xb5c0fbcf, 0xe9b5dba5,
 0x3956c25b, 0x59f111f1, 0x923f82a4, 0xab1c5ed5,
 0xd807aa98, 0x12835b01, 0x243185be, 0x550c7dc3,
 0x72be5d74, 0x80deb1fe, 0x9bdc06a7, 0xc19bf174,
 0xe49b69c1, 0xefbe4786, 0x0fc19dc6, 0x240ca1cc,
 0x2de92c6f, 0x4a7484aa, 0x5cb0a9dc, 0x76f988da,
 0x983e5152, 0xa831c66d, 0xb00327c8, 0xbf597fc7,
 0xc6e00bf3, 0xd5a79147, 0x06ca6351, 0x14292967,
 0x27b70a85, 0x2e1b2138, 0x4d2c6dfc, 0x53380d13,
 0x650a7354, 0x766a0abb, 0x81c2c92e, 0x92722c85,
 0xa2bfe8a1, 0xa81a664b, 0xc24b8b70, 0xc76c51a3,
 0xd192e819, 0xd6990624, 0xf40e3585, 0x106aa070,
 0x19a4c116, 0x1e376c08, 0x2748774c, 0x34b0bcb5,
 0x391c0cb3, 0x4ed8aa4a, 0x5b9cca4f, 0x682e6ff3,
 0x748f82ee, 0x78a5636f, 0x84c87814, 0x8cc70208,
 0x90befffa, 0xa4506ceb, 0xbef9a3f7, 0xc67178f2
 };

 // Convert input string to byte array using UTF-8
 encoding.
 byte[] data = Encoding.UTF8.GetBytes(rawData);
 ulong bitLength = (ulong)data.Length * 8; // Calculate
 the bit length of the input.

 // Pad the input message to make it compatible with
 SHA-256 block size.
 data = PadMessage(data, bitLength);

 // Process each 512-bit block.
 for (int i = 0; i < data.Length; i += 64)
 {
 uint[] w = new uint[64]; // Message schedule array.

 // Initialize first 16 words with the current 512-bit
 block.
 for (int j = 0; j < 16; j++)
 {
```

## Rubik's Cube Solver & Trainer

```
 w[j] = (uint)((data[i + 4 * j] << 24) | (data[i +
4 * j + 1] << 16) | (data[i + 4 * j + 2] << 8) | data[i + 4 * j + 3]);
 }

 // Extend the first 16 words into the remaining 48
words of the schedule array.
 for (int j = 16; j < 64; j++)
 {
 uint s0 = RightRotate(w[j - 15], 7) ^
RightRotate(w[j - 15], 18) ^ (w[j - 15] >> 3);
 uint s1 = RightRotate(w[j - 2], 17) ^
RightRotate(w[j - 2], 19) ^ (w[j - 2] >> 10);
 w[j] = w[j - 16] + s0 + w[j - 7] + s1;
 }

 // Initialize working variables with current hash
values.
 uint a = h[0], b = h[1], c = h[2], d = h[3], e = h[4],
f = h[5], g = h[6], hh = h[7];

 // Main compression loop.
 for (int j = 0; j < 64; j++)
 {
 uint S1 = RightRotate(e, 6) ^ RightRotate(e, 11) ^
RightRotate(e, 25);
 uint ch = (e & f) ^ ((~e) & g);
 uint temp1 = hh + S1 + ch + k[j] + w[j];
 uint S0 = RightRotate(a, 2) ^ RightRotate(a, 13) ^
RightRotate(a, 22);
 uint maj = (a & b) ^ (a & c) ^ (b & c);
 uint temp2 = S0 + maj;

 hh = g; g = f; f = e; e = d + temp1;
 d = c; c = b; b = a; a = temp1 + temp2;
 }

 // Add the compressed chunk to the current hash value.
 h[0] += a; h[1] += b; h[2] += c; h[3] += d;
 h[4] += e; h[5] += f; h[6] += g; h[7] += hh;
}

// Convert hash values to hexadecimal string.
return string.Concat(Array.ConvertAll(h, x =>
x.ToString("x8")));
}
private static byte[] PadMessage(byte[] data, ulong bitLength)
{
 int padLength = 64 - (data.Length % 64); // Determine
padding length.
 if (padLength < 9) padLength += 64; // Ensure enough
space for bit length.

 byte[] paddedData = new byte[data.Length + padLength];
```

## Rubik's Cube Solver & Trainer

```
 Array.Copy(data, paddedData, data.Length); // Copy
original data.
 paddedData[data.Length] = 0x80; // Append 1 bit followed
by zeros.

 // Append the original bit length as a 64-bit big-endian
integer.
 for (int i = paddedData.Length - 8; i < paddedData.Length;
i++)
 {
 paddedData[i] = (byte)((bitLength >>
((paddedData.Length - 1 - i) * 8)) & 0xff);
 }

 return paddedData;
 }
 private static uint RightRotate(uint value, int count)
 {
 return (value >> count) | (value << (32 - count));
 }
}
```

---

Solver.cs

---

```
using System;
using System.Collections.Generic;
using System.Data;

namespace Rubiks_Cube_Solver_And_Trainer
{
 public class IDAStarWhiteCross
 {
 private List<int> solutionPath;

 public List<int> IDAStar(Cube start)
 {
 int threshold = start.Heuristic();
 solutionPath = new List<int>();

 while (true)
 {
 int temp = DepthLimitedSearch(start, 0, threshold);
 if (temp == int.MaxValue) return null; // No solution
found
 if (temp == -1) return solutionPath; // Solution
found
 threshold = temp; // Increase
threshold
 }
 }
 }
}
```

## Rubik's Cube Solver & Trainer

```
private int DepthLimitedSearch(Cube node, int g, int
threshold)
{
 int f = g + node.Heuristic();

 if (f > threshold) return f; // Exceeded threshold, prune
this path
 if (node.WhiteCrossSolved())
 {
 return -1; // Solution
found signal
 }

 int minimum = int.MaxValue;
 foreach (var move in node.PossibleMoves)
 {
 Cube child = ApplyMove(node, move);
 solutionPath.Add(move);
 int temp = DepthLimitedSearch(child, g + 1,
threshold);
 if (temp == -1) { return -1; } // Solution found
 if (temp < minimum) { minimum = temp; } // Track
minimum threshold that exceeded
 solutionPath.RemoveAt(solutionPath.Count() - 1);
 }

 return minimum; // Return minimum threshold that was
exceeded
}

private Cube ApplyMove(Cube node, int move)
{
 Cube nextState = node.Clone(node);
 bool AntiClockwise = false;
 if (move > 5)
 {
 move = move - 6;
 AntiClockwise = true;
 }

 nextState.RotateFace(move, AntiClockwise);
 return nextState;
}

public class Solver
{
 string[] F2LAlgorithms;
 string[] PLLAlgorithms;
 string[] OLLAlgorithms;
 public Solver()
 {
```

## Rubik's Cube Solver & Trainer

```
string sqlquery;
sqlquery = "SELECT CharMoves " +
 "FROM Algorithms " +
 "WHERE AlgCat = 'F2L' " +
 "ORDER BY AlgID ASC;";

DataTable dt =
DatabaseUtilities.ExecuteSqlQuery(sqlquery);
F2LAlgorithms = new string[dt.Rows.Count];
for(int i = 0; i < (dt.Rows.Count); i++)
{
 F2LAlgorithms[i] = Convert.ToString(dt.Rows[i][0]);
}
sqlquery = "SELECT CharMoves " +
 "FROM Algorithms " +
 "WHERE AlgCat = 'OLL' " +
 "ORDER BY AlgID ASC;";

dt = null;
dt = DatabaseUtilities.ExecuteSqlQuery(sqlquery);
OLLAlgorithms = new string[dt.Rows.Count];
for (int i = 0; i < (dt.Rows.Count); i++)
{
 OLLAlgorithms[i] = Convert.ToString(dt.Rows[i][0]);
}
sqlquery = "SELECT CharMoves " +
 "FROM Algorithms " +
 "WHERE AlgCat = 'PLL' " +
 "ORDER BY AlgID ASC;";

dt = null;
dt = DatabaseUtilities.ExecuteSqlQuery(sqlquery);
PLLAlgorithms = new string[dt.Rows.Count];
for (int i = 0; i < (dt.Rows.Count); i++)
{
 PLLAlgorithms[i] = Convert.ToString(dt.Rows[i][0]);
}
}

// check solvability
public bool IsCubeSolvable(Cube cube)
{
 Cube clone = cube.Clone(cube);
 if (clone.faces[2,4].getColour() == 'y' ||
clone.faces[3,4].getColour() == 'y')
 {
 clone.rotateDisplayLeft();
 }
 while (clone.faces[4,4].getColour() != 'w')
 {
 clone.rotateDisplayUp();
 }
 while (clone.faces[0, 4].getColour() != 'r')
 {
 clone.rotateDisplayLeft();
 }
 if (!CheckColorCount(clone))
```

## Rubik's Cube Solver & Trainer

```
 return false;

 if (!CheckCornerOrientation(clone))
 return false;

 if (!CheckEdgeOrientation(clone))
 return false;

 if (!CheckPermutationParity(clone))
 return false;

 return true;
}

public bool CheckColorCount(Cube cube)
{
 int redCount = 0;
 int orangeCount = 0;
 int greenCount = 0;
 int blueCount = 0;
 int whiteCount = 0;
 int yellowCount = 0;

 foreach (tile tile in cube.faces)
 {
 switch (tile.getColour())
 {
 case 'r':
 redCount++;
 break;
 case 'o':
 orangeCount++;
 break;
 case 'g':
 greenCount++;
 break;
 case 'b':
 blueCount++;
 break;
 case 'w':
 whiteCount++;
 break;
 case 'y':
 yellowCount++;
 break;
 }
 }

 if (redCount > 9 || orangeCount > 9 || greenCount > 9 ||
blueCount > 9 || whiteCount > 9 || yellowCount > 9)
 {
 return false; // impossible numbers
 }
}
```

## Rubik's Cube Solver & Trainer

```
 else
 {
 return true;
 }
 }
 public bool CheckCornerOrientation(Cube cube)
 {
 int[] cornerOrientations = new int[8]; // 0 = no rotation,
 1 = 120 degrees, 2 = 240 degrees
 int index = 0;
 int[] cornerIndexes = { 0, 2, 6, 8 };
 int face2, face3, pos2, pos3;
 foreach (int i in cornerIndexes)
 {
 (face2, pos2, face3, pos3) = CornerEdgeLookup[(0, i)];

 char color1 = cube.faces[0, i].getColour();
 char color2 = cube.faces[face2, pos2].getColour();
 char color3 = cube.faces[face3, pos3].getColour();
 cornerOrientations[index++] =
DetermineCornerOrientation(color1, color2, color3);
 }
 foreach (int i in cornerIndexes)
 {
 (face2, pos2, face3, pos3) = CornerEdgeLookup[(1, i)];

 char color1 = cube.faces[1, i].getColour();
 char color2 = cube.faces[face2, pos2].getColour();
 char color3 = cube.faces[face3, pos3].getColour();
 cornerOrientations[index++] =
DetermineCornerOrientation(color1, color2, color3);
 }

 int cornerOrientationSum = 0;
 for (int i = 0; i < 8; i++)
 {
 cornerOrientationSum += cornerOrientations[i];
 }
 if (cornerOrientationSum % 3 != 0)
 {
 return false; // Unsolvble configuration
 }

 return true;
 }

 public int DetermineCornerOrientation(char color1, char
color2, char color3)
 {
 if(color1 == 'r' || color1 == 'o')
 {
 return 0;
 }
 else if (color2 == 'r' || color2 == 'o')
```

## Rubik's Cube Solver & Trainer

```
 {
 return 1;
 }
 else
 {
 return 2;
 }
 }

 public bool CheckEdgeOrientation(Cube cube)
 {
 int[] edgeOrientations = new int[12]; // 0 = no rotation, 1
= flipped,
 int index = 0;
 int[] primaryEdgeIndices = { 1, 3, 5, 7 };
 int[] eEdgeIndices = { 3, 5 };
 foreach(int i in primaryEdgeIndices)
 {
 if (cube.faces[4,i].getColour() == 'r' ||
cube.faces[4,i].getColour() == 'o')
 {
 edgeOrientations[index++] = 1;
 }
 else if(cube.faces[4, i].getColour() == 'g' ||
cube.faces[4,i].getColour() == 'b')
 {
 (int face, int pos) = AdjacentEdgeLookup[(4, i)];
 if (cube.faces[face, pos].getColour() == 'y' ||
cube.faces[face, pos].getColour() == 'w')
 {
 edgeOrientations[index++] = 1;
 }
 else
 {
 edgeOrientations[index++] = 0;
 }
 }
 else
 {
 edgeOrientations[index++] = 0;
 }
 }
 foreach (int i in primaryEdgeIndices)
 {
 if (cube.faces[5, i].getColour() == 'r' ||
cube.faces[5, i].getColour() == 'o')
 {
 edgeOrientations[index++] = 1;
 }
 else if (cube.faces[5, i].getColour() == 'g' ||
cube.faces[5, i].getColour() == 'b')
 {
 (int face, int pos) = AdjacentEdgeLookup[(5, i)];
```

## Rubik's Cube Solver & Trainer

```
 if (cube.faces[face, pos].getColour() == 'y' ||
cube.faces[face, pos].getColour() == 'w')
 {
 edgeOrientations[index++] = 1;
 }
 else
 {
 edgeOrientations[index++] = 0;
 }
 }
 else
 {
 edgeOrientations[index++] = 0;
 }
}
foreach (int i in eEdgeIndices)
{
 if (cube.faces[0, i].getColour() == 'r' ||
cube.faces[0, i].getColour() == 'o')
 {
 edgeOrientations[index++] = 1;
 }
 else if (cube.faces[0, i].getColour() == 'g' ||
cube.faces[0, i].getColour() == 'b')
 {
 (int face, int pos) = AdjacentEdgeLookup[(0, i)];
 if (cube.faces[face, pos].getColour() == 'y' ||
cube.faces[face, pos].getColour() == 'w')
 {
 edgeOrientations[index++] = 1;
 }
 else
 {
 edgeOrientations[index++] = 0;
 }
 }
 else
 {
 edgeOrientations[index++] = 0;
 }
}
foreach (int i in eEdgeIndices)
{
 if (cube.faces[1, i].getColour() == 'r' ||
cube.faces[1, i].getColour() == 'o')
 {
 edgeOrientations[index++] = 1;
 }
 else if (cube.faces[1, i].getColour() == 'g' ||
cube.faces[1, i].getColour() == 'b')
 {
 (int face, int pos) = AdjacentEdgeLookup[(1, i)];
 if (cube.faces[face, pos].getColour() == 'y' ||
cube.faces[face, pos].getColour() == 'w')
```

## Rubik's Cube Solver & Trainer

```
 {
 edgeOrientations[index++] = 1;
 }
 else
 {
 edgeOrientations[index++] = 0;
 }
 }
 else
 {
 edgeOrientations[index++] = 0;
 }
}
int edgeOrientationSum = 0;
for (int i = 0; i < 12; i++)
{
 edgeOrientationSum += edgeOrientations[i];
}
if (edgeOrientationSum % 2 != 0)
{
 return false; // Unsolvble configuration
}
return true;
}

// Method to count cycle swaps needed for sorting the
permutation
private static int CountCycleSwaps(int[] permutation)
{
 bool[] visited = new bool[permutation.Length];
 int swaps = 0;

 for (int i = 0; i < permutation.Length; i++)
 {
 if (!visited[i])
 {
 int current = i;
 int cycleSize = 0;

 while (!visited[current])
 {
 visited[current] = true;
 current = permutation[current];
 cycleSize++;
 }

 if (cycleSize > 1)
 {
 swaps += cycleSize - 1;
 }
 }
 }
 return swaps;
}
```

## Rubik's Cube Solver & Trainer

```
// Method to check if the permutation of corners and edges is
valid (parity check)
private bool CheckPermutationParity(Cube cube)
{
 int[] cornerPermutation = GetCornerPermutation(cube);
 int[] edgePermutation = GetEdgePermutation(cube);
 int cornerSwaps = CountCycleSwaps(cornerPermutation);
 int edgeSwaps = CountCycleSwaps(edgePermutation);

 bool cornerParity = (cornerSwaps % 2 == 0);
 bool edgeParity = (edgeSwaps % 2 == 0);

 return cornerParity == edgeParity; // True if both have the
same parity, false otherwise
}
private int[] GetEdgePermutation(Cube cube)
{
 int[] edgePermutation = new int[12];
 int index = 0;
 int[] edgeIndexes = { 1, 3, 5, 7 };
 int[] extraEdgeIndexes = { 3, 5 };
 int face2, pos2;
 foreach(int i in edgeIndexes) // red face
 {
 (face2, pos2) = AdjacentEdgeLookup[(0, i)];
 char color1 = cube.faces[0, i].getColour(); // Get the
color of the adjacent sticker
 char color2 = cube.faces[face2, pos2].getColour(); //
Get the color of the adjacent sticker
 edgePermutation[index++] = DetermineEdgeIndex(color1,
color2);
 }
 foreach (int i in edgeIndexes) // orange face
 {
 (face2, pos2) = AdjacentEdgeLookup[(1, i)];
 char color1 = cube.faces[1, i].getColour(); // Get the
color of the adjacent sticker
 char color2 = cube.faces[face2, pos2].getColour(); //
Get the color of the adjacent sticker
 edgePermutation[index++] = DetermineEdgeIndex(color1,
color2);
 }
 foreach (int i in extraEdgeIndexes) // white face
 {
 (face2, pos2) = AdjacentEdgeLookup[(4, i)];
 char color1 = cube.faces[4, i].getColour(); // Get the
color of the adjacent sticker
 char color2 = cube.faces[face2, pos2].getColour(); //
Get the color of the adjacent sticker
 edgePermutation[index++] = DetermineEdgeIndex(color1,
color2);
 }
 foreach (int i in extraEdgeIndexes) // yellow face
```

## Rubik's Cube Solver & Trainer

```
{
 (face2, pos2) = AdjacentEdgeLookup[(5, i)];
 char color1 = cube.faces[5, i].getColour(); // Get the
color of the adjacent sticker
 char color2 = cube.faces[face2, pos2].getColour(); //
Get the color of the adjacent sticker
 edgePermutation[index++] = DetermineEdgeIndex(color1,
color2);
}

return edgePermutation;
}

private int DetermineEdgeIndex(char color1, char color2)
{
 // Define a mapping of color pairs to edge indices (solved
state).
 Dictionary<(char, char), int> edgeMapping = new
Dictionary<(char, char), int>
 {
 {('g', 'w'), 8},
 {('r', 'w'), 0},
 {('b', 'w'), 9},
 {('o', 'w'), 7},
 {('g', 'y'), 10},
 {('r', 'y'), 3},
 {('b', 'y'), 11},
 {('o', 'y'), 4},
 {('g', 'r'), 1},
 {('b', 'r'), 2},
 {('b', 'o'), 6},
 {('g', 'o'), 5}
 };

 // Ensure consistent ordering of colors (alphabetical, for
example)
 (char, char) sortedColors = (color1.CompareTo(color2) < 0)
? (color1, color2) : (color2, color1);
 return edgeMapping.ContainsKey(sortedColors) ?
edgeMapping[sortedColors] : -1;
}

int[] GetCornerPermutation(Cube cube)
{
 int[] cornerPermutation = new int[8];
 int index = 0;
 int[] cornerIndexes = { 0, 2, 6, 8 };
 int face2, face3, pos2, pos3;
 foreach(int i in cornerIndexes)
 {
 (face2, pos2, face3, pos3) = CornerEdgeLookup[(0, i)];

 char color1 = cube.faces[0, i].getColour();
 char color2 = cube.faces[face2, pos2].getColour();
 char color3 = cube.faces[face3, pos3].getColour();
 }
}
```

## Rubik's Cube Solver & Trainer

```
 cornerPermutation[index++] =
DetermineCornerIndex(color1, color2, color3);
 }
 foreach (int i in cornerIndexes)
 {
 (face2, pos2, face3, pos3) = CornerEdgeLookup[(1, i)];

 char color1 = cube.faces[1, i].getColour();
 char color2 = cube.faces[face2, pos2].getColour();
 char color3 = cube.faces[face3, pos3].getColour();
 cornerPermutation[index++] =
DetermineCornerIndex(color1, color2, color3);
 }
 return cornerPermutation;
}
private int DetermineCornerIndex(char color1, char color2, char
color3)
{
 // Define a mapping of color triplets to corner indices
(solved state).
 Dictionary<(char, char, char), int> cornerMapping = new
Dictionary<(char, char, char), int>
 {
 {('g', 'r', 'w'), 0},
 {('b', 'r', 'w'), 1},
 {('b', 'o', 'w'), 7},
 {('g', 'o', 'w'), 6},
 {('g', 'r', 'y'), 2},
 {('b', 'r', 'y'), 3},
 {('b', 'o', 'y'), 5},
 {('g', 'o', 'y'), 4}
 };

 // Sort colors to a consistent order for lookup
 char[] sortedColors = new char[] { color1, color2, color3
};
 Array.Sort(sortedColors);
 return cornerMapping.ContainsKey((sortedColors[0],
sortedColors[1], sortedColors[2])) ?
 cornerMapping[(sortedColors[0], sortedColors[1],
sortedColors[2])] : -1;
}
//F2L
public Cube AltSexyMove(Cube cube, int faceIndex, ref List<int>
moves)
{
 switch (faceIndex)
 {
 case 0:
 cube.RotateFace(2, true);
 cube.RotateFace(4, false);
 cube.RotateFace(2, false);
 cube.RotateFace(4, true);
 moves.AddRange(new int[4] { 8, 4, 2, 10 });
 }
}
```

## Rubik's Cube Solver & Trainer

```
 break;
 case 3:
 cube.RotateFace(0, true);
 cube.RotateFace(4, false);
 cube.RotateFace(0, false);
 cube.RotateFace(4, true);
 moves.AddRange(new int[4] { 6, 4, 0, 10 });
 break;
 case 1:
 cube.RotateFace(3, true);
 cube.RotateFace(4, false);
 cube.RotateFace(3, false);
 cube.RotateFace(4, true);
 moves.AddRange(new int[4] { 9, 4, 3, 10 });
 break;
 case 2:
 cube.RotateFace(1, true);
 cube.RotateFace(4, false);
 cube.RotateFace(1, false);
 cube.RotateFace(4, true);
 moves.AddRange(new int[4] { 7, 4, 1, 10 });
 break;
 }
 return cube;
}
}
public Cube SexyMove(Cube cube, int faceIndex, ref List<int>
moves)
{
 switch (faceIndex)
 {
 case 0:
 cube.RotateFace(3, false);
 cube.RotateFace(4, false);
 cube.RotateFace(3, true);
 cube.RotateFace(4, true);
 moves.AddRange(new int[4] { 3, 4, 9, 10 });
 break;
 case 3:
 cube.RotateFace(1, false);
 cube.RotateFace(4, false);
 cube.RotateFace(1, true);
 cube.RotateFace(4, true);
 moves.AddRange(new int[4] { 1, 4, 7, 10 });
 break;
 case 1:
 cube.RotateFace(2, false);
 cube.RotateFace(4, false);
 cube.RotateFace(2, true);
 cube.RotateFace(4, true);
 moves.AddRange(new int[4] { 2, 4, 8, 10 });
 break;
 case 2:
 cube.RotateFace(0, false);
 cube.RotateFace(4, false);
```

## Rubik's Cube Solver & Trainer

```
 cube.RotateFace(0, true);
 cube.RotateFace(4, true);
 moves.AddRange(new int[4] { 0, 4, 6, 10 });
 break;
 }
 return cube;
}
private Cube getCornerInPosition(Cube cube, int pairNumber, ref
List<int> moves)
{
 //find correct corner
 char[,] edgeComposition = new char[4, 2]
 {
 { 'b', 'r' },
 { 'r', 'g' },
 { 'g', 'o' },
 { 'o', 'b' },
 };
 Dictionary<int, int> facePairs = new Dictionary<int, int>()
 {
 { 0, 1 },
 { 1, 0 },
 { 2, 3 },
 { 3, 2 },
 { 4, 5 },
 { 5, 4 },
 };
 int[] corners = new int[4]
 {
 0, 2, 6, 8
 };
 int face1;
 int index1;
 int index2;
 int face2;
 char colour1 = edgeComposition[pairNumber, 0];
 char colour2 = edgeComposition[pairNumber, 1];

 int edgeFace = -1;
 if (edgeInTopLayer(cube, pairNumber))
 {
 int[] edgeIndices = { 1, 3, 5, 7 };
 foreach (int i in edgeIndices)
 {
 if (cube.faces[4, i].getColour() == colour1)
 {
 (int face, int index) = getAdjacentEdgeIndex(4,
i);
 if (cube.faces[face, index].getColour() ==
colour2)
 {
 edgeFace = face;
 }
 }
 }
 }
}
```

## Rubik's Cube Solver & Trainer

```

 else if (cube.faces[4, i].getColour() == colour2)
 {
 (int face, int index) = getAdjacentEdgeIndex(4,
i);
 if (cube.faces[face, index].getColour() ==
colour1)
 {
 edgeFace = face;
 }
 }
 }

 for (int i = 0; i < 6; i++)
 {
 for (int j = 0; j < 4; j++)
 {
 if (cube.faces[i, corners[j]].getColour() == 'w')
 {
 (face1, index1, face2, index2) =
CornerEdgeLookup[(i, corners[j])];
 if (cube.faces[face1, index1].getColour() ==
colour1 || cube.faces[face1, index1].getColour() == colour2)
 {
 if (cube.faces[face2, index2].getColour()
== colour1 || cube.faces[face2, index2].getColour() == colour2) //
found correct corner
 {
 if (facePairs[i] == edgeFace && !(face2
== 4 || face1 == 4)) // if sexy move affects edge colour and corner is
not already in top layer
 {
 cube.RotateFace(4, false);
 moves.Add(4);
 }

 switch (i)
 {
 case 0:
 switch (corners[j])
 {
 case 0:
 cube.RotateFace(4,
true);

 moves.Add(10);
 break;
 case 2:
 return cube;
 case 6:
 cube =
AltSexyMove(cube, i, ref moves);

 break;
 case 8:

```

# Rubik's Cube Solver & Trainer

```

 return cube;
 }
 break;

 case 1:
 switch (corners[j])
 {
 case 0:
 cube = SexyMove(cube,
 i, ref moves);

 break;
 case 2:
 cube =
 AltSexyMove(cube, i, ref moves);

 break;
 case 6:
 cube.RotateFace(4,
 false);

 cube.RotateFace(4,
 false);

 moves.Add(4);
 moves.Add(4);
 break;
 case 8:
 cube.RotateFace(4,
 true);

 moves.Add(10);
 break;
 }
 break;

 case 2:
 switch (corners[j])
 {
 case 0:
 cube.RotateFace(4,
 false);

 cube.RotateFace(4,
 false);

 moves.Add(4);
 moves.Add(4);
 break;
 case 2:
 cube.RotateFace(4,
 true);

 moves.Add(10);
 break;
 case 6:
 cube =
 AltSexyMove(cube, i, ref moves);

 break;
 case 8:
 cube.RotateFace(4,
 false);

 cube.RotateFace(4,
 false);

 moves.Add(4);
 moves.Add(4);
 break;
 case 10:
 cube.RotateFace(4,
 true);

 moves.Add(10);
 break;
 }
 break;
 }
}

```

# Rubik's Cube Solver & Trainer

```

 cube = SexyMove(cube,
i, ref moves);
 break;
 }
 break;
case 3:
 switch (corners[j])
 {
 case 0:
 return cube;
 case 2:
 cube.RotateFace(4,
 moves.Add(10);
 break;
 case 6:
 return cube;
 case 8:
 cube = SexyMove(cube, i
 break;
 }
 break;
case 4:
 switch (corners[j])
 {
 case 0:
 cube.RotateFace(4,
 cube.RotateFace(4,
 moves.Add(4);
 moves.Add(4);
 break;
 case 2:
 cube.RotateFace(4,
 moves.Add(4);
 break;
 case 6:
 cube.RotateFace(4,
 moves.Add(10);
 break;
 case 8:
 return cube;
 }
 break;
case 5:
 switch (corners[j])

```

# Rubik's Cube Solver & Trainer

```

 {
 case 0:
 cube =
 break;
 case 2:
 return cube;
 case 6:
 cube = SexyMove(cube,
 break;
 case 8:
 cube = SexyMove(cube, 3
 break;
 }
 break;
 }
}
}
}
}
}
}
cube = getCornerInPosition(cube, pairNumber, ref moves);
return cube;
}

private Cube getEdgeInPosition(Cube cube, int pairnumber, ref
List<int> fullSolution)
{
 if (edgeInCorrectColumn(cube, pairnumber))
 {
 return cube;
 }
 else if (edgeInTopLayer(cube, pairnumber))
 {
 fullSolution.AddRange(rotateEdgeInTopLayer(cube,
pairnumber));
 }
 else
 {
 fullSolution.AddRange(getEdgeIntoTopLayer(cube,
pairnumber));
 fullSolution.AddRange(rotateEdgeInTopLayer(cube,
pairnumber));
 }
 return cube;
}

private bool edgeInCorrectColumn(Cube cube, int pairnumber)
{
 char[,] edgeComposition = new char[4, 2]
 {

```

## Rubik's Cube Solver & Trainer

```
 {'b', 'r'},
 {'r', 'g'},
 {'g', 'o'},
 {'o', 'b'},
 };
 char color1 = edgeComposition[pairnumber, 0];
 char color2 = edgeComposition[pairnumber, 1];
 if (cube.faces[0,5].getColour() == color1)
 {
 if (cube.faces[3, 3].getColour() == color2)
 {
 return true;
 }
 }
 else if (cube.faces[0, 5].getColour() == color2)
 {
 if (cube.faces[3, 3].getColour() == color1)
 {
 return true;
 }
 }

 return false;
}

private bool edgeInTopLayer(Cube cube, int pairnumber)
{
 char[,] edgeComposition = new char[4, 2]
 {
 {'b', 'r'},
 {'r', 'g'},
 {'g', 'o'},
 {'o', 'b'},
 };
 char color1 = edgeComposition[pairnumber, 0];
 char color2 = edgeComposition[pairnumber, 1];
 int[] edgeIndices = { 1, 3, 5, 7 };
 foreach(int i in edgeIndices)
 {
 if (cube.faces[4,i].getColour() == color1)
 {
 (int face, int index) = getAdjacentEdgeIndex(4, i);
 if (cube.faces[face, index].getColour() == color2)
 {
 return true;
 }
 }
 else if (cube.faces[4, i].getColour() == color2)
 {
 (int face, int index) = getAdjacentEdgeIndex(4, i);
 if (cube.faces[face, index].getColour() == color1)
 {
 return true;
 }
 }
 }
}
```

## Rubik's Cube Solver & Trainer

```
 }
 }
 return false;
}

private List<int> rotateEdgeInTopLayer(Cube cube, int
pairnumber)
{
 List<int> moves = new List<int>();
 char[,] edgeComposition = new char[4, 2]
 {
 {'b', 'r'},
 {'r', 'g'},
 {'g', 'o'},
 {'o', 'b'},
 };
 char color1 = edgeComposition[pairnumber, 0];
 char color2 = edgeComposition[pairnumber, 1];
 while (true)
 {
 if (cube.faces[0, 1].getColour() == color1)
 {
 if (cube.faces[4, 7].getColour() == color2)
 {
 return moves;
 }
 }
 if (cube.faces[3, 1].getColour() == color2)
 {
 if (cube.faces[4, 5].getColour() == color1)
 {
 return moves;
 }
 }
 moves.Add(4);
 cube.RotateFace(4, false);
 }
}

private List<int> getEdgeIntoTopLayer(Cube cube, int
pairnumber)
{
 List<int> moves = new List<int>();
 char[,] edgeComposition = new char[4, 2]
 {
 {'b', 'r'},
 {'r', 'g'},
 {'g', 'o'},
 {'o', 'b'},
 };
 char color1 = edgeComposition[pairnumber, 0];
 char color2 = edgeComposition[pairnumber, 1];
```

## Rubik's Cube Solver & Trainer

```
 if (cube.faces[3, 5].getColour() == color1 ||
cube.faces[3, 5].getColour() == color2)
 {
 if (cube.faces[1, 5].getColour() == color1 ||
cube.faces[1, 5].getColour() == color2)
 {
 SexyMove(cube, 3, ref moves);
 }
 }

 if (cube.faces[1, 3].getColour() == color1 ||
cube.faces[1, 3].getColour() == color2)
 {
 if (cube.faces[2, 3].getColour() == color1 ||
cube.faces[2, 3].getColour() == color2)
 {
 SexyMove(cube, 1, ref moves);
 }
 }

 if (cube.faces[2, 5].getColour() == color1 ||
cube.faces[2, 5].getColour() == color2)
 {
 if (cube.faces[0, 3].getColour() == color1 ||
cube.faces[0, 3].getColour() == color2)
 {
 SexyMove(cube, 2, ref moves);
 }
 }
 return moves;
 }
 public (List<int>, List<int>) f2lbruteforce(Cube cube, int
pairnumber)
 {
 List<int> solution = new List<int>();
 List<int> fullSolution = new List<int>();
 cube = getEdgeInPosition(cube, pairnumber, ref
fullSolution);
 cube = getCornerInPosition(cube, pairnumber, ref
fullSolution);
 int AlgNum = 0;
 Cube clone = cube.Clone(cube);
 foreach (string moves in F2LAlgorithms)
 {
 if (clone.F2LPairSolved(pairnumber))
 {
 return (solution, fullSolution);
 }
 AlgNum++;
 clone = cube.Clone(cube);
 foreach (char move in moves)
 {
 (int decodedMove, bool AntiClockwise) =
decodeCharMove(move);
```

## Rubik's Cube Solver & Trainer

```
 if (decodedMove < 6)
 {
 clone.RotateFace(decodedMove, AntiClockwise);
 }
 else
 {
 if (AntiClockwise)
 {
 clone.RotateHorizontalMiddleLayer();
 clone.RotateHorizontalMiddleLayer();
 clone.RotateHorizontalMiddleLayer(); //3
times for the same as a prime rotation
 }
 else
 {
 clone.RotateHorizontalMiddleLayer();
 }
 clone.RotateFace(5, AntiClockwise);
 if (decodedMove == 7)
 {
 clone.RotateFace(4, AntiClockwise);
 }
 }
 }

 if (clone.F2LPairSolved(pairnumber))
 {
 foreach (char move in moves)
 {
 (int decodedMove, bool AntiClockwise) =
decodeCharMove(move);
 if (decodedMove == 6 || decodedMove == 7)
 {
 if (AntiClockwise)
 {
 solution.Add(decodedMove + 20); // put
special moves outside of standard move set
 }
 else
 {
 solution.Add(decodedMove + 10); // put
special moves outside of standard move set
 }
 }
 else if (AntiClockwise)
 {
 solution.Add(decodedMove + 6);
 }
 else
 {
 solution.Add(decodedMove);
 }
 }
 break;
 }
}
```

## Rubik's Cube Solver & Trainer

```
 }
 }
 if (!clone.F2LPairSolved(pairnumber))
 {
 solution = new List<int>();

 }
 return (solution, fullSolution);
}

public (int, bool) decodeCharMove(char move)
{
 int decodedMove = -1;
 bool anticlockwise = false;
 if (char.IsUpper(move))
 {
 anticlockwise = false;
 switch (move)
 {
 case 'R':
 decodedMove = 3;
 break;
 case 'L':
 decodedMove = 2;
 break;
 case 'U':
 decodedMove = 4;
 break;
 case 'F':
 decodedMove = 0;
 break;
 case 'B':
 decodedMove = 1;
 break;
 case 'D': //since D is never used in f2l, it can
be used for a d-slice instead
 decodedMove = 6;
 break;
 case 'Y':
 decodedMove = 7;
 break;
 }
 }
 else
 {
 anticlockwise = true;
 switch (move)
 {
 case 'r':
 decodedMove = 3;
 break;
 case 'l':
 decodedMove = 2;
 break;
 }
 }
}
```

## Rubik's Cube Solver & Trainer

```
 break;
 case 'u':
 decodedMove = 4;
 break;
 case 'f':
 decodedMove = 0;
 break;
 case 'b':
 decodedMove = 1;
 break;
 case 'd': //since D is never used in f2l, it can
be used for a d-slice instead
 decodedMove = 6;
 break;
 case 'y':
 decodedMove = 7;
 break;
 }
}
return (decodedMove, anticlockwise);
}

 public Dictionary<(int, int), (int, int)> AdjacentEdgeLookup =
new Dictionary<(int, int), (int, int)>()
{
 {(0,1), (4,7)},
 {(0,3), (2,5)},
 {(0,5), (3,3)},
 {(0,7), (5,1)},
 {(1,1), (5,7)},
 {(1,3), (2,3)},
 {(1,5), (3,5)},
 {(1,7), (4,1)},
 {(3,1), (4,5)},
 {(3,3), (0,5)},
 {(3,5), (1,5)},
 {(3,7), (5,5)},
 {(2,1), (4,3)},
 {(2,3), (1,3)},
 {(2,5), (0,3)},
 {(2,7), (5,3)},
 {(4,1), (1,7)},
 {(4,3), (2,1)},
 {(4,5), (3,1)},
 {(4,7), (0,1)},
 {(5,1), (0,7)},
 {(5,3), (2,7)},
 {(5,5), (3,7)},
 {(5,7), (1,1)},
};

 public Dictionary<(int, int), (int, int, int, int)>
CornerEdgeLookup = new Dictionary<(int, int), (int, int, int, int)>()
{
}
```

## Rubik's Cube Solver & Trainer

```
 {(0,0), (2,2,4,6)},
 {(0,2), (4,8,3,0)},
 {(0,6), (5,0,2,8)},
 {(0,8), (3,6,5,2)},
 {(1,0), (2,6,5,6)},
 {(1,2), (5,8,3,8)},
 {(1,6), (4,0,2,0)},
 {(1,8), (3,2,4,2)},
 {(2,0), (4,0,1,6)},
 {(2,2), (4,6,0,0)},
 {(2,6), (1,0,5,6)},
 {(2,8), (0,6,5,0)},
 {(3,0), (4,8,0,2)},
 {(3,2), (4,2,1,8)},
 {(3,6), (0,8,5,2)},
 {(3,8), (1,2,5,8)},
 {(4,0), (1,6,2,0)},
 {(4,2), (1,8,3,2)},
 {(4,6), (0,0,2,2)},
 {(4,8), (0,2,3,0)},
 {(5,0), (0,6,2,8)},
 {(5,2), (0,8,3,6)},
 {(5,6), (2,6,1,0)},
 {(5,8), (1,2,3,8)},
 };

 public (int, int) getAdjacentEdgeIndex(int currentFace, int
currentEdge)
 {
 (int face, int Edge) = AdjacentEdgeLookup[(currentFace,
currentEdge)];
 return (face, Edge);
 }

 public (int, int, int, int) getCornerEdgeIndexes(int
currentFace, int currentSticker)
 {
 (int face1, int edge1, int face2, int edge2) =
CornerEdgeLookup[(currentFace, currentSticker)];
 return (face1, edge1, face2, edge2);
 }

 private int Checkf2lTopPosition(Cube cube, int pairNumber)
 {
 int[] f2lFaceIndex = new int[4]
 {
 0, 3, 1, 2
 };
 int[] topFaceIndex = new int[4]
 {
 7, 5, 1, 3
 };
 char[,] edgeComposition = new char[4, 2]
```

## Rubik's Cube Solver & Trainer

```
{
 {'b', 'r'},
 {'r', 'g'},
 {'g', 'o'},
 {'o', 'b'},
};

for (int i = 0; i < 4; i++)
{
 if (f2lFaceIndex[i] == 1) // orange face case
 {
 if (cube.faces[f2lFaceIndex[i], 7].getColour() ==
edgeComposition[pairNumber, 0] || (cube.faces[f2lFaceIndex[i],
7].getColour() == edgeComposition[pairNumber, 1]))
 {
 if (cube.faces[4, 1].getColour() ==
edgeComposition[pairNumber, 0] || cube.faces[4, 1].getColour() ==
edgeComposition[pairNumber, 1])
 {
 return f2lFaceIndex[i];
 }
 }
 }
 if (cube.faces[f2lFaceIndex[i], 1].getColour() ==
edgeComposition[pairNumber, 0] || (cube.faces[f2lFaceIndex[i],
1].getColour() == edgeComposition[pairNumber, 1]))
 {
 if (cube.faces[4, topFaceIndex[i]].getColour() ==
edgeComposition[pairNumber, 0] || (cube.faces[4,
topFaceIndex[i]].getColour() == edgeComposition[pairNumber, 1]))
 {
 return f2lFaceIndex[i];
 }
 }
}
return -1;
}

private (int, int) Checkf2lEdgeLayer(Cube cube, int
pairNumber)
{
 char[,] edgeComposition = new char[4, 2]
 {
 {'b', 'r'},
 {'r', 'g'},
 {'g', 'o'},
 {'o', 'b'},
 };
 int[] f2lFaceIndex = new int[4]
 {
 0, 3, 1, 2
 };
 char colour1 = edgeComposition[pairNumber, 0];
 char colour2 = edgeComposition[pairNumber, 1];
 for (int i = 0; i < 4; i++)
```

## Rubik's Cube Solver & Trainer

```
 {
 if (cube.faces[f2lFaceIndex[i], 5].getColour() ==
colour1)
 {
 if (cube.faces[f2lFaceIndex[(i + 1) % 4],
3].getColour() == colour2)
 {
 return (f2lFaceIndex[i], f2lFaceIndex[(i + 1)
% 4]);
 }
 }
 if (cube.faces[f2lFaceIndex[i], 5].getColour() ==
colour2)
 {
 if (cube.faces[f2lFaceIndex[(i + 1) % 4],
3].getColour() == colour1)
 {
 return (f2lFaceIndex[i], f2lFaceIndex[(i + 1)
% 4]);
 }
 }
 }
 return (-1, -1);
 }
}

//OLL
public (int, bool) decodeOLLCharMove(char move)
{
 int decodedMove = -1;
 bool anticlockwise = false;
 if (char.IsUpper(move))
 {
 anticlockwise = false;
 switch (move)
 {
 case 'R':
 decodedMove = 3;
 break;
 case 'L':
 decodedMove = 2;
 break;
 case 'U':
 decodedMove = 4;
 break;
 case 'F':
 decodedMove = 0;
 break;
 case 'B':
 decodedMove = 1;
 break;
 case 'D':
 decodedMove = 5;
 break;
 case 'M':
```

# Rubik's Cube Solver & Trainer

```
 decodedMove = 12;
 break;
 }
}
else
{
 anticlockwise = true;
 switch (move)
 {
 case 'r':
 decodedMove = 3;
 break;
 case 'l':
 decodedMove = 2;
 break;
 case 'u':
 decodedMove = 4;
 break;
 case 'f':
 decodedMove = 0;
 break;
 case 'b':
 decodedMove = 1;
 break;
 case 'd':
 decodedMove = 5;
 break;
 case 'm':
 decodedMove = 12;
 break;
 }
}
return (decodedMove, anticlockwise);
}
public List<int> SolveOLL(Cube cube)
{
 List<int> solution;
 solution = OLLFirstLook(cube);
 return solution;
}
public List<int> OLLFirstLook(Cube cube)
{
 List<int> solution = new List<int>();
 int decodedMove;
 bool Anticlockwise;
 while (!YellowCross(cube))
 {
 List<int> stepSolution = new List<int>();
 if (YellowDiagonalPair(cube) != -1)
 {
 switch (YellowDiagonalPair(cube))
 {
```

## Rubik's Cube Solver & Trainer

```
 case 1:
 stepSolution.Add(10);
 break;
 case 3://correctly oriented yellow diagonal
 stepSolution.Add(0);
 stepSolution.Add(3);
 stepSolution.Add(4);
 stepSolution.Add(9);
 stepSolution.Add(10);
 stepSolution.Add(6);
 break;
 case 5:
 stepSolution.Add(4);
 stepSolution.Add(4);
 break;
 case 7:
 stepSolution.Add(4);
 break;
 }
}
else if (YellowLine(cube) != -1)
{
 if (YellowLine(cube) == 1)
 {
 stepSolution.Add(4);
 }
 stepSolution.Add(0);
 stepSolution.Add(3);
 stepSolution.Add(4);
 stepSolution.Add(9);
 stepSolution.Add(10);
 stepSolution.Add(6);
}
else // yellow dot
{
 stepSolution.Add(0);
 stepSolution.Add(3);
 stepSolution.Add(4);
 stepSolution.Add(9);
 stepSolution.Add(10);
 stepSolution.Add(6);
}
foreach (int move in stepSolution)
{
 solution.Add(move);
 if (move > 5)
 {
 decodedMove = move - 6;
 Anticlockwise = true;
 }
 else
 {
 decodedMove = move;
 Anticlockwise = false;
 }
}
```

## Rubik's Cube Solver & Trainer

```
 }
 cube.RotateFace(decodedMove, Anticlockwise);
 }

 }
 return solution;
}

private bool YellowCross(Cube cube)
{
 char crossPiece1 = cube.faces[4, 1].getColour();
 char crossPiece3 = cube.faces[4, 3].getColour();
 char crossPiece5 = cube.faces[4, 5].getColour();
 char crossPiece7 = cube.faces[4, 7].getColour();
 return (crossPiece1 == 'y' && crossPiece3 == 'y' &&
crossPiece5 == 'y' && crossPiece7 == 'y');
}

private int YellowDiagonalPair(Cube cube)
{
 bool crossPiece1 = cube.faces[4, 1].getColour() == 'y';
 bool crossPiece3 = cube.faces[4, 3].getColour() == 'y';
 bool crossPiece5 = cube.faces[4, 5].getColour() == 'y';
 bool crossPiece7 = cube.faces[4, 7].getColour() == 'y';
 if (crossPiece1 && crossPiece3)
 {
 return 3;
 }
 if (crossPiece1 && crossPiece5)
 {
 return 1;
 }
 if (crossPiece7 && crossPiece3)
 {
 return 7;
 }
 if (crossPiece7 && crossPiece5)
 {
 return 5;
 }

 return -1; // No diagonal pair
}

private int YellowLine(Cube cube)
{
 if (cube.faces[4, 3].getColour() == 'y' && cube.faces[4,
5].getColour() == 'y')
 {
 return 3;
 }
 if (cube.faces[4, 1].getColour() == 'y' && cube.faces[4,
7].getColour() == 'y')
 {

```

## Rubik's Cube Solver & Trainer

```
 return 1;
 }
 return -1;
}
public List<int> OLLSecondLook(Cube cube) //test commit
{
 List<int> solution = new List<int>();
 int yellowCornerCounter = countYellowCorners(cube);
 switch (yellowCornerCounter)
 {
 case 0: // Either H or Pi
 if (H(cube))
 {
 solution =
ConvertMovesToRotations(OLLAlgorithms[2]); // H
 if (cube.faces[0, 0].getColour() != 'y')
 {
 solution.Insert(0, 4); // top face needs
rotating
 }
 }
 else
 {
 solution =
ConvertMovesToRotations(OLLAlgorithms[3]); // pi
 if (cube.faces[0, 0].getColour() == 'y' &&
cube.faces[0, 2].getColour() == 'y')
 {
 solution.Insert(0, 4); // top face needs
rotating
 }
 else if (cube.faces[0, 0].getColour() == 'y')
 {
 solution.Insert(0, 4);
 solution.Insert(0, 4); // top face needs
rotating twice
 }
 else if (cube.faces[0, 2].getColour() != 'y')
 {
 solution.Insert(0, 10); // top face needs
rotating anticlockwise
 }
 }
 break;

 case 1: // Either sune or antisune
 if (Sune(cube))
 {
 solution =
ConvertMovesToRotations(OLLAlgorithms[0]); // sune
 if (cube.faces[4, 0].getColour() == 'y')
 {
 solution.Insert(0, 10); // top face needs
rotating anticlockwise
 }
 }
 }
 }
}
```

## Rubik's Cube Solver & Trainer

```

 }
 else if (cube.faces[4, 2].getColour() == 'y')
 {
 solution.Insert(0, 4);
 solution.Insert(0, 4); // top face needs
rotating twice
 }
 else if (cube.faces[4, 8].getColour() == 'y')
 {
 solution.Insert(0, 4); // top face needs
rotating
 }
}
else
{
 solution =
ConvertMovesToRotations(OLLAlgorithms[1]); // antisune
 if (cube.faces[4, 0].getColour() == 'y')
 {
 solution.Insert(0, 4);
 solution.Insert(0, 4); // top face needs
rotating twice
 }
 else if (cube.faces[4, 2].getColour() == 'y')
 {
 solution.Insert(0, 4); // top face needs
rotating
 }
 else if (cube.faces[4, 6].getColour() == 'y')
 {
 solution.Insert(0, 10); // top face needs
rotating anticlockwise
 }
}
break;
case 2: // Either bowtie, T or headlights
 if (Bowtie(cube)) // Bowtie
 {
 solution =
ConvertMovesToRotations(OLLAlgorithms[6]);
 if (cube.faces[3, 2].getColour() == 'y')
 {
 solution.Insert(0, 4);
 }
 else if (cube.faces[1, 6].getColour() == 'y')
 {
 solution.Insert(0, 4);
 solution.Insert(0, 4);
 }
 else if (cube.faces[2, 2].getColour() == 'y')
 {
 solution.Insert(0, 10);
 }
 }
}
```

## Rubik's Cube Solver & Trainer

```
Else
{
 if (Headlights(cube)) // Headlights
 {
 solution =
ConvertMovesToRotations(OLLAlgorithms[4]);
 if (cube.faces[3, 0].getColour() == 'y')
 {
 solution.Insert(0, 4); // Top layer
needs rotating
 }
 else if (cube.faces[2, 0].getColour() ==
'y')
 {
 solution.Insert(0, 10); // Top layer
needs rotating anticlockwise
 }
 else if (cube.faces[1, 6].getColour() ==
'y')
 {
 solution.Insert(0, 4);
 solution.Insert(0, 4); // Top layer
needs rotating twice
 }
 }
 else // T
 {
 solution =
ConvertMovesToRotations(OLLAlgorithms[5]);
 if (cube.faces[3, 0].getColour() == 'y')
 {
 solution.Insert(0, 4); // Top layer
needs rotating
 }
 else if (cube.faces[2, 0].getColour() ==
'y')
 {
 solution.Insert(0, 10); // Top layer
needs rotating anticlockwise
 }
 else if (cube.faces[1, 8].getColour() ==
'y')
 {
 solution.Insert(0, 4);
 solution.Insert(0, 4); // Top layer
needs rotating twice
 }
 }
}
break;
// case 3 is impossible to achieve assuming all
previous steps are correct
case 4:
```

## Rubik's Cube Solver & Trainer

```
 solution = new List<int>(); // OLL solved
 break;
 }
 return solution;
}

private int countYellowCorners(Cube cube)
{
 int[] cornerIndices = new int[4] { 0, 2, 6, 8 };
 int count = 0;
 foreach (int index in cornerIndices)
 {
 if (cube.faces[4, index].getColour() == 'y')
 {
 count++;
 }
 }
 return count;
}

private List<int> ConvertMovesToRotations(string moves)
{
 List<int> solution = new List<int>();
 int iMove;
 bool anticlockwise;
 foreach (char move in moves)
 {
 (iMove, anticlockwise) = decodeOLLCharMove(move);
 if (anticlockwise)
 {
 iMove += 6;
 }
 solution.Add(iMove);
 }
 return solution;
}

private bool H(Cube cube)
{
 // First H case
 char c1 = cube.faces[0, 0].getColour();
 char c2 = cube.faces[0, 2].getColour();
 char c3 = cube.faces[1, 6].getColour();
 char c4 = cube.faces[1, 8].getColour();

 // Second H case

 char c5 = cube.faces[2, 0].getColour();
 char c6 = cube.faces[2, 2].getColour();
 char c7 = cube.faces[3, 0].getColour();
 char c8 = cube.faces[3, 2].getColour();

 if (c1 == 'y' && c2 == 'y' && c3 == 'y' && c4 == 'y')
 {
 }
```

## Rubik's Cube Solver & Trainer

```
 return true;
 }
 else if (c5 == 'y' && c6 == 'y' && c7 == 'y' && c8 == 'y')
 {
 return true; // also requires u move
 }
 else
 {
 return false;
 }
}

private bool Sune(Cube cube)
{
 // Check if 2 stickers on index 2 are yellow,
 // if 1+ are yellow, sune
 // else (0) antisune

 if (cube.faces[0, 2].getColour() == 'y' && cube.faces[3,
2].getColour() == 'y')
 {
 return true; // sune
 }
 else
 {
 return false; // antisune
 }
}

private bool Bowtie(Cube cube)
{
 if (cube.faces[4, 0].getColour() == 'y' && cube.faces[4,
8].getColour() == 'y')
 {
 return true;
 }
 else if (cube.faces[4, 2].getColour() == 'y' &&
cube.faces[4, 6].getColour() == 'y')
 {
 return true;
 }
 else
 {
 return false;
 }
}

private bool Headlights(Cube cube)
{
 if (cube.faces[0, 0].getColour() == 'y' && cube.faces[0,
2].getColour() == 'y')
 {
 return true;
 }
}
```

## Rubik's Cube Solver & Trainer

```
 else if (cube.faces[3, 0].getColour() == 'y' &&
cube.faces[3, 2].getColour() == 'y')
 {
 return true;
 }
 else if (cube.faces[1, 6].getColour() == 'y' &&
cube.faces[1, 8].getColour() == 'y')
 {
 return true;
 }
 else if (cube.faces[2, 0].getColour() == 'y' &&
cube.faces[2, 2].getColour() == 'y')
 {
 return true;
 }
 else
 {
 return false;
 }
 }

 // PLL
 public (List<int>, int) PLLFirstLook(Cube cube)
 {
 List<int> solution = new List<int>();
 int numCorrect = GetAmountTopLayerCorrect(cube);
 int Case = -1;
 switch (numCorrect)
 {
 case 0:
 solution =
ConvertMovesToRotations(PLLAlgorithms[1]); // No Headlights
 Case = 0;
 break;
 case 1:
 solution =
ConvertMovesToRotations(PLLAlgorithms[0]); // 1 Headlight
 Case = 1;
 break;
 }
 return (solution, Case);
 }

 private int GetAmountTopLayerCorrect(Cube cube)
 {
 int numCorrect = 0;
 for(int i = 0; i < 4; i++)
 {
 if(CheckTopLayerCorrect(cube, i))
 {
 numCorrect++;
 }
 }
 }
```

## Rubik's Cube Solver & Trainer

```
 return numCorrect;
 }
 private bool CheckTopLaterCorrect(Cube cube, int face)
 {
 switch (face)
 {
 case 0:
 return cube.faces[0, 0].getColour() ==
cube.faces[0, 2].getColour();
 case 1:
 return cube.faces[3, 0].getColour() ==
cube.faces[3, 2].getColour();
 case 2:
 return cube.faces[1, 6].getColour() ==
cube.faces[1, 8].getColour();
 case 3:
 return cube.faces[2, 0].getColour() ==
cube.faces[2, 2].getColour();
 }
 return false;
 }

 public int GetTopLayerHeadlights(Cube cube)
 {
 int[] faceIndices = new int[4] { 0, 3, 1, 2 };
 foreach (int faceIndex in faceIndices)
 {
 if (faceIndex == 1)
 {
 if (cube.faces[faceIndex, 6].getColour() ==
cube.faces[faceIndex, 8].getColour())
 {
 return faceIndex;
 }
 }
 else if (cube.faces[faceIndex, 0].getColour() ==
cube.faces[faceIndex, 2].getColour())
 {
 return faceIndex;
 }
 }
 return -1;
 }

 public List<int> PLLSecondLook(Cube cube)
 {
 List<int> solution = new List<int>();
 if (uPerm(cube) == 1) // uPerm
 {
 List<int> orientSolution;
 orientSolution = orientBackEdge(cube);
 foreach (int move in orientSolution)
 {
 bool Anticlockwise;
 int decodedMove;
```

## Rubik's Cube Solver & Trainer

```
 if (move > 5)
 {
 decodedMove = move - 6;
 Anticlockwise = true;
 }
 else
 {
 decodedMove = move;
 Anticlockwise = false;
 }
 cube.RotateFace(decodedMove, Anticlockwise);
 }
 if (cube.faces[0,1].getColour() == cube.faces[3,
0].getColour()) // Ua Perm
 {
 solution =
ConvertMovesToRotations(PLLAlgorithms[2]);
 }
 else // Ub Perm
 {
 solution =
ConvertMovesToRotations(PLLAlgorithms[3]);
 }
 orientSolution.Reverse();
 foreach(int move in orientSolution)
 {
 solution.Insert(0, move);
 }
}
else if (uPerm(cube) == 0) // not uPerm
{
 if (cube.faces[0, 0].getColour() == cube.faces[2,
1].getColour()) // Z perm
 {
 solution =
ConvertMovesToRotations(PLLAlgorithms[4]);
 }
 else if (cube.faces[0, 1].getColour() == cube.faces[3,
0].getColour()) // Z perm incorrectly oriented
 {
 solution =
ConvertMovesToRotations(PLLAlgorithms[4]);
 solution.Insert(0, 4);
 }
 else // H perm
 {
 solution =
ConvertMovesToRotations(PLLAlgorithms[5]);
 }
}
// solved
return solution;
}
private int uPerm(Cube cube)
```

## Rubik's Cube Solver & Trainer

```
{
 int correctLayers = 0;
 if (cube.faces[0, 0].getColour() == cube.faces[0,
1].getColour() && cube.faces[0, 1].getColour() == cube.faces[0,
2].getColour())
 {
 correctLayers++;
 }
 if (cube.faces[3, 0].getColour() == cube.faces[3,
1].getColour() && cube.faces[3, 1].getColour() == cube.faces[3,
2].getColour())
 {
 correctLayers++;
 }
 if (cube.faces[1, 6].getColour() == cube.faces[1,
7].getColour() && cube.faces[1, 7].getColour() == cube.faces[1,
8].getColour())
 {
 correctLayers++;
 }
 if (cube.faces[2, 0].getColour() == cube.faces[2,
1].getColour() && cube.faces[2, 1].getColour() == cube.faces[2,
2].getColour())
 {
 correctLayers++;
 }
 return correctLayers;
}

private List<int> orientBackEdge(Cube cube)
{
 List<int> solution = new List<int>();
 if (cube.faces[3, 0].getColour() == cube.faces[3,
1].getColour() && cube.faces[3, 1].getColour() == cube.faces[3,
2].getColour())
 {
 solution.Add(10);
 }
 if (cube.faces[0, 0].getColour() == cube.faces[0,
1].getColour() && cube.faces[0, 1].getColour() == cube.faces[0,
2].getColour())
 {
 solution.Add(4);
 solution.Add(4);
 }
 if (cube.faces[2, 0].getColour() == cube.faces[2,
1].getColour() && cube.faces[2, 1].getColour() == cube.faces[2,
2].getColour())
 {
 solution.Add(4);
 }
 return solution;
}

public bool WhiteCrossSolved(Cube cube)
```

## Rubik's Cube Solver & Trainer

```
{
 int whiteFaceIndex = -1;
 for (int i = 0; i < 6; i++)
 {
 if (cube.faces[i,4].getColour() == 'w')
 {
 whiteFaceIndex = i;
 break;
 }
 }
 if ((cube.faces[whiteFaceIndex,1].getColour() == 'w') &&
(cube.faces[whiteFaceIndex, 3].getColour() == 'w') &&
(cube.faces[whiteFaceIndex, 5].getColour() == 'w') &&
(cube.faces[whiteFaceIndex, 7].getColour() == 'w'))
 {
 return true;
 }
 else
 {
 return false;
 }
}
public bool F2LSolved(Cube cube)
{
 if (WhiteCrossSolved(cube))
 {
 Cube clone = cube.Clone(cube);
 if (clone.faces[2, 4].getColour() == 'y' ||
clone.faces[3, 4].getColour() == 'y') //Orient cube correctly
 {
 clone.rotateDisplayLeft();
 }
 while (clone.faces[4, 4].getColour() != 'y')
 {
 clone.rotateDisplayDown();
 }
 while (clone.faces[0, 4].getColour() !=
clone.edgeComposition[0, 0])
 {
 clone.rotateDisplayLeft();
 }
 for (int i = 0; i < 4; i++)
 {
 if (!clone.F2LPairSolved(i))
 {
 return false;
 }
 clone.rotateDisplayLeft();
 }
 return true;
 }
 else
 {
 return false;
 }
}
```

## Rubik's Cube Solver & Trainer

```
 }
}
public bool Oll1Solved(Cube cube)
{
 if (WhiteCrossSolved(cube) && F2LSolved(cube))
 {
 int yellowFaceIndex = -1;
 for (int i = 0; i < 6; i++)
 {
 if (cube.faces[i, 4].getColour() == 'y')
 {
 yellowFaceIndex = i;
 break;
 }
 }
 if ((cube.faces[yellowFaceIndex, 1].getColour() ==
'y') && (cube.faces[yellowFaceIndex, 3].getColour() == 'y') &&
(cube.faces[yellowFaceIndex, 5].getColour() == 'y') &&
(cube.faces[yellowFaceIndex, 7].getColour() == 'y'))
 {
 return true;
 }
 else
 {
 return false;
 }
 }
 else
 {
 return false;
 }
}
public bool Oll2Solved(Cube cube)
{
 if (WhiteCrossSolved(cube) && F2LSolved(cube) &&
Oll1Solved(cube))
 {
 int yellowFaceIndex = -1;
 for (int i = 0; i < 6; i++)
 {
 if (cube.faces[i, 4].getColour() == 'y')
 {
 yellowFaceIndex = i;
 break;
 }
 }

 for (int i = 0; i < 9; i++)
 {
 if (cube.faces[yellowFaceIndex, i].getColour() !=
'y')
 {
 return false;
 }
 }
 }
}
```

# Rubik's Cube Solver & Trainer

```
 }
 return true;
 }
 else
 {
 return false;
 }
}
public bool Pll1Solved(Cube cube)
{
 if(WhiteCrossSolved(cube) && F2LSolved(cube) &&
 Oll1Solved(cube) && Oll2Solved(cube))
 {
 if(PLLFirstLook(cube).Item2 == -1)
 {
 return true;
 }
 }
 return false;
}
}
```

## Testing

<https://youtu.be/Ac8AzeleAnw> Main Testing Video

[https://youtu.be/zXM\\_dtFisdK](https://youtu.be/zXM_dtFisdK) Real Cube Testing Video

| Test                                              | Test NO. | Objective | Input Data                              | Expected Result                                                                                         | Outcome     | Passed ? | Evidence |
|---------------------------------------------------|----------|-----------|-----------------------------------------|---------------------------------------------------------------------------------------------------------|-------------|----------|----------|
| To check if database will be successfully created | 1        | 1a.i      | Start program with no existing database | An output to the user stating the database has been created<br>A database has been created in the files | As expected | Yes      | 0:04     |

## Rubik's Cube Solver & Trainer

|                                                                                  |   |        |                                         |                                                                                                                              |             |     |      |
|----------------------------------------------------------------------------------|---|--------|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------------|-------------|-----|------|
| To check database is not overwritten                                             | 2 | 1 a.ii | Start program with an existing database | An output to the user stating the database already exists<br>The existing database exists and hasn't been modified/recreated | As expected | Yes | 0:25 |
| To check if a non-existing user is prompted to create an account                 | 3 | 2b     | User12                                  | The user will be prompted to create an account                                                                               | As expected | Yes | 0:45 |
| To check if a Log in with a username less than 4 characters is correctly handled | 4 | 2e     | Use                                     | Output stating that the username or password is too short                                                                    | As expected | Yes | 1:36 |
| To check if a Log in with a username with 4 characters is correctly handled      | 5 | 2e     | User                                    | Account creation prompt                                                                                                      | As expected | Yes | 2:01 |
| To check if a Log in with a password less than 6 characters is correctly handled | 6 | 2f     | Pass                                    | Output stating that the username or password is too short                                                                    | As expected | Yes | 2:26 |

## Rubik's Cube Solver & Trainer

|                                                                               |    |      |                 |                                                                                |                                                                        |     |      |
|-------------------------------------------------------------------------------|----|------|-----------------|--------------------------------------------------------------------------------|------------------------------------------------------------------------|-----|------|
| To check if a Log in with a password with 6 characters is correctly handled   | 7  | 2f   | Passwo          | Account creation prompt                                                        | As expected                                                            | Yes | 2:42 |
| To check if a user can create an account                                      | 8  | 2b.i | User12 Password | Account successfully created and details stored in Users table in the database | As expected , user is also told incorrect username/password afterwards | No  | 2:57 |
| To check if a user can log in to an existing account                          | 9  | 2d   | User12 Password | Program launches into main menu                                                | As expected                                                            | Yes | 4:37 |
| To check if a log in with the wrong password is denied access                 | 10 | 2c   | User12 Passcode | Output stating username or password is incorrect                               | As expected                                                            | Yes | 5:12 |
| To check if a Log in with a differently capitalised password is denied access | 11 | 2c   | User12 password | Output stating username or password is incorrect                               | As expected                                                            | Yes | 5:35 |

## Rubik's Cube Solver & Trainer

|                                                                                                 |    |        |                       |                                                                        |             |     |      |
|-------------------------------------------------------------------------------------------------|----|--------|-----------------------|------------------------------------------------------------------------|-------------|-----|------|
| To check if two different users with the same password have different salts and password hashes | 12 |        | User13 Password       | The users have different salts and hashed passwords in the Users table | As expected | Yes | 5:50 |
| Entering Training Mode functionality                                                            | 13 | 3a.ii  | Training button click | The user is taken to the training mode                                 | As expected | Yes | 6:22 |
| Entering Solving Mode functionality                                                             | 14 | 3a.iii | Solve button click    | The user is taken to the solving mode                                  | As expected | Yes | 6:48 |
| Return from secondary mode back to playground                                                   | 15 | 3c     | Back button click     | The user is returned to the playground from the previous mode          | As expected | Yes | 6:38 |
| Check if rotating the whole cube works as expected                                              | 16 | 5a.i   | Rotate Up             | Cube display rotates 90 Degrees up                                     | As expected | Yes | 7:01 |
| Check if rotating the whole cube works as expected                                              | 17 | 5a.ii  | Rotate Down           | Cube display rotates 90 Degrees down                                   | As expected | Yes | 7:12 |

## Rubik's Cube Solver & Trainer

|                                                                                             |    |        |              |                                                     |             |     |      |
|---------------------------------------------------------------------------------------------|----|--------|--------------|-----------------------------------------------------|-------------|-----|------|
| Check if rotating the whole cube works as expected                                          | 18 | 5a.iii | Rotate Left  | Cube display rotates 90 Degrees left                | As expected | Yes | 7:24 |
| Check if rotating the whole cube works as expected                                          | 19 | 5a.iv  | Rotate Right | Cube display rotates 90 Degrees right               | As expected | Yes | 7:35 |
| Check if face rotation affects the correct face and rotates in the correct direction        | 20 | 6b.i   | R            | The Rightmost face rotates 90 degrees clockwise     | As expected | Yes | 7:45 |
| Check if prime face rotation affects the correct face and rotates in the opposite direction | 21 | 6b.i   | R'           | The Rightmost face rotates 90 degrees anticlockwise | As expected | Yes | 7:45 |
| Check if face rotation affects the correct face and rotates in the correct direction        | 22 | 6b.ii  | L            | The Leftmost face rotates 90 degrees clockwise      | As expected | Yes | 7:54 |
| Check if prime face rotation                                                                | 23 | 6b.ii  | L'           | The Leftmost face rotates 90 degrees                | As expected | Yes | 7:54 |

## Rubik's Cube Solver & Trainer

|                                                                                             |    |        |    |                                                  |             |     |      |
|---------------------------------------------------------------------------------------------|----|--------|----|--------------------------------------------------|-------------|-----|------|
| affects the correct face and rotates in the opposite direction                              |    |        |    | anticlockwise                                    |             |     |      |
| Check if face rotation affects the correct face and rotates in the correct direction        | 24 | 6b.iii | U  | The Top face rotates 90 degrees clockwise        | As expected | Yes | 8:04 |
| Check if prime face rotation affects the correct face and rotates in the opposite direction | 25 | 6b.iii | U' | The Top face rotates 90 degrees anticlockwise    | As expected | Yes | 8:04 |
| Check if face rotation affects the correct face and rotates in the correct direction        | 26 | 6b.iv  | D  | The Bottom face rotates 90 degrees clockwise     | As expected | Yes | 8:10 |
| Check if prime face rotation affects the correct face and rotates in the opposite           | 27 | 6b.iv  | D' | The Bottom face rotates 90 degrees anticlockwise | As expected | Yes | 8:10 |

## Rubik's Cube Solver & Trainer

|                                                                                             |    |                       |    |                                                 |             |     |      |
|---------------------------------------------------------------------------------------------|----|-----------------------|----|-------------------------------------------------|-------------|-----|------|
| direction                                                                                   |    |                       |    |                                                 |             |     |      |
| Check if face rotation affects the correct face and rotates in the correct direction        | 28 | 6b.v                  | F  | The Front face rotates 90 degrees clockwise     | As expected | Yes | 8:18 |
| Check if prime face rotation affects the correct face and rotates in the opposite direction | 29 | 6b.v                  | F' | The Front face rotates 90 degrees anticlockwise | As expected | Yes | 8:18 |
| Check if face rotation affects the correct face and rotates in the correct direction        | 30 | <a href="#">6b.vi</a> | B  | The Back face rotates 90 degrees clockwise      | As expected | Yes | 8:24 |
| Check if prime face rotation affects the correct face and rotates in the opposite direction | 31 | <a href="#">6b.vi</a> | B' | The Back face rotates 90 degrees anticlockwise  | As expected | Yes | 8:24 |
| Check if face rotation affects the                                                          | 32 | 6c                    | M  | The Middle Slice rotates 90 degrees clockwise   | As expected | Yes | 8:30 |

## Rubik's Cube Solver & Trainer

|                                                                                             |    |         |                                   |                                                                                  |                            |     |      |
|---------------------------------------------------------------------------------------------|----|---------|-----------------------------------|----------------------------------------------------------------------------------|----------------------------|-----|------|
| correct face and rotates in the correct direction                                           |    |         |                                   |                                                                                  |                            |     |      |
| Check if prime face rotation affects the correct face and rotates in the opposite direction | 33 | 6c      | M'                                | The Middle Slice rotates 90 degrees anticlockwise                                | As expected                | Yes | 8:30 |
| Check if the scramble generates 20 moves                                                    | 34 | 7       | Scramble Button Click             | 20 moves are generated and output to the text box                                | 21 moves generated, not 20 | No  | 8:40 |
| Check consecutive moves do not undo each other                                              | 35 | 7.ii    | Scramble Button Click             | The moves generated do not contain any consecutive clockwise anticlockwise pairs | As expected                | Yes | 9:10 |
| Check if user cannot start timer on a solved cube                                           | 36 | 8d      | Timer start with a solved cube    | The timer should not start and an output stating the cube is already solved      | As expected                | Yes | 9:25 |
| Check if timer starts and updates display                                                   | 37 | 8a + 8c | Timer start with an unsolved cube | The timer should start and the display should show the current time              | As expected                | Yes | 9:36 |
| Check if timer automatically stops upon cube                                                | 38 | 8b      | Solve cube with a timer started   | The timer should stop when the cube is solved<br>The time should be saved along  | As expected                | Yes | 9:36 |

## Rubik's Cube Solver & Trainer

|                                                                                                |    |          |                                                                                                                    |                                                                      |             |     |       |
|------------------------------------------------------------------------------------------------|----|----------|--------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|-------------|-----|-------|
| solve                                                                                          |    |          |                                                                                                                    | with the current user into the UserTimes table                       |             |     |       |
| Test whether loading a cube without a saved cube displays an error                             | 39 | 9d.i     | Load cube - No saved cube in files                                                                                 | Error displayed stating that user has no saved cubes                 | As expected | Yes | 10:08 |
| Test whether saving and loading a cube works                                                   | 40 | 9ci + 9d | Save a scrambled cube, then regenerate the cube, then attempt to load the saved cube                               | The previously scrambled cube returns in the same state it was saved | As expected | Yes | 10:18 |
| Check that loading a cube from a different user that has a saved cube does not open their cube | 41 | 9c       | Save cube and switch to an account without a saved cube, then attempt to load a cube                               | Error displayed stating that user has no saved cubes                 | As expected | Yes | 10:41 |
| Check that different users can save individual cubes                                           | 42 | 9a       | Save cube on 2 different accounts. Then attempt to reload each of the different cubes on their respective accounts | The cubes save/load individually for each account                    | As expected | Yes | 11:08 |

## Rubik's Cube Solver & Trainer

|                                                                     |    |         |                                                     |                                                                         |             |     |                       |
|---------------------------------------------------------------------|----|---------|-----------------------------------------------------|-------------------------------------------------------------------------|-------------|-----|-----------------------|
| Check middle stickers are unable to be recoloured                   | 43 | 11a.i   | attempt to recolour the middle sticker on each face | No colours should change                                                | As expected | Yes | 11:42                 |
| Check all 6 colours can be cycled through                           | 44 | 11b     | click the colour selector 6 times                   | Each of the 6 colours appears once and loops back after they all appear | As expected | Yes | 12:12                 |
| Check all other stickers are able to be recoloured                  | 45 | 11c.ii  | change the colour of every sticker                  | every sticker is recoloured                                             | As expected | Yes | 12:12                 |
| Check both the cube and the net reflect the updated colour          | 46 | 11c.iii | Change the colour of stickers                       | Both the cube and net should reflect the new cube states                | As expected | Yes | 12:58                 |
| Check scrambling the cube updates the net colours                   | 47 |         | Scramble Button Click                               | The tile grid is updated to reflect the scramble                        | As expected | Yes | 13:16                 |
| Check loading the cube updates the net colours                      | 48 |         | load cube                                           | The tile grid is updated to reflect the new cube                        | As expected | Yes | 13:40                 |
| Check inputting an invalid cube does not allow solver to be entered | 49 | 11e     | Input invalid cube<br>Enter Solver                  | The solver does not open upon button press and an error output.         | As expected | Yes | See Screen shot Below |

## Rubik's Cube Solver & Trainer

|                                                                      |    |     |                                                                       |                                                                                                                             |                                                                                |     |       |
|----------------------------------------------------------------------|----|-----|-----------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|-----|-------|
| Check all algorithms for each category are loaded into the dropdowns | 50 | 12b |                                                                       | All algorithms appear in the dropdowns                                                                                      | As expected                                                                    | Yes | 13:56 |
| Test algorithm loading                                               | 51 | 12c | Select an algorithm from any dropdown                                 | A new cube that requires the selected algorithm is displayed<br>The correct moves for the algorithm are displayed           | As expected                                                                    | Yes | 14:45 |
| Test algorithm playback                                              | 52 | 12d | Select an algorithm, change between play and paused                   | Algorithm is played back in full while play is enabled                                                                      | As expected                                                                    | Yes | 15:05 |
| Test changing algorithms during playback                             | 53 | 12c | Have an algorithm running playback, attempt to select a new algorithm | New algorithm should be displayed and not executing playback                                                                | Playback continues when switching algorithm during previous algorithm playback | No  | 16:06 |
| Test adding a custom algorithm                                       | 54 | 12e | R U R' U'                                                             | Output stating algorithm was successfully added<br>Database has new entry with the new algorithm<br>Dropdown updates to add | As expected                                                                    | Yes | 17:18 |

## Rubik's Cube Solver & Trainer

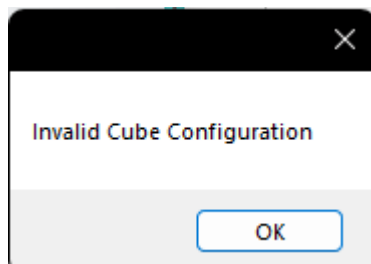
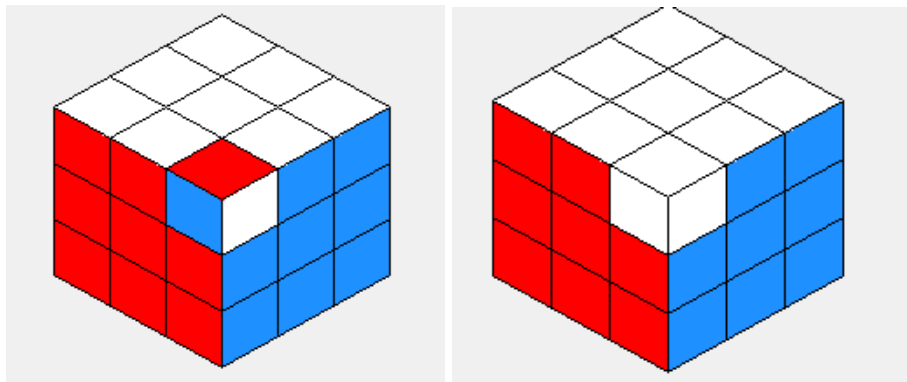
|                                                                     |    |             |                                                      |                                                                                                |             |     |       |
|---------------------------------------------------------------------|----|-------------|------------------------------------------------------|------------------------------------------------------------------------------------------------|-------------|-----|-------|
|                                                                     |    |             |                                                      | new algorithm                                                                                  |             |     |       |
| Test adding an algorithm with at least 3 moves is correctly handled | 55 | 12e.i       | R U<br>R U R                                         | First set should be rejected and an error displayed. Second set should continue as normal      | As expected | Yes | 18:07 |
| Test adding an algorithm requires a name                            | 56 | 12e.i       | Dont input a name for the algorithm when requested   | The algorithm is not added until the name is entered                                           | As expected | Yes | 18:37 |
| Test playback of custom algorithm                                   | 57 | 12e.iii     |                                                      | Program outputs R U R' U' into text box and performs those 4 moves, resulting in a solved cube | As expected | Yes | 18:49 |
| Check custom algorithm persists between application instances       | 58 | 12f         | Restart the application                              | Custom algorithm still exists in the dropdown                                                  | As expected | Yes | 19:00 |
| Test white cross                                                    | 59 | 13b.i + 14  | Scramble cube<br>Enter Solver<br>Attempt white cross | White cross is solved, Correct moves are output to the text box                                | As expected | Yes | 19:26 |
| Test F2l                                                            | 60 | 13b.ii + 15 | Attempt f2l                                          | The first 2 layers of the cube are solved with the correct moves output to the text box        | As expected | Yes | 19:45 |

## Rubik's Cube Solver & Trainer

|                                                                                    |    |                 |                                                                                           |                                                                                       |                |     |                                                                         |
|------------------------------------------------------------------------------------|----|-----------------|-------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|----------------|-----|-------------------------------------------------------------------------|
| Test OLL<br>1st Look                                                               | 61 | 13b.iii<br>+ 16 | Attempt<br>OLL 1st<br>Look                                                                | Yellow cross is<br>solved with the<br>correct moves<br>output to the text<br>box      | As<br>expected | Yes | 19:55                                                                   |
| Test OLL<br>2nd Look                                                               | 62 | 13b.iv<br>+ 16  | Attempt<br>OLL 2nd<br>Look                                                                | Yellow face is<br>solved with the<br>correct moves<br>output to the text<br>box       | As<br>expected | Yes | 20:00                                                                   |
| Test PLL<br>1st Look                                                               | 63 | 13b.v +<br>16   | Attempt PLL<br>1st Look                                                                   | Yellow corners<br>are solved with<br>the correct<br>moves output to<br>the text box   | As<br>expected | Yes | 20:08                                                                   |
| Test PLL<br>2nd Look                                                               | 64 | 13b.vi<br>+ 16  | Attempt PLL<br>2nd Look                                                                   | Cube is fully<br>solved with the<br>correct moves<br>output to the text<br>box        | As<br>expected | Yes | 20:08                                                                   |
| Check<br>stages<br>cannot be<br>executed<br>until<br>previous<br>stage<br>complete | 65 | 13c             | attempt<br>each stage<br>when the<br>previous<br>stage/s<br>have not<br>been<br>completed | A stage is not<br>able to begin<br>solving until all<br>previous stages<br>are solved | As<br>expected | Yes | 20:19                                                                   |
| Confirm<br>outputted<br>moves<br>work on a<br>real rubiks<br>cube                  | 66 | 13              | Scramble<br>cube -<br>execute<br>each stage<br>and perform<br>moves on a<br>real cube     | Cube is fully<br>solved                                                               | As<br>expected | Yes | <a href="https://youtu.be/zXM_dtFisdK">https://youtu.be/zXM_dtFisdK</a> |

# Rubik's Cube Solver & Trainer

Test 49:



## Re-Testing

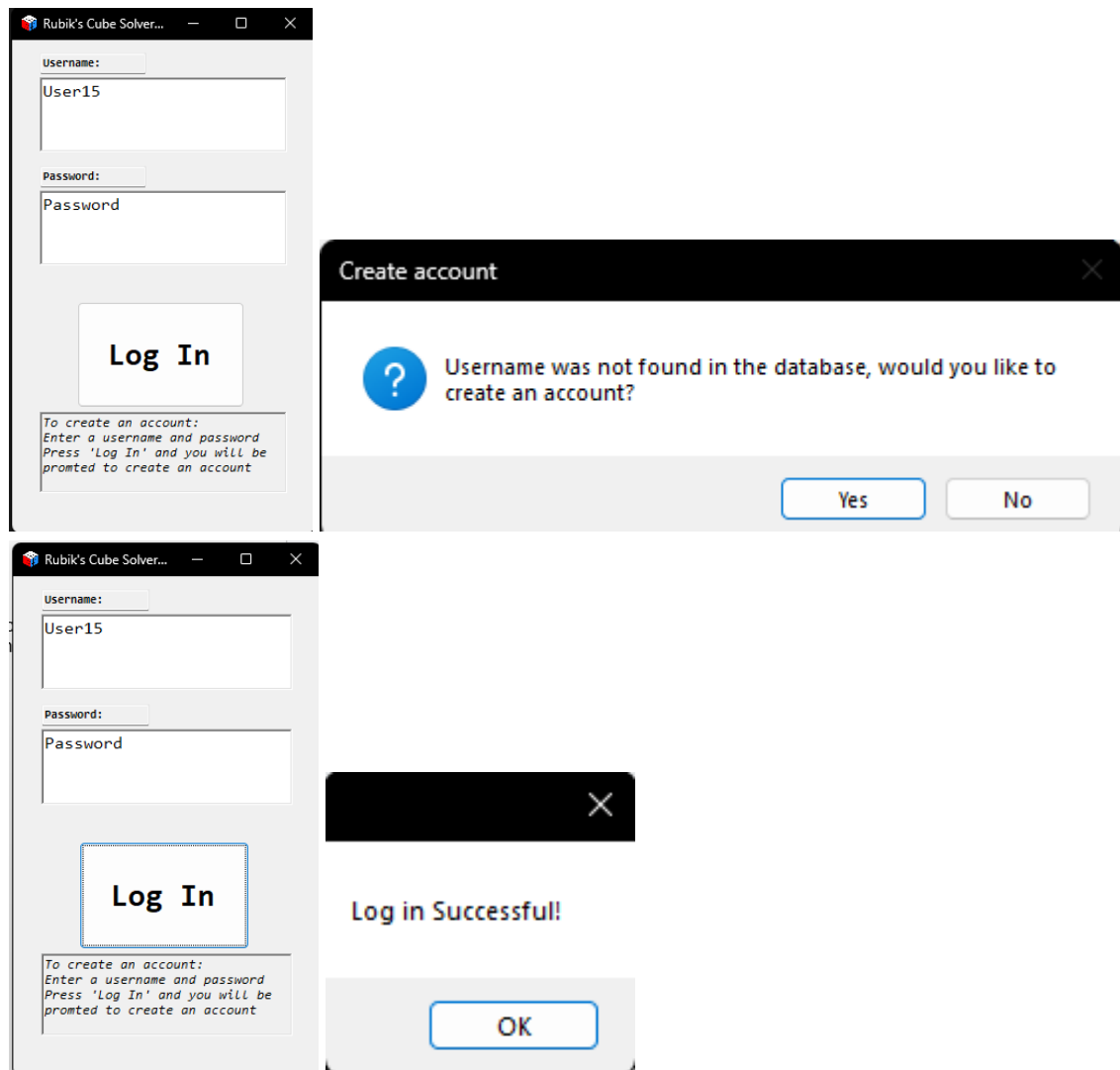
### Test 8:

Whilst the primary objective of creating an account was successfully accomplished, when the user creates the account, an additional popup appears stating that an incorrect username/password was entered. This popup should not appear here.

In order to fix this, when it is discovered that there is no stored username for the entered username and the user wishes to create an account, the error message will not be displayed.

# Rubik's Cube Solver & Trainer

Evidence of fix:



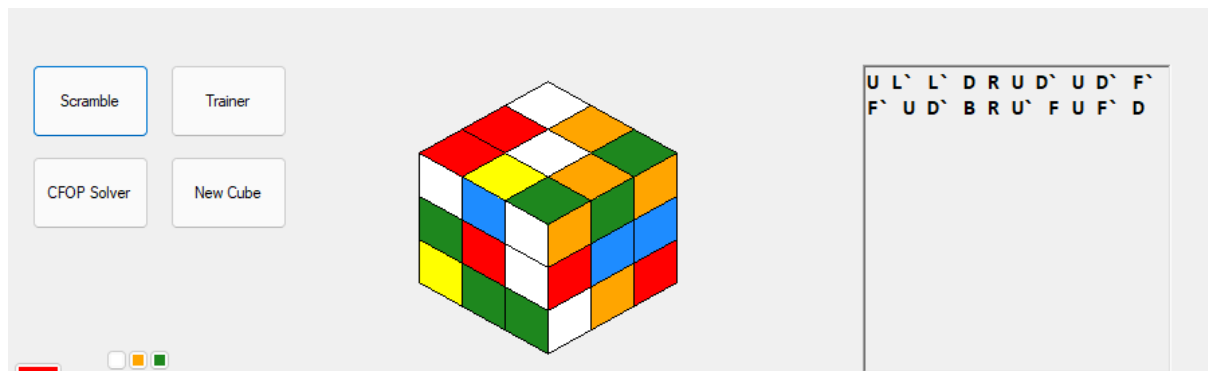
## Test 34:

The random move generator created 21 random moves, but the FCA declares that it must be 20 for an official scramble.

To fix this I have changed the for loop to iterate 19 times rather than 20 since a move is generated before entering the for loop.

# Rubik's Cube Solver & Trainer

Evidence of fix:

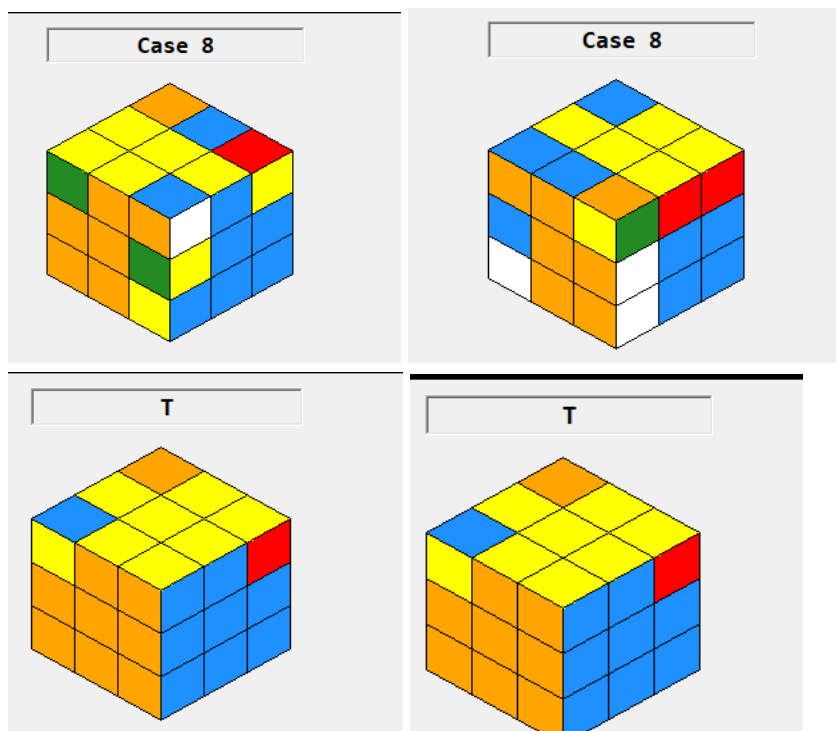


## Test 53:

When changing algorithms whilst an algorithm is already being displayed results in the new algorithm being played back immediately. As well as this, any new algorithm loaded results in autoplay too, and seemingly at an accelerated speed.

To fix this problem, a check has been implemented that sees whether a timer object exists and if it does it is disposed of and recreated.

Evidence of fix:



# Evaluation

## Objectives

1.a. Database created if it does not exist. This was met by a check that determines if the database exists, running upon application startup.

1.b. Database created using DDL commands. This objective has been met since the database can only be created using preconstructed DDL commands that have been placed inside the program. The DDL script executes if the existing database check returns that the database does not exist.

1.c. Interlinked tables. This has been achieved since the users database holds a unique user id for each user which is then used as a foreign key to index the user times table.

2a.i. Password encrypted using SHA-256. This was achieved using A custom implementation of the SHA-256 algorithm, used to encrypt a users inputted password which is then stored in a table.

2a.ii. Unique salt for each user. This was completed by generating a random 16 character string when the create account button is pressed. This is then combined with the inputted password to create a salted password

2.a. Table will store a username and encrypted password along with a unique salt. This is achieved by generating a unique salt, appending it to the entered password and then hashing using SHA-256. The username and Encrypted password and salt are all stored as a record in a table.

2.b. If the input username does not exist in the database, request account creation. This was achieved by having a query check if the input username is a record in the database when the login button is pressed. If not, an output box is displayed asking if the user would like to create an account with the input data.

2.c. Incorrect password results in error being displayed. This was met by hashing the input password, with the salt queried from the database, and checking whether that matches the encrypted password in the database. If they do not match, a message box is displayed stating that an incorrect password was entered.

2.d. Upon successful login, the user is taken to the main menu. This was achieved by rescaling the form to the size used for the rest of the program and hiding all the login components. Then the main items are made visible, making it feel like the scene has changed from a login to a program.

2.e. Usernames should be at least 4 characters. This was achieved by having a check to determine if the number of characters in the username text box is less than 4 characters.

2.f. Passwords should be at least 6 characters. This was achieved by having a check to determine if the number of characters in the password text box is less than 6 characters.

3.a. The main menu will have 4 buttons: scramble, trainer, solver, new cube. This was achieved by creating 4 buttons with the respective text.

# Rubik's Cube Solver & Trainer

3.b. Upon selecting a choice, the user will be taken to that section of the program. This has been achieved since the trainer and solver buttons take the user to a new menu with their respective buttons. Scramble and New Cube affect the cube on the main menu.

3.c. A back button will appear when inside a menu option. This has been achieved by having a back button exist on each menu option.

3.c.i. The back button will bring the user back to the main menu. This was met by making the back button rehide all the elements from the entered menu and making the main menu elements visible again.

4.a. The cube will be displayed to the user as a 2D projection. This was achieved using the drawing component of the system library, the lines are drawn onto a panel on the form. Along with this each tile is drawn in between the grid created previously.

4.b. + 4.c. The projection will be an isometric view of the cube. This was achieved by using trigonometry to find the equations of lines to create a grid of lines that create the illusion of a cube on the 2D screen.

5.a. There will be 4 buttons to control the cubes orientation. This was met by creating 4 buttons that have the respective text as labels.

5.b. Pressing a rotation button will have the effect of rotating the whole cube 90 degrees in that direction. This was achieved by rotating the 3 layers of the cube that are affected by the rotation. I.e. rotating right would be the same as a top face, middle-horizontal slice, and bottom face rotating right.

6.a. The faces will be addressed using rubik's cube notation, consisting of 6 letters for a clockwise rotation and the same letters followed by an apostrophe for anticlockwise rotation. This was achieved by creating a button for each move and their prime counterparts.

6.b. Rotating a face will be a rotation of 90 degrees in the respective direction on that face. This was achieved by creating two sets of 2D arrays, one storing an array of adjacent faces for each face, and the other storing the indices of the tiles that would be for each face.

6.c. There will also be an M / M' move which rotates the middle slice of the cube. This was met by using the same logic created before but applied to this special case.

7.a. The scramble button will generate 20 moves. This was met by running a while loop that created random moves until 20 were made.

7.a.i. A move will be a primary face rotation or their prime counterpart. This was achieved by having the array of moves that could be included to only contain the primary moves.

7.a.ii. Two consecutive moves should not undo each other. This was achieved by checking if the previous move is the same move as the newly generated move but opposite in direction.

7.b. The moves will be automatically applied to the digital cube. This was achieved by having a foreach loop iterate over each move that was generated and applying it to the cube.

7.c. The moves will be outputted to a text box for the user to read This was met by converting each of the random moves into its string based counterpart and concatenating them to form one output string which was set to the text of the textbox.

# Rubik's Cube Solver & Trainer

8.a. A timer button will start a stopwatch to time the users solve speed. This was met by creating a button and a display that starts a stopwatch.

8.b. After solving the cube, the stopwatch will automatically stop. A check is called every update interval which checks if the cube is solved. If solved, the timer is stopped.

8.b.i. the time to solve will be recorded into a table in the database with the username. This was met by having an sql statement insert a new record with the current users username and the time taken when the timer is stopped.

8.c. The current time should be constantly displayed and updated regularly, This was met by having a timer run alongside the stopwatch that every 200 milliseconds calls for the interval function that updates the display with the current time.

8.d. The timer should not start if the cube is already solved. This was achieved by having a check when the button is pressed to see if the cube is solved. If it is solved already, an error is output using a message box.

9.a. The current position of all tiles will be stored into a text file. This was achieved by iterating over each face and getting the colour of each tile on that face and storing that as characters onto a text file. Each line represents a face.

9.b. A user should only be able to access their own saved cube. This was met by having the text file that is searched for be the users username which must be unique in order to create the account and so the user can only access their cube. Since it is their username, only 1 cube is storable per user which is as required by the objective

9.c. Loading the cube will overwrite the currently displayed cube with the users stored cube. This is done by iterating over each of the tiles on the current cube and setting their tile colour to be what is found in the text file.

9.c.i. If the user has no stored cube, an error will be displayed. This was met by checking if a file exists with the user's username. If no file is found, an error message is displayed using a message box.

10.a. + 10.b. Pressing the new cube button will generate a new cube in the solved state. This was achieved by destroying the previous cube object and instantiating a new one which already is constructed into the solved state.

11.a.+ 11.c. Every tile on the rubik's cube will be able to be recoloured. This was achieved by having a button grid where each button represents a tile on the cube. Clicking on a tile button would update that tiles colour to be the selected colour.

11.a.i. The centres of the face will not be able to be recoloured. This is achieved by not giving the centre tile buttons a click event resulting in no action when pressed.

11.b. The colour selector will be a button that cycles through the 6 colours upon clicking. This is achieved by having a button that displays one of the 6 colours. When clicked a counter is incremented. Numbers 0-5 represent one of the 6 colours. Once the index reaches 6, it undergoes modular division and is wrapped around to 0 to keep closure true.

11.c.iii. the cube and button grid will update to reflect the change in colour. This is met by calling for both the cube and tile grid to be redrawn which will reflect the new change.

# Rubik's Cube Solver & Trainer

11.d. The recolouring will only be available in the playground. This is achieved by only having the recolouring grid visible in the playground.

11.e. If the user enters an invalid cube configuration and attempts to enter the solver, they will not be able to. This has been achieved by implementing a custom set of functions that check the 3 rules of configuration.

12.a. Upon entering the training mode, 4 dropdowns will be displayed along with a play and pause button. This is achieved by creating 4 combo boxes and 2 buttons, all labeled with the respective text.

12.b. The user can select any algorithm from the CFOP steps and a custom section. This is achieved by querying every algorithm using their category name and using that as a datasource for each of the combo boxes

12.c.i. A new cube that requires the selected algorithm will be displayed. This is achieved by querying the selected algorithm for its moves. Reversing the order and rotation of each move and applying that to a solved cube. This is then displayed to the user.

12.c.ii. The moves are outputted to a textbox. This is achieved by querying the moves and concatenating them into a string that is set to the text of the textbox.

12.d. pressing the play button will sequentially do the moves of the algorithm and solve the case. This is achieved by having a 1 second timer that after each second calls for the next move to be completed. The move is applied to the cube and a current move index is incremented. Once this index reaches the number of moves. The timer is stopped. Pressing play again will not do anything due to a check.

12.d.ii. The current move will be highlighted in the textbox. This is achieved by making the current move bold in the text box.

12.e. There will be an add button that will get the user to input the moves of an algorithm and give it a name. This has been achieved by creating a button that displays a series of input boxes. Once information is entered, a new record will be created in the algorithms table with the respective information.

12.e.i. Algorithms must have at least 3 moves. This is achieved by having a check that sees how many matches of moves exist. If this number is less than 3, the input is requested again.

12.e.ii. All moves should be valid moves that can be performed by the program. This is achieved by checking the input characters against a move set array. If there is no match, the inputted move is invalid and the input is requested again.

12.e.iii. Any custom algorithms will appear under the custom dropdown. This is achieved by setting the 'custom' combo box datasource to the custom algorithms category.

12.f. All algorithms will be stored in a table in the database. This is achieved by a series of DDL statements that insert the predefined algorithms into the database. Any custom algorithms will be inserted by the user.

13.a. The solver will solve the current rubik's cube. This is met because the cube the solver attempts to solve is the same cube within the playground.

13.b. Each of the CFOP steps will be there own button to be pressed. This is achieved by creating a button with the respective text that does each of the solution stages.

# Rubik's Cube Solver & Trainer

13.c. The stage will not work until all previous steps have been completed. This is achieved by having a check implemented for each stage. For example, if the OLL 1st look is attempted, it will only run if the white cross check is true and F2L check is true.

13.d. Each algorithm will be stored in a database and queried when needed. When the solver finds which algorithm to use next, a query will get the algorithms moves using its algorithm ID.

13.e. After each stage, the moves required will be outputted to a text box. Once the stage is completed. All of the used moves will be concatenated together and set as the text for the text box.

14.a. The white cross will use a shortest path algorithm. This is achieved by using the IDA\* shortest path algorithm.

14.b The algorithm will be a heuristic algorithm in order to optimise for speed rather than move count. This is achieved by using the IDA\* algorithm which uses a heuristic function to guess roughly how many moves it should take to solve the cross.

14.c. An output should be shown to the user when selecting the white cross stage so that they know the algorithm is processing. This is achieved using a progress bar that is displayed during the runtime of the shortest path algorithm.

15.a. The F2L solver will be ran for each of the 4 pairs. This is met by using a for loop that iterates 4 times.

15.b.i. If the edge piece is not in the correct column, the edge piece will be placed into the top layer above the correct middle layer. This is achieved by finding the current position of the edge piece, if it is in the correct column - nothing to be moved, if it is in the top layer, it is rotated until the front piece is the same colour as the middle piece. If it is in a different column, a right move algorithm is performed and then it is rotated until the front sticker matches the middle piece.15.

15.b.ii. + 15.b.ii. The corner piece will be placed into the top face if it is not already. This is achieved by finding the current position of the corner piece, if it is in the correct column - nothing happens. If it is in the top layer, it is rotated to the correct column. If it is in a different column, a right move algorithm is performed and then the corner is rotated to the correct column.

15.c. It will iterate over each of the F2L algorithms, queried from a table, until it finds the correct one. This is done by first storing the queried algorithms' moves into an array. Then a foreach loop iterates over each of the algorithms.

15.d. Once the correct algorithm has been located, it will be added to a list and the next pair will begin searching. This is achieved by checking whether the F2L pair is solved and if so, the algorithm that was just applied is passed out off the loop and the next pair is then searched for.

15.e. The output string will consist of 4 sets of moves, separated by a newline. This is met by having the moves for each pair output with the newline control character.

16.a. For each of the OLL and PLL stages, detection logic will select which algorithm to use. This is achieved by having functions that check specific indexes on each face to determine what algorithm is to be applied.

# Rubik's Cube Solver & Trainer

16.b After each algorithm, that move set will be outputted to the text box. This is achieved by concatenating the algorithm moves together to form a string which is then set to the output text box text.

## End-User Feedback

### **Does the project meet your expectations?**

All my requirements for the application have been successfully implemented. I am more than happy with how different components act together within the system.

### **What do you like about the project?**

I am happy with how clear the application is to interact with; it is clear what each button press is going to do. I like how the solver gets the user to click on each stage of the CFOP method as it more clearly demonstrates how to solve a cube using the whole method. The trainer is nice and works smoothly; the playback feature is a very useful addition that makes it even easier to see the moves in practice. I like the ability to add my own algorithms to the system, however I don't think I would ever use it.

### **What do you dislike about the project?**

I don't like the speed that the white cross takes to be solved. It makes the whole solving process feel somewhat sluggish despite that one being the only slow stage. Sometimes I find I press the wrong move, but I am not certain which move I just entered. This sometimes results in me having to redo previous stages due to me messing up one move.

### **What improvements would you like to see?**

I would like for the white cross to be more responsive.

I think the ability to undo previous moves would greatly improve the experience in the playground.

I'd like to see the moves more visually being done to the cube, possibly using animations to show the face of the cube rotating.

### **Evaluation of feedback**

Clearly, my end-user is very happy with the standard of my application, however, I agree with his criticisms. If the cube were able to more clearly communicate moves, a user would have an easier time understanding algorithm concepts. An undo feature would greatly improve the user experience given that I have also had similar issues in my own testing during development. One way I can think to implement this would be to push moves to a stack, popping an element would return the move that needs to be undone. Whilst the white cross can be annoyingly slow at times, I am unsure at this time how it would be improved upon without drastically increasing the move count. It is possible that a better heuristic function could provide faster results.

## Final Evaluation

Overall, I'm really pleased with what I have been able to accomplish with this project. I thoroughly enjoyed researching new methods that I can incorporate into future applications. Designing my program before developing it is not something I have done before, but it is an incredibly helpful step and saved a lot of head scratching when it became time to code this program. It allowed me to think through how each component of the program will work without needing to know exactly how to execute it in `c#` resulting in a much faster overall development time.

If I had more time or I ever came back to this project I would love to learn about 3D graphics and real time animation. Being able to manipulate the cube using cursor inputs would make the system feel even more intuitive. Face rotation animation would make what is happening obvious to the user, in its current state it sometimes feels unintuitive with how the cube is moving.

Furthermore, with more education, I would love to apply the field of robotics to this project. If I were able to create a cube that can detect rotations made by a human and translate that into moves on the software, the learning process could be vastly improved. The user could practice algorithms on a real cube and gain muscle memory rather than their ability to recall the moves of their head.